PROYECTO DE EXAMEN PARCIAL II

Ingeniería de Requisitos

NovaSol Salud

Sistema Web de Gestión de Citas Médicas

Caso de estudio: Clínica Vida Salud

GRUPO # 20

Integrantes	Registro
Sergio Estaca Paco	221089421
Frank Miguel Yerbales Molina	218164629

Docente: Ing. Angelica Garzon Cuellar

Materia: Ingeniería de Requisitos

Octubre de 2026

Repositorio GitHub
github.com/Kato-001/novasol-salud

Sistema en línea
novasol-salud.vercel.app

TABLA DE CONTENIDO

1. PERFIL....4
1.1 Introducción....4
1.2 Objetivos....4
Objetivo general....4
Objetivos específicos....4
1.3 Descripción del problema....5
1.4 Alcance....5
2. MARCO TEÓRICO....7
2.1 Sistemas de información web....7
2.2 Ingeniería de requisitos....7
2.3 Proceso Unificado de Desarrollo de Software (PUDS)....7
2.4 Lenguaje Unificado de Modelado (UML)....7
2.5 PHP....7
2.6 PostgreSQL....8
2.7 PDO y consultas parametrizadas....8
2.8 Arquitectura Modelo-Vista-Controlador (MVC)....8
2.9 HTML5, CSS3 y JavaScript....8
2.10 Seguridad en aplicaciones web....8
2.11 Computación en la nube: Vercel y Supabase....8
2.12 Control de versiones: Git y GitHub....9
3. MODELO DE NEGOCIO....10
3.1 Descripción de la organización....10
3.2 Diagrama de casos de uso del negocio....11
3.3 Proceso actual....11
3.4 Proceso propuesto....12
4. FT CAPTURA DE REQUISITOS....14
4.1 Encontrar actores y casos de uso....14
4.1.1 Actores del sistema....14
4.1.2 Lista de casos de uso....14
4.1.3 Diagrama general de casos de uso....16
4.1.4 Diagramas de casos de uso por paquete....17
4.2 Priorizar casos de uso....20
4.3 Detallar casos de uso....21
CU1 Iniciar sesión....21
CU2 Gestionar roles y permisos....22
CU3 Gestionar usuarios....23
CU4 Gestionar especialidades....23
CU5 Gestionar médicos....24
CU6 Gestionar horarios....25
CU7 Gestionar pacientes....25
CU8 Registrarse como paciente....26
CU9 Consultar disponibilidad....26
CU10 Reservar ficha....27
CU11 Reprogramar cita....28
CU12 Cancelar cita....28
CU13 Registrar asistencia....29

CU14 Consultar agenda médica....29
CU15 Registrar consulta médica....30
CU16 Gestionar historial clínico....31
CU17 Registrar tratamiento....31
CU18 Gestionar estudios médicos....32
CU19 Consultar historial y resultados....32
CU20 Generar reportes....33
4.4 Prototipar la interfaz de usuario....33
4.5 Estructurar el modelo de casos de uso....35
4.6 Requerimientos y reglas de negocio....36
5. FT ANÁLISIS....39
5.1 Análisis de la arquitectura....39
5.2 Analizar casos de uso....39
5.3 Análisis de clases....46
5.4 Analizar paquetes....48
6. FT DISEÑO....50
6.1 Diseño de la arquitectura....50
6.1.1 Diseño físico de la arquitectura · Diagrama de despliegue....50
6.1.2 Diseño lógico de la arquitectura · Diagrama de paquetes....51
6.2 Diseñar casos de uso · Diagramas de secuencia....52
6.3 Diseño de datos....64
6.3.1 Diseño lógico....64
6.3.2 Diseño físico....65
6.4 Trazabilidad....67
7. FT IMPLEMENTACIÓN....69
7.1 Herramientas de desarrollo de la aplicación web....69
7.2 Implementación de la arquitectura del sistema....69
7.2.1 Flujo de una petición....69
7.2.2 Seguridad implementada....70
7.3 Implementación de la arquitectura de los subsistemas....71
P1 · Seguridad y acceso....71
P2 · Gestión médica....80
P3 · Gestión de pacientes....83
P4 · Gestión de citas....87
P5 · Atención médica....94
P6 · Reportes....99
Portal del paciente....101
Diseño adaptable y modo oscuro....106
8. FT PRUEBAS....110
8.1 Estrategia de pruebas....110
8.2 Pruebas mínimas requeridas....110
8.3 Pruebas de integridad en la base de datos....112
8.4 Pruebas de seguridad y permisos....113
8.5 Pruebas del flujo completo....113
CONCLUSIONES....114
RECOMENDACIONES....115
BIBLIOGRAFÍA....116
ANEXOS....117

Anexo A · Repositorio y sistema en línea.....117
Anexo B · Cuentas de acceso.....117
Anexo C · Instalación local.....118
Anexo D · Manual breve de uso.....118

1. PERFIL

1.1 Introducción

La Clínica Vida Salud es un establecimiento médico (ficticio) que atiende las especialidades de medicina general, pediatría, ginecología, odontología, cardiología y dermatología. Hasta ahora administra sus citas con cuadernos, llamadas telefónicas y hojas de cálculo, lo que provoca citas duplicadas, pérdida de datos de los pacientes y falta de información para la administración.

El presente documento describe el análisis, diseño, implementación y pruebas de **NovaSol Salud**, un sistema web de gestión de citas médicas desarrollado con **PHP 8** y **PostgreSQL**, siguiendo el **Proceso Unificado de Desarrollo de Software (PUDS)** con notación **UML**. El sistema centraliza la información de pacientes, médicos, especialidades y horarios; programa citas mostrando únicamente los horarios realmente disponibles; controla los estados de las citas; y genera reportes administrativos. Además, incluye un portal para que los pacientes reserven y sigan sus propias citas.

El sistema se encuentra publicado en **https://novasol-salud.vercel.app** y su código fuente en **https://github.com/Kato-001/novasol-salud**.

1.2 Objetivos

Objetivo general

Desarrollar un sistema web de gestión de citas médicas utilizando PHP como lenguaje del lado del servidor y PostgreSQL como sistema gestor de base de datos, con el propósito de mejorar el registro, la programación, el seguimiento y el control de las citas médicas de la Clínica Vida Salud.

Objetivos específicos

- Analizar los procesos de registro y programación de citas de la clínica.
- Diseñar una base de datos relacional normalizada con claves primarias y foráneas.
- Implementar formularios para el registro, consulta, modificación y desactivación de pacientes y médicos.
- Gestionar las especialidades médicas y los horarios de atención.
- Programar citas verificando la disponibilidad del médico en el servidor.
- Controlar los estados de las citas médicas y sus transiciones.
- Generar reportes de citas por fecha, médico, especialidad y estado.
- Aplicar validaciones de datos y mecanismos de seguridad.
- Realizar pruebas para verificar el correcto funcionamiento del sistema.

1.3 Descripción del problema

La recepción de la clínica registra a los pacientes, consulta la disponibilidad de los médicos, programa las citas y comunica la fecha y hora de atención. Al hacerlo de forma manual se presentan los siguientes problemas:

- **Duplicación de citas** para un mismo médico y horario.
- Pérdida o registro incorrecto de los datos de los pacientes.
- Dificultad para conocer los horarios disponibles de los médicos.
- Falta de control sobre las citas canceladas y las inasistencias.
- Demoras en la búsqueda del historial de citas de un paciente.
- Dificultad para obtener reportes de las atenciones programadas.
- Falta de información actualizada para la administración de la clínica.

Planteamiento del problema: ¿Cómo desarrollar un sistema web utilizando PHP y PostgreSQL que permita administrar pacientes, médicos, especialidades y citas médicas, evitando conflictos de horarios y facilitando el control de las atenciones de la Clínica Vida Salud?

1.4 Alcance

El sistema es una aplicación web accesible desde cualquier navegador (computadora o celular), desarrollada con PHP, PostgreSQL, HTML5, CSS3 y JavaScript. Está destinado principalmente al **personal de la clínica** (administrador, recepcionistas y médicos) y ofrece además un **portal para pacientes**, ambos con el mismo inicio de sesión y un menú distinto según el rol.

Tabla 1. *Alcance del sistema por módulo*

Módulo	Alcance
P1 Seguridad y acceso	Inicio y cierre de sesión, roles y permisos, usuarios, configuración y bitácora de auditoría.
P2 Gestión médica	Especialidades, médicos (con una o varias especialidades) y horarios de atención con bloqueos de agenda.
P3 Gestión de pacientes	Registro, búsqueda, modificación y desactivación; historia clínica y contactos de emergencia; autoregistro.
P4 Gestión de citas	Disponibilidad, reserva, reprogramación, cancelación, asistencia y agenda médica, sin citas superpuestas.
P5 Atención médica	Consulta médica, tratamientos, estudios médicos con resultados e historial clínico.
P6 Reportes	Reportes por fecha, médico, especialidad, estado, pacientes, resumen del período y agenda diaria; CSV y PDF.
Portal del paciente	Reserva y cancelación de citas, perfil, información médica, recordatorios y resultados.

Fuera del alcance: facturación y cobros, envío de recordatorios por correo o WhatsApp (los recordatorios se muestran dentro del sistema; el único correo que se envía es el código para recuperar la contraseña), videollamadas para citas virtuales y la integración con equipos de laboratorio.

Distribución por ciclos del Proceso Unificado:

Tabla 2. *Distribución de casos de uso por ciclo*

Ciclo	Paquetes	Casos de uso
Ciclo 1	P1 Seguridad y acceso · P2 Gestión médica · P3 Gestión de pacientes	CU1 a CU7 (base del sistema: seguridad y datos maestros)
Ciclo 2	P4 Gestión de citas · P5 Atención médica · P6 Reportes · Portal	CU8 a CU20 (núcleo del negocio: citas, atención y reportes)

2. MARCO TEÓRICO

2.1 Sistemas de información web

Un sistema de información web es una aplicación que se ejecuta en un servidor y se usa desde un navegador mediante el protocolo HTTP. Permite el acceso desde distintos dispositivos sin instalar software en cada equipo, centraliza la información en una base de datos y facilita la actualización del sistema, porque los cambios se aplican en un solo lugar.

2.2 Ingeniería de requisitos

La ingeniería de requisitos es el proceso de descubrir, analizar, documentar y validar los servicios que el sistema debe ofrecer y sus restricciones (Sommerville, 2011). Distingue los **requisitos funcionales**, que describen lo que el sistema debe hacer, de los **no funcionales**, que describen cualidades como la seguridad, el rendimiento o la usabilidad. Las **reglas de negocio** expresan políticas que el sistema debe cumplir siempre. En este proyecto, los requisitos se modelan con **casos de uso**, que describen la interacción entre un actor y el sistema para lograr un objetivo.

2.3 Proceso Unificado de Desarrollo de Software (PUDS)

El Proceso Unificado (Jacobson, Booch y Rumbaugh, 2000) es un proceso de desarrollo **dirigido por casos de uso, centrado en la arquitectura, iterativo e incremental**. Se organiza en flujos de trabajo (captura de requisitos, análisis, diseño, implementación y pruebas) que se repiten en cada ciclo. Cada flujo produce modelos relacionados entre sí por **trazabilidad**: el modelo de casos de uso se refina en el modelo de análisis, este en el de diseño, y el diseño se realiza en el modelo de despliegue y en la implementación.

2.4 Lenguaje Unificado de Modelado (UML)

UML es un lenguaje gráfico estándar para especificar, visualizar y documentar sistemas de software. En este proyecto se utilizan los diagramas de casos de uso, actividades (modelo de negocio), comunicación (análisis), clases, estados, secuencia, paquetes, componentes y despliegue. Las clases de análisis se clasifican en **interfaz «boundary»**, **control «control»** y **entidad «entity»**.

2.5 PHP

PHP (Hypertext Preprocessor) es un lenguaje de programación interpretado del lado del servidor, diseñado para el desarrollo web. Su versión 8 incorpora tipado estricto, expresiones match, propiedades de solo lectura y mejoras de rendimiento. Ofrece funciones nativas de seguridad, como `password_hash()` y `password_verify()`, que cifran las contraseñas con el algoritmo bcrypt y una sal aleatoria.

2.6 PostgreSQL

PostgreSQL es un sistema gestor de bases de datos relacional y objeto-relacional de código abierto. Garantiza las propiedades ACID de las transacciones y ofrece restricciones avanzadas: claves primarias y foráneas, `UNIQUE`, `CHECK`, restricciones de exclusión (`EXCLUDE USING gist`) que impiden rangos superpuestos, disparadores (triggers) en PL/pgSQL y seguridad a nivel de fila (RLS).

2.7 PDO y consultas parametrizadas

PDO (PHP Data Objects) es una capa de acceso a datos uniforme para PHP. Con el controlador `pdo_pgsql` se conecta a PostgreSQL. Las **consultas parametrizadas** separan el código SQL de los datos del usuario: los valores se envían como parámetros (por ejemplo `:ci`) y nunca se concatenan al texto SQL, lo que previene la **inyección SQL**.

2.8 Arquitectura Modelo-Vista-Controlador (MVC)

MVC separa la aplicación en tres responsabilidades: el **modelo** accede a los datos y aplica las reglas de negocio, la **vista** presenta la información, y el **controlador** recibe las peticiones, coordina al modelo y elige la vista. Esta separación facilita el mantenimiento y las pruebas. NovaSol Salud agrega una capa de **servicios** para la lógica de citas y un **controlador frontal** que recibe todas las peticiones.

2.9 HTML5, CSS3 y JavaScript

HTML5 estructura el contenido de las páginas e incluye validaciones nativas de formularios. CSS3 define la presentación mediante variables, grillas, animaciones y consultas de medios (diseño adaptable a celulares y modo oscuro). JavaScript agrega interactividad en el navegador: validación inmediata de formularios, búsqueda de pacientes y consulta de horarios disponibles sin recargar la página.

2.10 Seguridad en aplicaciones web

El proyecto aplica las recomendaciones de OWASP: contraseñas cifradas con hash, **consultas parametrizadas** contra la inyección SQL, escape de la salida HTML contra **XSS**, tokens **CSRF** en los formularios, cookies de sesión `HttpOnly` y `SameSite`, regeneración del identificador de sesión al ingresar, control de acceso por permisos en el servidor, limitación de intentos de ingreso, bitácora de auditoría y manejo de errores sin mostrar detalles internos.

2.11 Computación en la nube: Vercel y Supabase

Vercel es una plataforma de despliegue que ejecuta el código como **funciones serverless**: cada petición se atiende en una instancia que se crea bajo demanda. Gracias al entorno comunitario `vercel-php`, ejecuta PHP 8.3. **Supabase** ofrece bases de datos PostgreSQL administradas, con un *pooler* de conexiones (Supavisor) adecuado para entornos serverless. Ambos servicios se ubicaron en São Paulo para reducir la latencia desde Bolivia.

2.12 Control de versiones: Git y GitHub

Git es un sistema de control de versiones distribuido que registra la historia de los cambios del código. GitHub aloja los repositorios en la nube y permite compartir el código fuente; el código de NovaSol Salud se publica allí junto con su documentación.

3. MODELO DE NEGOCIO

3.1 Descripción de la organización

La Clínica Vida Salud brinda atención ambulatoria en varias especialidades. Su recepción atiende a los pacientes en persona o por teléfono, los médicos atienden en horarios semanales fijos y la administración necesita información sobre las atenciones para tomar decisiones.

Tabla 3. *Actores y trabajadores del negocio*

Actor / trabajador del negocio	Responsabilidad
Paciente	Solicita citas, asiste a la consulta y recibe tratamientos y estudios.
Recepcionista	Registra pacientes, consulta la disponibilidad, programa, reprograma y cancela citas y registra la asistencia.
Médico	Atiende a los pacientes según su agenda, registra la consulta e indica tratamientos y estudios.
Administración de la clínica	Gestiona médicos, especialidades y horarios, y consulta los reportes generales.

3.2 Diagrama de casos de uso del negocio

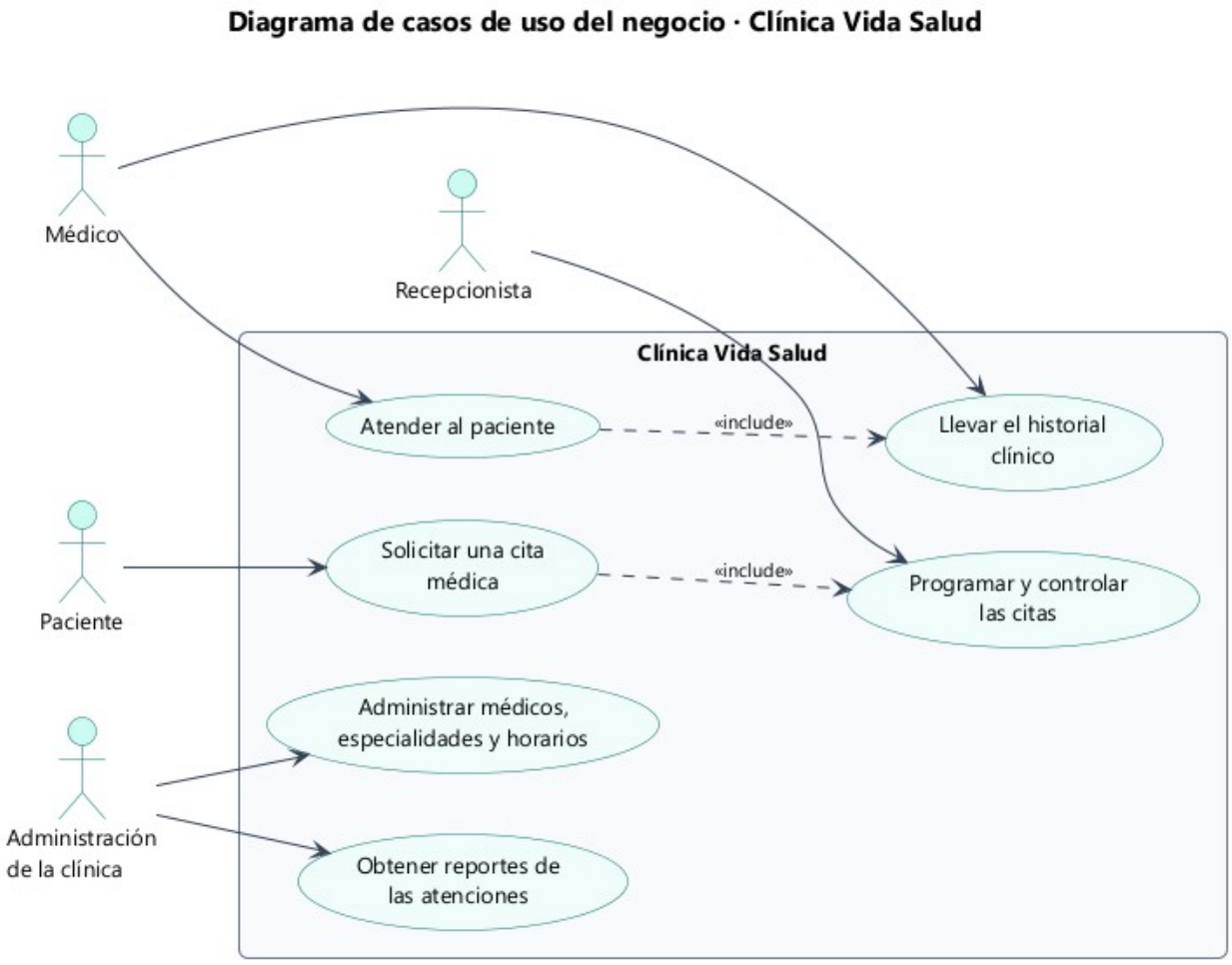


Figura 1. *Diagrama de casos de uso del negocio*

3.3 Proceso actual

El siguiente diagrama de actividades muestra cómo se programan hoy las citas. Los puntos críticos son la búsqueda manual de horarios, el riesgo de citas duplicadas y la falta de control sobre cancelaciones, inasistencias y reportes.

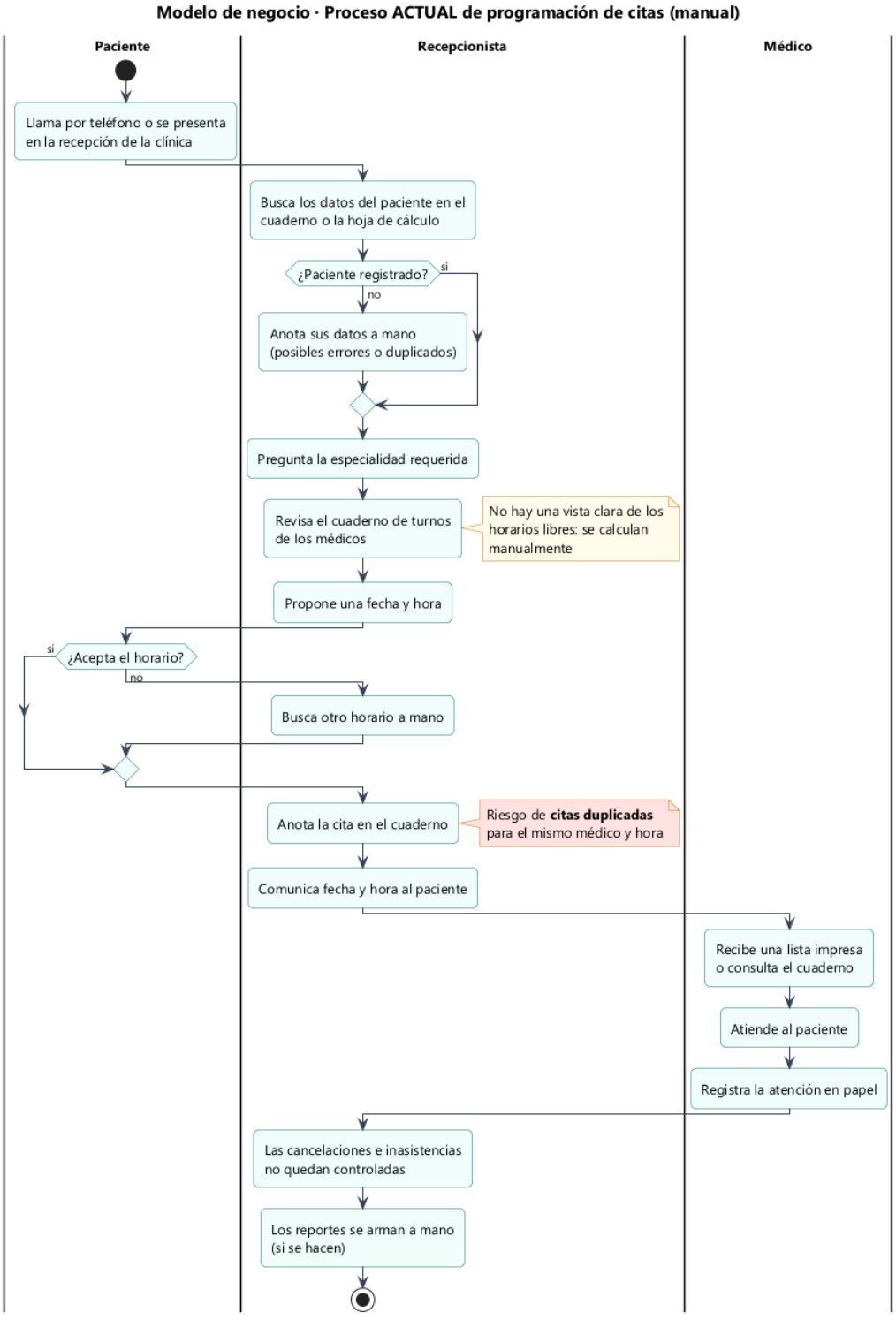


Figura 2. *Diagrama de actividades del proceso actual (manual)*

3.4 Proceso propuesto

Con NovaSol Salud, el sistema calcula la disponibilidad, muestra solo los horarios libres, impide los cruces, registra el historial de cada cita y genera los reportes automáticamente.

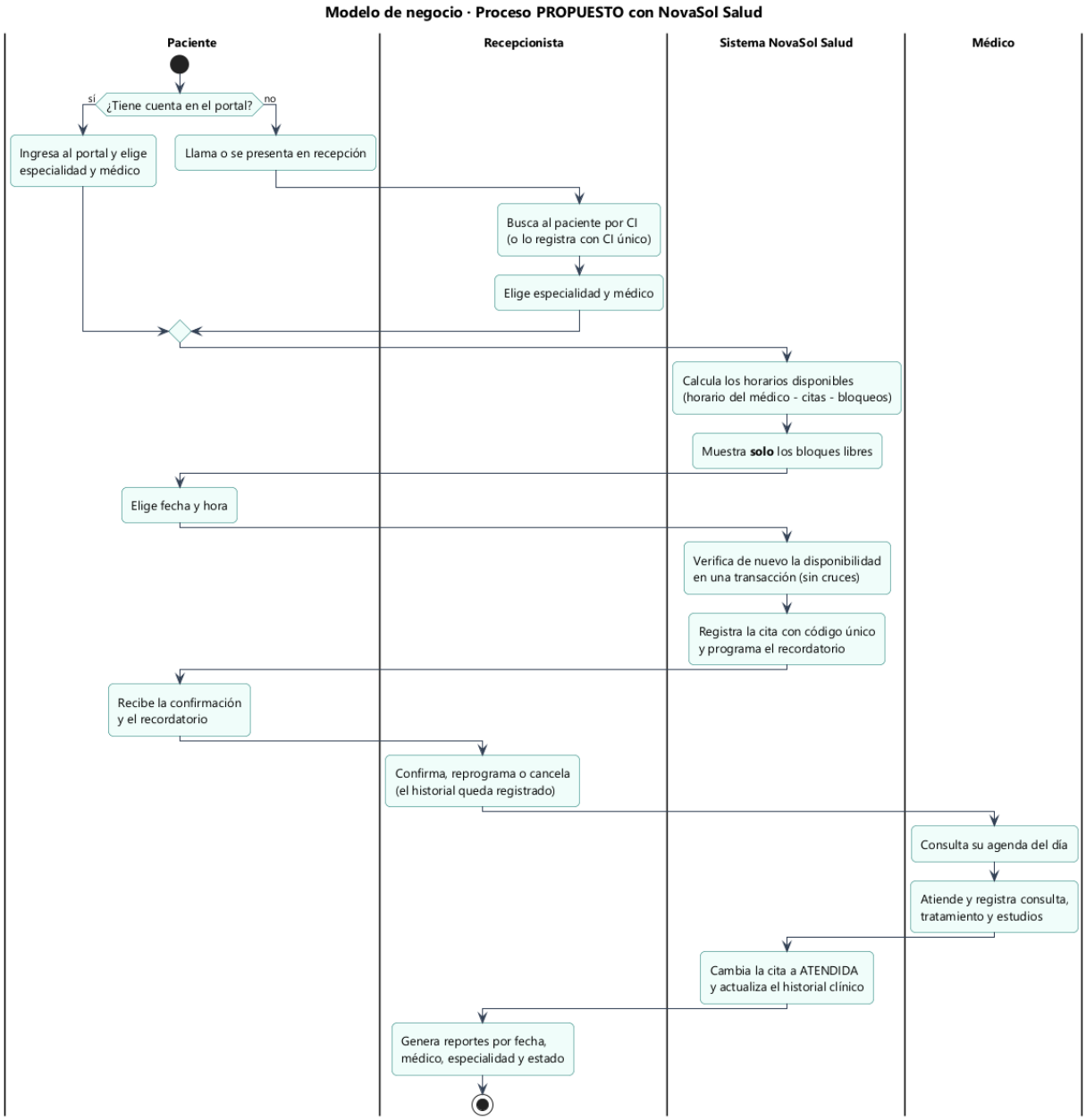


Figura 3. *Diagrama de actividades del proceso propuesto con NovaSol Salud*

4. FT CAPTURA DE REQUISITOS

4.1 Encontrar actores y casos de uso

4.1.1 Actores del sistema

Tabla 4. *Actores del sistema*

Actor	Descripción	Permisos principales
Usuario	Actor abstracto: cualquier persona con cuenta. Generaliza a los demás actores.	Iniciar y cerrar sesión, cambiar su contraseña.
Administrador	Configura y supervisa el sistema. Hereda los casos de uso del recepcionista.	Usuarios, roles y permisos, configuración, bitácora, médicos, especialidades, horarios, catálogo de estudios, reportes.
Recepcionista	Personal que gestiona a los pacientes y las citas.	Pacientes, programar, reprogramar y cancelar citas, asistencia, agenda de todos los médicos, estudios, reportes.
Médico	Profesional que atiende a los pacientes.	Su agenda, atender citas, registrar consulta, tratamiento y estudios, historial clínico.
Paciente	Persona que se atiende en la clínica y usa el portal.	Registrarse, reservar y cancelar sus citas, perfil, historial y resultados.

4.1.2 Lista de casos de uso

Tabla 5. *Casos de uso del sistema*

CU	Nombre	Paquete	Actores	Ciclo
CU1	Iniciar sesión	P1 Seguridad y acceso	Administrador, Recepcionista, Médico, Paciente (Usuario)	Ciclo 1
CU2	Gestionar roles y permisos	P1 Seguridad y acceso	Administrador	Ciclo 1
CU3	Gestionar usuarios	P1 Seguridad y acceso	Administrador	Ciclo 1
CU4	Gestionar especialidades	P2 Gestión médica	Administrador	Ciclo 1
CU5	Gestionar médicos	P2 Gestión médica	Administrador (registra), Recepcionista (consulta)	Ciclo 1
CU6	Gestionar horarios	P2 Gestión médica	Administrador	Ciclo 1
CU7	Gestionar pacientes	P3 Gestión de pacientes	Recepcionista, Administrador	Ciclo 1
CU8	Registrarse como paciente	P3 Gestión de pacientes	Paciente	Ciclo 2
CU9	Consultar disponibilidad	P4 Gestión de citas	Recepcionista, Paciente	Ciclo 2
CU10	Reservar ficha	P4 Gestión de citas	Recepcionista, Administrador, Paciente	Ciclo 2

CU	Nombre	Paquete	Actores	Ciclo
			(portal)	
CU11	Reprogramar cita	P4 Gestión de citas	Recepcionista	Ciclo 2
CU12	Cancelar cita	P4 Gestión de citas	Recepcionista, Paciente (portal)	Ciclo 2
CU13	Registrar asistencia	P4 Gestión de citas	Recepcionista, Médico	Ciclo 2
CU14	Consultar agenda médica	P4 Gestión de citas	Médico, Recepcionista, Administrador	Ciclo 2
CU15	Registrar consulta médica	P5 Atención médica	Médico	Ciclo 2
CU16	Gestionar historial clínico	P5 Atención médica	Médico, Recepcionista, Paciente	Ciclo 2
CU17	Registrar tratamiento	P5 Atención médica	Médico	Ciclo 2
CU18	Gestionar estudios médicos	P5 Atención médica	Médico (solicita), Recepcionista (registra resultado), Administrador (catálogo)	Ciclo 2
CU19	Consultar historial y resultados	P5 Atención médica	Paciente	Ciclo 2
CU20	Generar reportes	P6 Reportes	Administrador, Recepcionista	Ciclo 2

4.1.3 Diagrama general de casos de uso

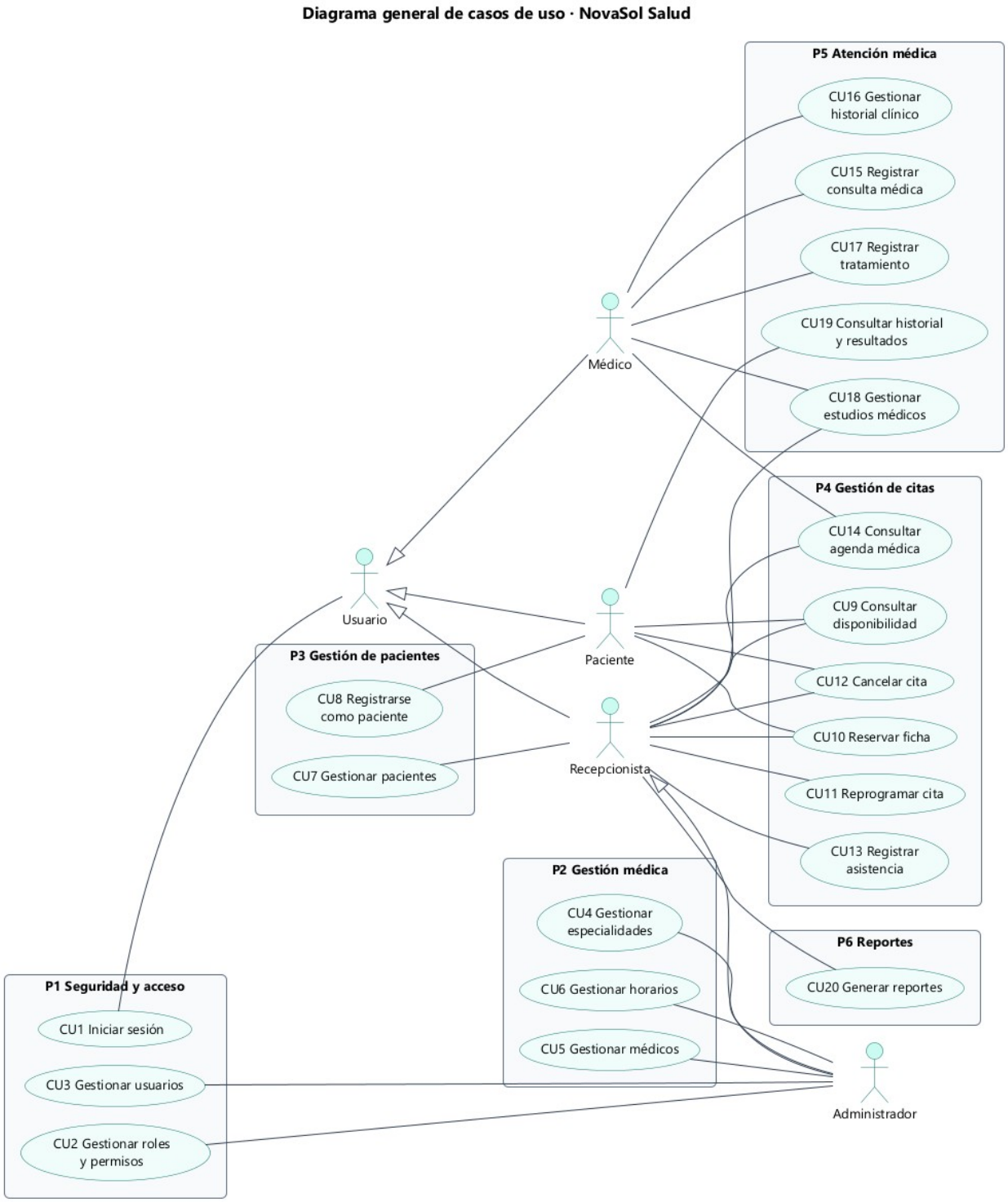


Figura 4. *Diagrama general de casos de uso*

4.1.4 Diagramas de casos de uso por paquete

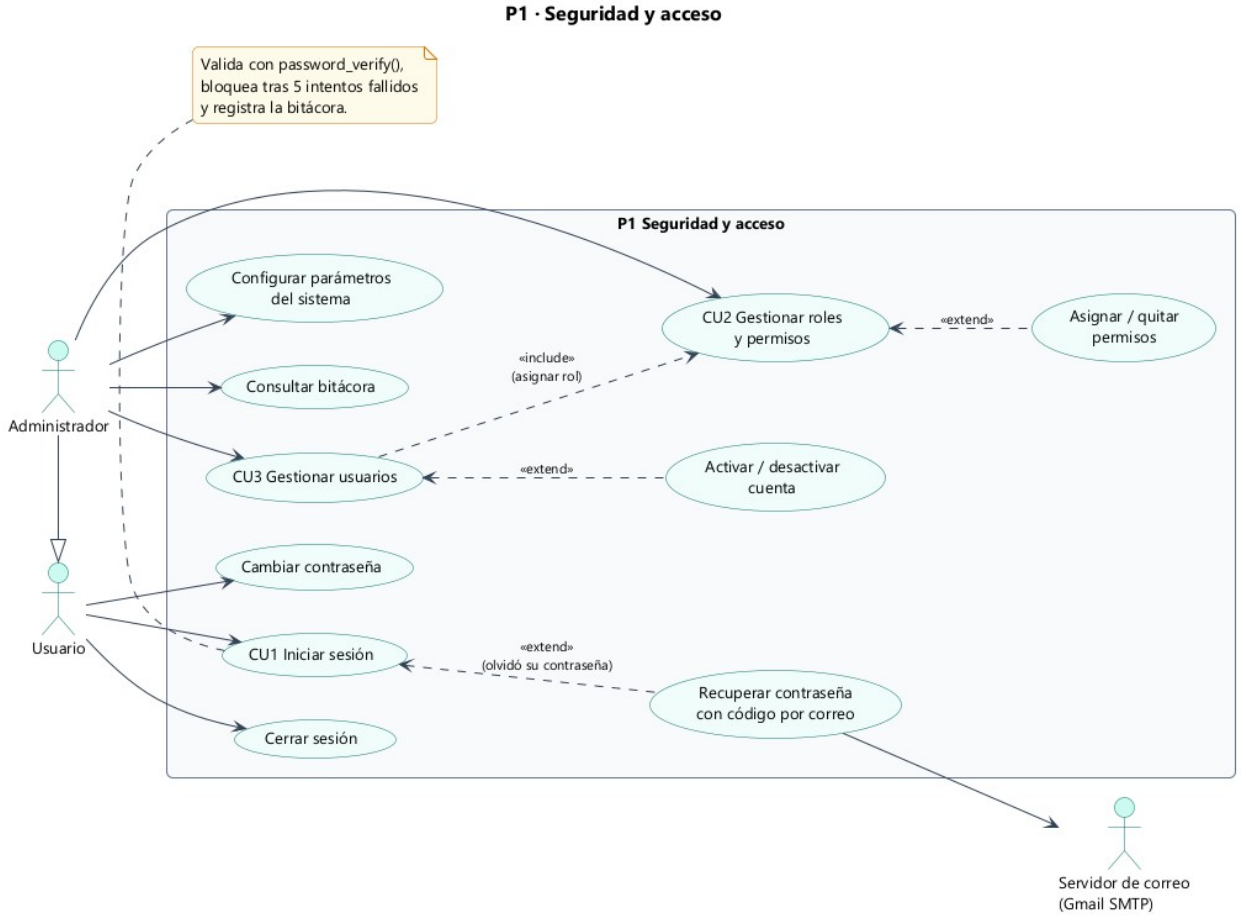


Figura 5. *Diagrama de casos de uso del paquete P1 Seguridad y acceso*

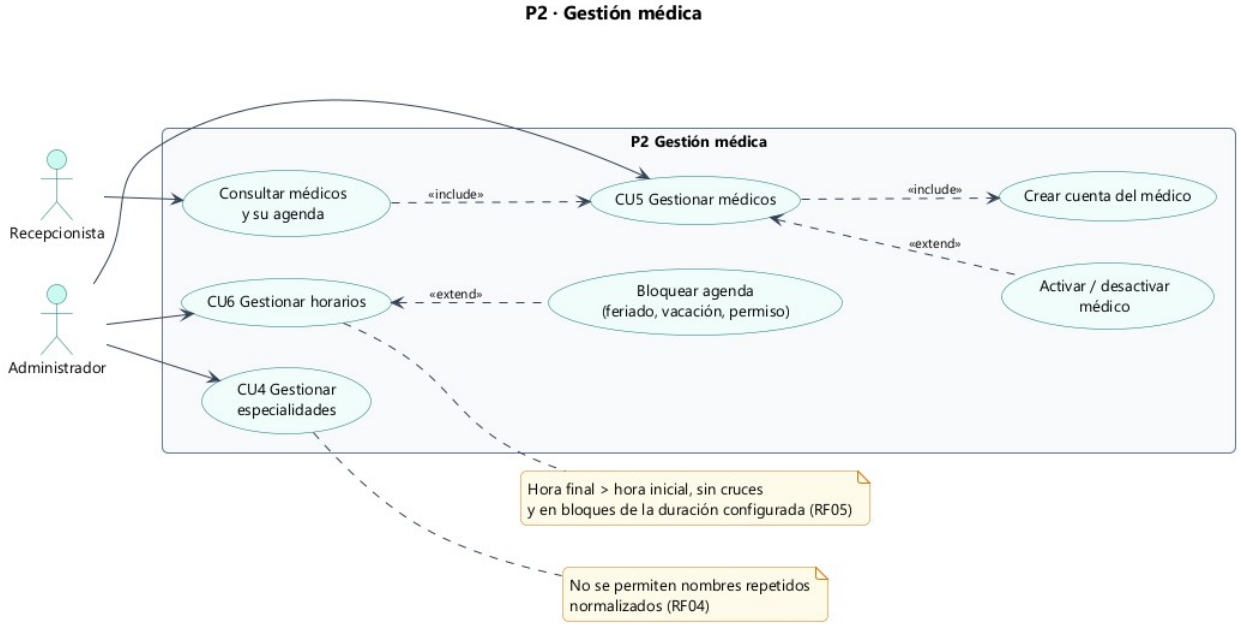


Figura 6. *Diagrama de casos de uso del paquete P2 Gestión médica*

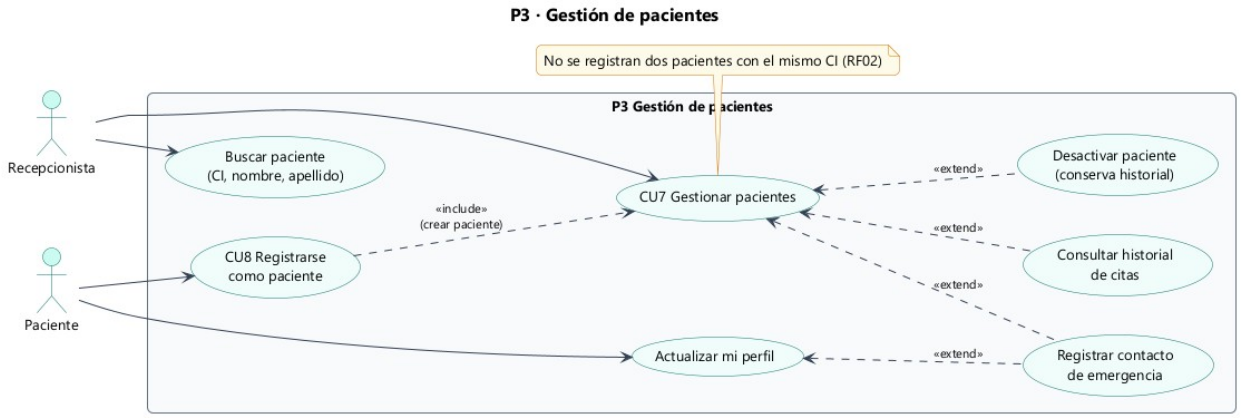


Figura 7. *Diagrama de casos de uso del paquete P3 Gestión de pacientes*

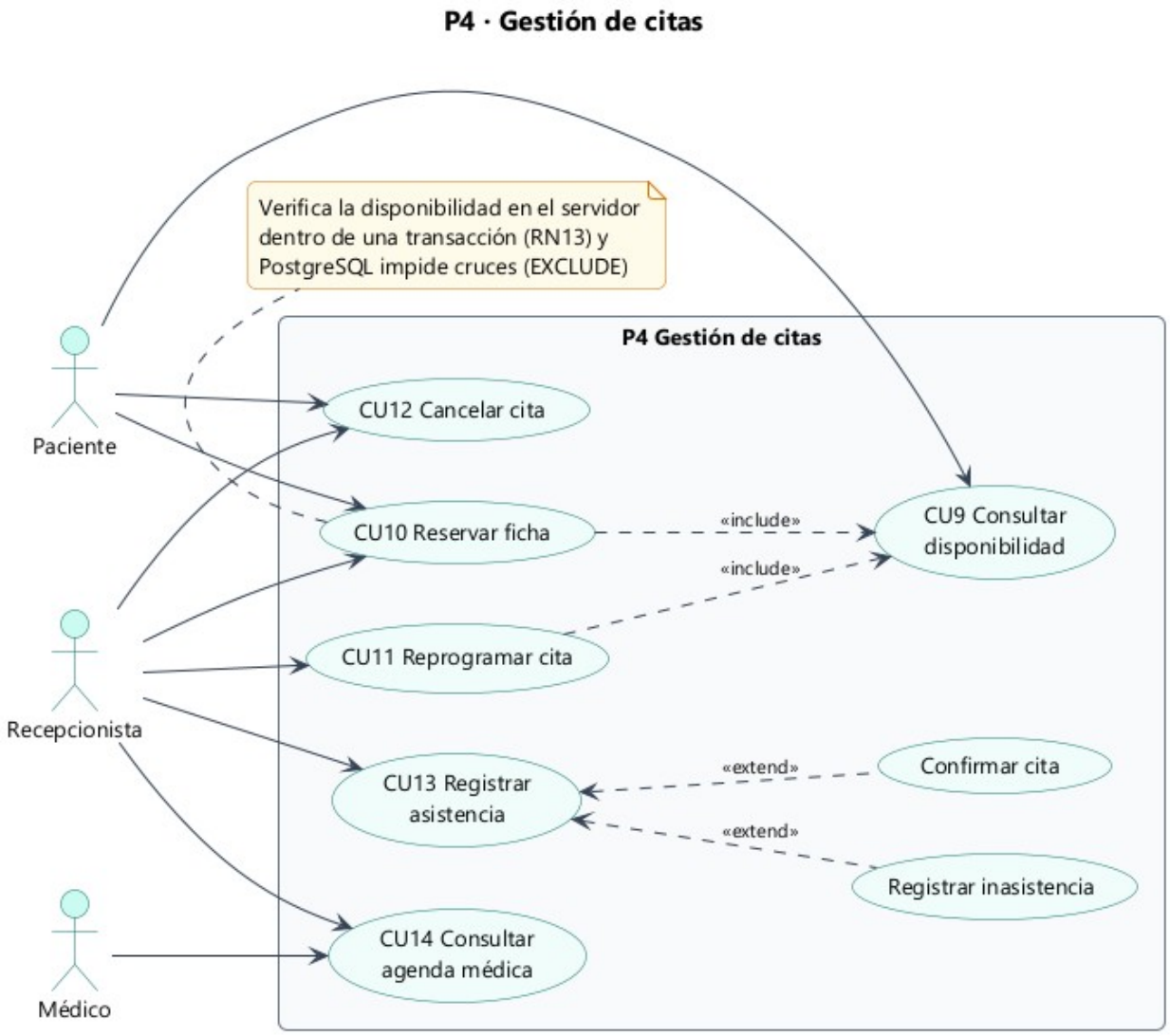


Figura 8. *Diagrama de casos de uso del paquete P4 Gestión de citas*

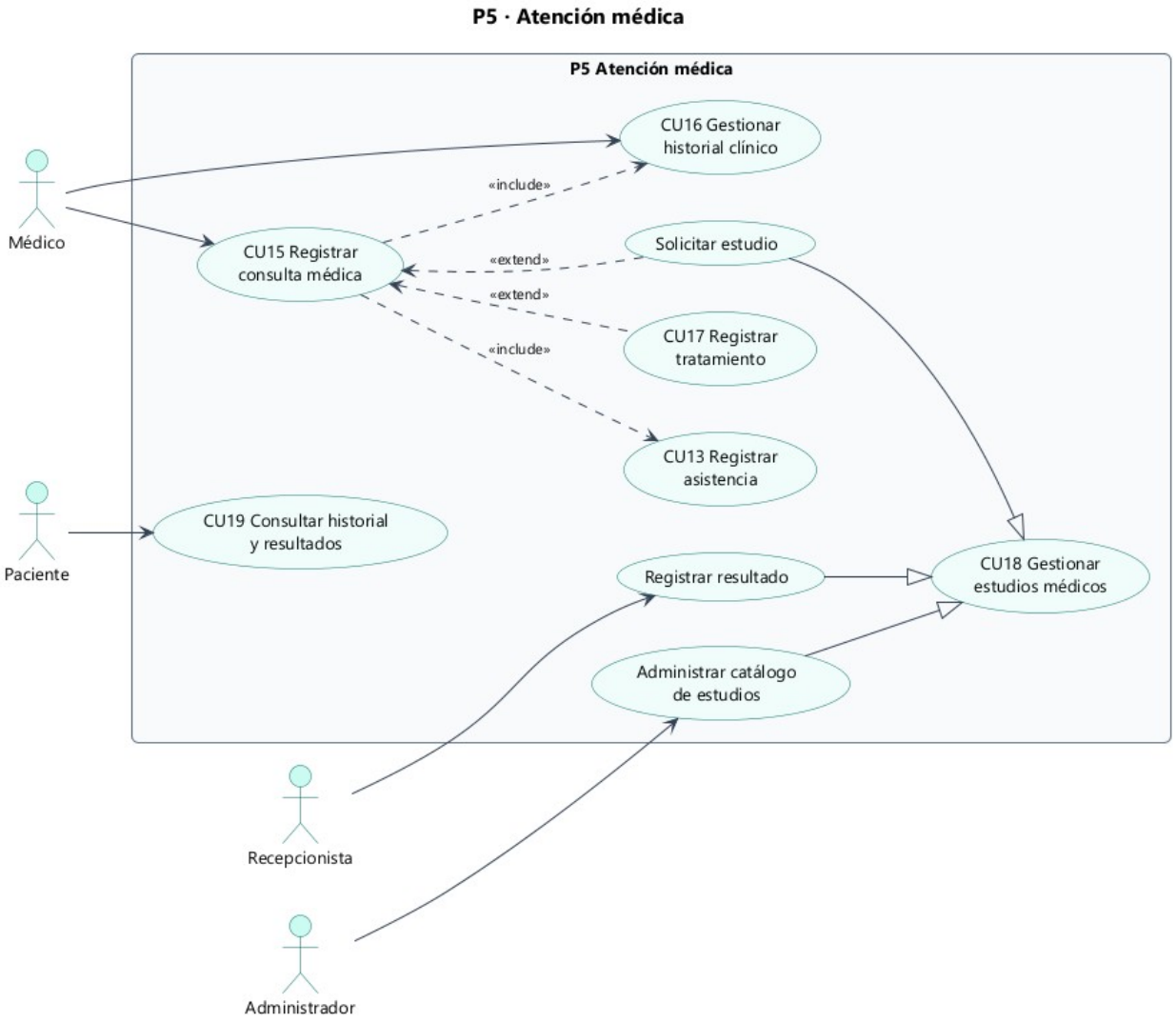


Figura 9. *Diagrama de casos de uso del paquete P5 Atención médica*

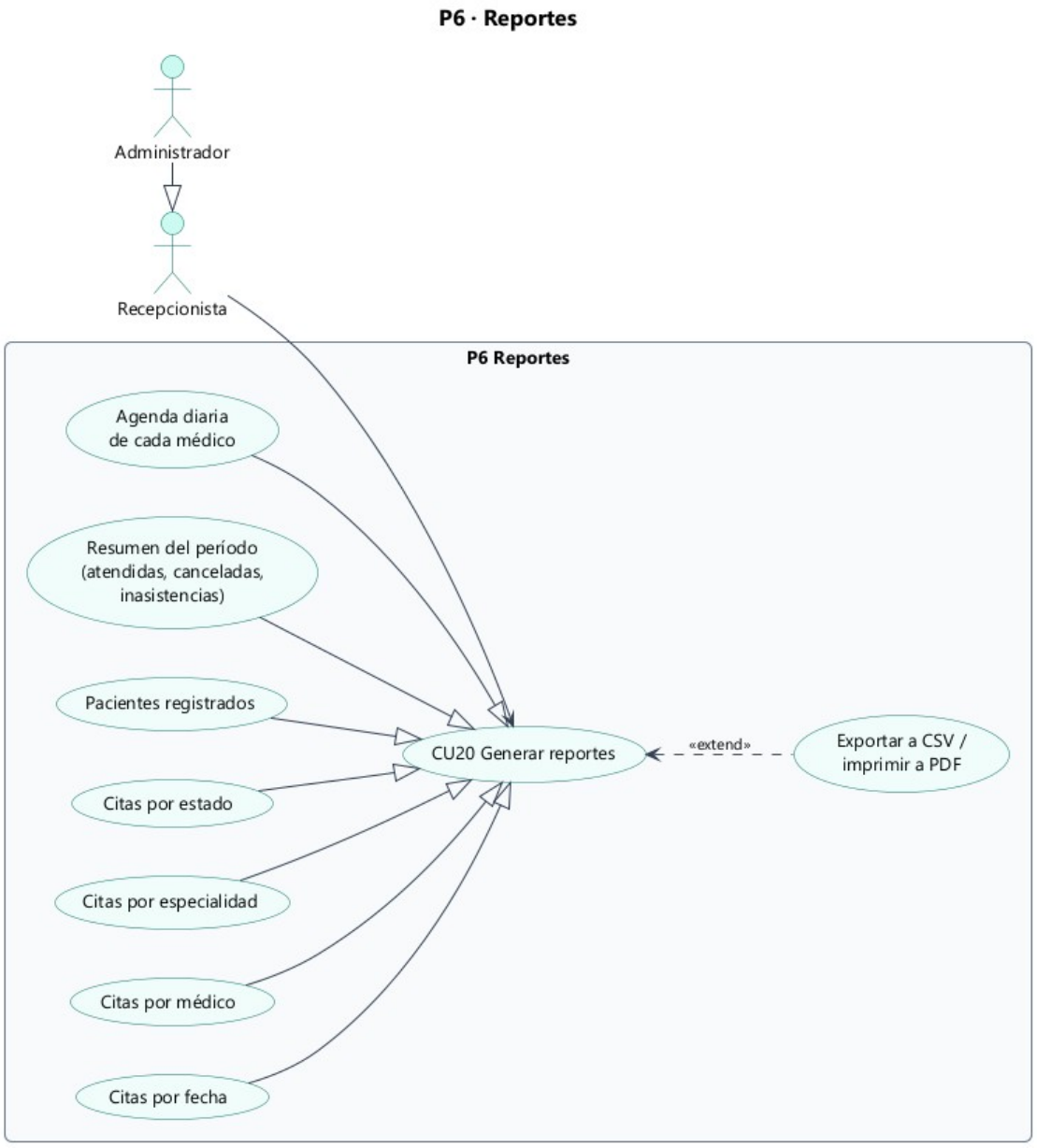


Figura 10. *Diagrama de casos de uso del paquete P6 Reportes*

4.2 Priorizar casos de uso

Se priorizan primero los casos de uso **arquitectónicamente significativos**, que obligan a definir la base de datos, la seguridad y la regla central del negocio (sin citas superpuestas), y después los que dependen de ellos.

Tabla 6. *Priorización de casos de uso*

Prioridad	Caso de uso	Justificación	Ciclo
1 · Crítica	CU1 Iniciar sesión	Toda operación exige sesión y permisos (RF01, RN12).	1

Prioridad	Caso de uso	Justificación	Ciclo
2 · Crítica	CU10 Reservar ficha	Módulo principal; concentra RN03 a RN09 y RN13.	2
3 · Alta	CU9 Consultar disponibilidad	CU10 y CU11 dependen de él.	2
4 · Alta	CU6 Gestionar horarios	Sin horarios no hay disponibilidad.	1
5 · Alta	CU7 Gestionar pacientes	Una cita necesita un paciente válido.	1
6 · Alta	CU5 Gestionar médicos	Una cita necesita un médico activo.	1
7 · Alta	CU11, CU12, CU13	Control de estados de la cita (RF08 a RF10).	2
8 · Alta	CU14 Agenda · CU20 Reportes	Consulta y control administrativo (RF11, RF13).	2
9 · Media	CU2, CU3, CU4	Administración del sistema.	1
10 · Media	CU15, CU16, CU17, CU18	Atención médica.	2
11 · Media	CU8, CU19	Portal del paciente (sección 3.4).	2

4.3 Detallar casos de uso

A continuación se presenta la descripción expandida de cada caso de uso, con su flujo principal, flujos alternativos y las reglas que cumple.

CU1 Iniciar sesión

Tabla 7. *Descripción del caso de uso CU1 Iniciar sesión*

Caso de uso	CU1 Iniciar sesión
Paquete · Ciclo	P1 Seguridad y acceso · Ciclo 1
Propósito	Permitir que un usuario autorizado ingrese al sistema con su usuario, correo o CI y contraseña, y acceda solo a las opciones de su rol.
Actores	Administrador, Recepcionista, Médico, Paciente (Usuario)
Actor iniciador	Usuario
Precondición	El usuario tiene una cuenta registrada y ACTIVA.
Flujo principal	1. El usuario abre la página de ingreso (/login). 2. Escribe su usuario, correo electrónico o CI, y su contraseña. 3. El sistema valida que los campos no estén vacíos (JavaScript y PHP) y verifica el token CSRF. 4. El sistema comprueba que la IP no tenga 5 o más intentos fallidos en los últimos 10 minutos. 5. El sistema busca la cuenta y verifica la contraseña con password_verify(). 6. El sistema regenera el identificador de sesión, actualiza el último acceso y registra el ingreso en la bitácora. 7. El sistema muestra el panel o el portal con el menú según los permisos del rol.

Flujos alternativos	A1. Credenciales incorrectas: se registra LOGIN_FALLIDO en la bitácora y se muestra "Usuario o contraseña incorrectos". A2. Cuenta inactiva: se muestra "Su cuenta está desactivada". A3. Demasiados intentos: se bloquea el ingreso desde esa IP durante 10 minutos. A4. Cerrar sesión: el usuario pulsa Salir, el sistema destruye la sesión y registra CERRAR_SESION. A5. Olvidó su contraseña: elige "¿Olvidaste tu contraseña?" y escribe su correo. El sistema genera un código de 6 dígitos (guardado con password_hash, vence en 15 minutos) y lo envía por correo (Gmail SMTP). El usuario escribe el código y su nueva contraseña; el sistema la cambia, invalida el código y cierra sus sesiones abiertas. Siempre responde lo mismo exista o no el correo, admite 5 intentos por código y limita las solicitudes por cuenta e IP.
Postcondición	Existe una sesión válida con el rol y los permisos del usuario.
Requisitos / reglas	RF01, RN12

CU2 Gestionar roles y permisos

Tabla 8. *Descripción del caso de uso CU2 Gestionar roles y permisos*

Caso de uso	CU2 Gestionar roles y permisos
Paquete · Ciclo	P1 Seguridad y acceso · Ciclo 1
Propósito	Crear roles y asignar o quitar los permisos que determinan el menú y las operaciones de cada rol.
Actores	Administrador
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso roles.gestionar.
Flujo principal	1. El administrador abre Roles y permisos. 2. El sistema muestra los roles con su cantidad de usuarios y la matriz de permisos agrupada por módulo. 3. El administrador registra un nuevo rol (nombre en mayúsculas y descripción) o edita la descripción de uno existente. 4. El administrador marca o desmarca permisos en la columna de un rol y pulsa Guardar. 5. El sistema reemplaza los permisos del rol en una transacción y registra la operación en la bitácora.
Flujos alternativos	A1. Nombre repetido o con caracteres no permitidos: el sistema rechaza el registro. A2. Rol del sistema (ADMINISTRADOR, RECEPCIONISTA, MEDICO, PACIENTE): no se puede renombrar ni eliminar. A3. El ADMINISTRADOR conserva siempre los permisos de usuarios y roles, para no quedar sin acceso. A4. Un rol solo se elimina si no tiene usuarios.
Postcondición	Los permisos del rol quedan actualizados y se aplican en la siguiente petición de sus usuarios.
Requisitos / reglas	RN12

CU3 Gestionar usuarios

Tabla 9. *Descripción del caso de uso CU3 Gestionar usuarios*

Caso de uso	CU3 Gestionar usuarios
Paquete · Ciclo	P1 Seguridad y acceso · Ciclo 1
Propósito	Registrar, modificar, activar y desactivar las cuentas del personal y asignarles un rol. Incluye la configuración del sistema y la bitácora.
Actores	Administrador
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso usuarios.gestionar.
Flujo principal	1. El administrador abre Usuarios. El sistema lista las cuentas con búsqueda, filtro por rol y por estado, y paginación. 2. El administrador elige Nuevo usuario e ingresa nombre, usuario, correo, rol y contraseña con su confirmación. 3. El sistema valida los datos (formato, unicidad de usuario y correo, contraseña de mínimo 8 caracteres con letras y números). 4. El sistema guarda la cuenta con la contraseña cifrada mediante password_hash() y registra la bitácora. 5. Para desactivar, el administrador confirma en el diálogo. El sistema cambia el estado y cierra las sesiones abiertas de esa cuenta.
Flujos alternativos	A1. Usuario o correo ya existentes: se muestra el error en el campo. A2. El administrador no puede desactivarse ni cambiarse el rol a sí mismo. A3. Configuración: el administrador modifica parámetros (duración de cita, días máximos de reserva, etc.) con validación de rangos. A4. Bitácora: el administrador filtra los registros por texto, acción y rango de fechas.
Postcondición	La cuenta queda registrada o actualizada; una cuenta inactiva no puede iniciar sesión.
Requisitos / reglas	RF01, RN12

CU4 Gestionar especialidades

Tabla 10. *Descripción del caso de uso CU4 Gestionar especialidades*

Caso de uso	CU4 Gestionar especialidades
Paquete · Ciclo	P2 Gestión médica · Ciclo 1
Propósito	Administrar las especialidades que ofrece la clínica.
Actores	Administrador
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso especialidades.gestionar.

Flujo principal	1. El administrador abre Especialidades. El sistema lista cada especialidad con sus médicos activos, citas y estado. 2. El administrador ingresa nombre y descripción opcional. 3. El sistema compara el nombre normalizado (sin tildes, mayúsculas ni espacios repetidos) con los existentes. 4. El sistema guarda la especialidad y registra la bitácora.
Flujos alternativos	A1. Nombre repetido ("pediatria" frente a "Pediatría"): se rechaza. A2. Desactivar: la especialidad deja de ofrecerse al programar citas; las citas existentes se conservan.
Postcondición	La especialidad queda disponible (o no) para programar citas.
Requisitos / reglas	RF04

CU5 Gestionar médicos

Tabla 11. *Descripción del caso de uso CU5 Gestionar médicos*

Caso de uso	CU5 Gestionar médicos
Paquete · Ciclo	P2 Gestión médica · Ciclo 1
Propósito	Registrar y mantener la información de los médicos, sus especialidades y su cuenta de acceso.
Actores	Administrador (registra), Recepcionista (consulta)
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso medicos.gestionar (o medicos.ver para consultar). Existen especialidades.
Flujo principal	1. El administrador abre Médicos y elige Registrar médico. El sistema propone el siguiente código (MED-###). 2. Ingresa código, registro profesional, nombres, apellidos, teléfono, correo, una o varias especialidades, y usuario y contraseña de la cuenta. 3. El sistema valida formatos y unicidad (código, registro profesional, usuario, correo). 4. En una transacción, el sistema crea la cuenta (rol MEDICO), el médico y sus especialidades. 5. El sistema redirige a la definición de horarios del médico.
Flujos alternativos	A1. Buscar: por nombre, código o especialidad, y por estado. A2. Desactivar: el médico no recibe nuevas citas ni puede iniciar sesión; sus citas históricas se conservan. Si tiene citas pendientes, el sistema lo advierte. A3. Ver ficha: datos, horario semanal, bloqueos, estadísticas y agenda por fecha.
Postcondición	El médico queda registrado con su cuenta y sus especialidades.
Requisitos / reglas	RF03, RN02, RN04, RN11

CU6 Gestionar horarios

Tabla 12. *Descripción del caso de uso CU6 Gestionar horarios*

Caso de uso	CU6 Gestionar horarios
Paquete · Ciclo	P2 Gestión médica · Ciclo 1
Propósito	Definir los horarios de atención de cada médico y bloquear fechas (feriados, vacaciones, permisos).
Actores	Administrador
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso horarios.gestionar. El médico existe.
Flujo principal	1. El administrador abre Horarios y elige un médico. El sistema muestra su semana y los bloques de cita por día. 2. El administrador ingresa día, hora inicial y hora final. 3. El sistema valida que la hora final sea posterior a la inicial, que cubra al menos una cita y que no se cruce con otro horario activo del mismo día. 4. El sistema guarda el horario. 5. Opcionalmente, el administrador bloquea una fecha (todo el día o un rango de horas) con tipo y motivo.
Flujos alternativos	A1. Horario cruzado o inválido: se rechaza con un mensaje explicativo. A2. Bloqueo sobre citas existentes: se crea y se advierte cuántas citas debe reprogramar la recepción. A3. Un horario con citas pendientes no se elimina: solo se desactiva.
Postcondición	La disponibilidad del médico refleja los horarios y bloqueos vigentes.
Requisitos / reglas	RF05, RN05, RN08

CU7 Gestionar pacientes

Tabla 13. *Descripción del caso de uso CU7 Gestionar pacientes*

Caso de uso	CU7 Gestionar pacientes
Paquete · Ciclo	P3 Gestión de pacientes · Ciclo 1
Propósito	Registrar, buscar, modificar y desactivar pacientes y consultar su historial de citas.
Actores	Recepcionista, Administrador
Actor iniciador	Recepcionista
Precondición	Sesión iniciada con el permiso pacientes.gestionar (o pacientes.ver para consultar).
Flujo principal	1. La recepcionista busca al paciente por CI, nombre o apellido. 2. Si no existe, elige Registrar paciente e ingresa CI, nombres, apellidos, fecha de nacimiento, sexo, teléfono y, opcionalmente, correo, dirección y tipo de sangre. 3. El sistema valida los datos en el navegador y en el servidor, y verifica que el CI no esté registrado.

	4. En una transacción, el sistema crea el paciente (código PAC-####) y su historia clínica (HC-####). Opcionalmente crea su cuenta para el portal. 5. El sistema muestra la ficha del paciente con su historial de citas.
Flujos alternativos	A1. CI duplicado: "Ya existe un paciente registrado con ese CI". A2. Desactivar: el paciente no recibe nuevas citas y su historial se conserva. Si tiene citas pendientes, se advierte. A3. Contactos de emergencia: agregar o quitar nombre, parentesco y teléfono.
Postcondición	El paciente queda registrado y disponible para programar citas.
Requisitos / reglas	RF02, RF12, RN01, RN11

CU8 Registrarse como paciente

Tabla 14. *Descripción del caso de uso CU8 Registrarse como paciente*

Caso de uso	CU8 Registrarse como paciente
Paquete · Ciclo	P3 Gestión de pacientes · Ciclo 2
Propósito	Permitir que una persona cree su propia cuenta en el portal para reservar y seguir sus citas.
Actores	Paciente
Actor iniciador	Paciente
Precondición	La persona no tiene una cuenta registrada.
Flujo principal	1. El visitante elige "Regístrese aquí" en la página de ingreso. 2. Completa sus datos personales, de contacto y de la cuenta (usuario, correo y contraseña) y acepta el uso de sus datos. 3. El sistema valida los datos y la unicidad del CI, el usuario y el correo. 4. En una transacción, el sistema crea la cuenta (rol PACIENTE), el paciente y su historia clínica. 5. El sistema inicia la sesión y muestra el portal.
Flujos alternativos	A1. Datos inválidos o duplicados: el sistema muestra el error en cada campo.
Postcondición	El paciente tiene una cuenta y accede a su portal.
Requisitos / reglas	Sección 3.4 del enunciado, RN01

CU9 Consultar disponibilidad

Tabla 15. *Descripción del caso de uso CU9 Consultar disponibilidad*

Caso de uso	CU9 Consultar disponibilidad
Paquete · Ciclo	P4 Gestión de citas · Ciclo 2
Propósito	Mostrar los horarios realmente libres de un médico en una fecha.

Actores	Recepcionista, Paciente
Actor iniciador	Recepcionista o Paciente
Precondición	El médico está activo y tiene horarios de atención.
Flujo principal	1. El actor elige especialidad, médico y fecha. El sistema marca los días en que el médico atiende. 2. El sistema divide cada horario del día en bloques de la duración configurada (30 minutos). 3. Descarta los bloques pasados, los ocupados por citas que ocupan horario y los bloqueados en la agenda. 4. Muestra los bloques; solo los libres se pueden elegir.
Flujos alternativos	A1. Fecha pasada o mayor al máximo de días de reserva: se informa el motivo. A2. El médico no atiende ese día o tiene la agenda bloqueada: se informa. A3. Médico inactivo: no se ofrecen horarios.
Postcondición	El actor conoce los horarios libres.
Requisitos / reglas	RF05, RN04, RN05, RN06, RN08, RN09

CU10 Reservar ficha

Tabla 16. *Descripción del caso de uso CU10 Reservar ficha*

Caso de uso	CU10 Reservar ficha
Paquete · Ciclo	P4 Gestión de citas · Ciclo 2
Propósito	Registrar una cita para un paciente con un médico en un horario realmente disponible.
Actores	Recepcionista, Administrador, Paciente (portal)
Actor iniciador	Recepcionista
Precondición	Sesión iniciada con citas.gestionar (o portal.paciente). Existen especialidades, médicos activos y horarios.
Flujo principal	1. El actor elige Programar cita. El sistema muestra el asistente: paciente, especialidad y médico, fecha, hora. 2. El actor busca al paciente por CI o nombre; el sistema muestra solo pacientes activos. 3. El actor elige la especialidad; el sistema carga los médicos activos que la atienden. 4. El actor elige médico y fecha; el sistema ejecuta CU9 y muestra solo los bloques libres. 5. El actor elige la hora, la modalidad y el motivo opcional, y confirma. 6. El sistema, dentro de una transacción, bloquea al médico (SELECT … FOR UPDATE) y vuelve a verificar la disponibilidad. 7. El sistema registra la cita con código único CM-#### y fecha de registro, el historial (PROGRAMAR), el aviso y el recordatorio, y la bitácora. 8. El sistema muestra el detalle de la cita.
Flujos alternativos	A1. El paciente no existe: se registra primero (CU7). A2. Otro usuario tomó el mismo horario segundos antes: "El horario ya no está disponible: el médico tiene otra cita".

	A3. Datos manipulados (fecha pasada, hora fuera de horario, médico que no atiende la especialidad): el servidor rechaza la operación. A4. Como última defensa, PostgreSQL rechaza cualquier cruce con la restricción EXCLUDE.
Postcondición	La cita queda PROGRAMADA y el horario deja de estar disponible.
Requisitos / reglas	RF06, RF07, RN03–RN09, RN13

CU11 Reprogramar cita

Tabla 17. *Descripción del caso de uso CU11 Reprogramar cita*

Caso de uso	CU11 Reprogramar cita
Paquete · Ciclo	P4 Gestión de citas · Ciclo 2
Propósito	Cambiar la fecha y hora de una cita programada o confirmada.
Actores	Recepcionista
Actor iniciador	Recepcionista
Precondición	La cita está en estado PROGRAMADA o CONFIRMADA.
Flujo principal	1. La recepcionista abre el detalle de la cita y elige Reprogramar. 2. El sistema recupera la cita y muestra los días de atención del mismo médico. 3. La recepcionista elige la nueva fecha y hora libres (CU9, excluyendo la propia cita) y escribe el motivo. 4. En una transacción, el sistema verifica la disponibilidad, actualiza la cita (vuelve a PROGRAMADA) y registra el historial con la fecha anterior y la nueva. 5. El sistema reemplaza el recordatorio del paciente.
Flujos alternativos	A1. Cita atendida, cancelada o con inasistencia: no se reprograma; el sistema ofrece crear una cita nueva con los mismos datos. A2. Horario ocupado: se rechaza.
Postcondición	La cita tiene la nueva fecha y hora; el historial conserva el cambio.
Requisitos / reglas	RF08, RN13

CU12 Cancelar cita

Tabla 18. *Descripción del caso de uso CU12 Cancelar cita*

Caso de uso	CU12 Cancelar cita
Paquete · Ciclo	P4 Gestión de citas · Ciclo 2
Propósito	Cancelar una cita sin eliminarla y liberar el horario.
Actores	Recepcionista, Paciente (portal)
Actor iniciador	Recepcionista

Precondición	La cita está en estado PROGRAMADA o CONFIRMADA.
Flujo principal	1. El actor elige Cancelar cita. El sistema pide confirmación y el motivo. 2. En una transacción, el sistema cambia el estado a CANCELADA, registra el historial con el motivo, descarta los recordatorios y envía un aviso. 3. El sistema muestra el detalle con el historial actualizado.
Flujos alternativos	A1. Cita atendida o en estado final: no se puede cancelar. A2. Desde el portal, el paciente solo cancela con la anticipación mínima configurada (2 horas) y solo sus propias citas.
Postcondición	La cita queda CANCELADA, permanece en el historial y deja de ocupar el horario.
Requisitos / reglas	RF09, RN09, RN10

CU13 Registrar asistencia

Tabla 19. *Descripción del caso de uso CU13 Registrar asistencia*

Caso de uso	CU13 Registrar asistencia
Paquete · Ciclo	P4 Gestión de citas · Ciclo 2
Propósito	Confirmar la cita y registrar si el paciente asistió (atendida) o no (inasistencia).
Actores	Recepcionista, Médico
Actor iniciador	Recepcionista
Precondición	La cita está PROGRAMADA o CONFIRMADA. Para asistencia o inasistencia, la fecha de la cita ya llegó.
Flujo principal	1. El actor abre el detalle de la cita. El sistema muestra solo las acciones válidas para su estado. 2. El actor elige Confirmar asistencia, Marcar como atendida o El paciente no asistió, y confirma. 3. El sistema valida la transición y la fecha, actualiza el estado y registra el historial y la bitácora.
Flujos alternativos	A1. Transición no permitida (por ejemplo, de Cancelada a Atendida): se rechaza en PHP y en el trigger de la base de datos. A2. Cita futura: no se registra asistencia.
Postcondición	La cita cambia de estado según el diagrama de estados.
Requisitos / reglas	RF10

CU14 Consultar agenda médica

Tabla 20. *Descripción del caso de uso CU14 Consultar agenda médica*

Caso de uso	CU14 Consultar agenda médica
Paquete · Ciclo	P4 Gestión de citas · Ciclo 2

Propósito	Consultar las citas de un médico para una fecha, con filtros por médico, fecha y estado.
Actores	Médico, Recepcionista, Administrador
Actor iniciador	Médico
Precondición	Sesión iniciada con agenda.ver (todos los médicos) o agenda.propia (médico).
Flujo principal	1. El actor abre la Agenda; por defecto, la del día de hoy. 2. Elige la fecha (o navega al día anterior o siguiente), el médico y el estado. 3. El sistema muestra hora, código, paciente, especialidad, estado y motivo de cada cita, intercalados con los bloques libres del médico.
Flujos alternativos	A1. El médico solo ve su propia agenda, aunque intente pedir la de otro. A2. Sin citas: el sistema lo informa. A3. La agenda se puede imprimir.
Postcondición	El actor conoce la agenda solicitada.
Requisitos / reglas	RF11

CU15 Registrar consulta médica

Tabla 21. *Descripción del caso de uso CU15 Registrar consulta médica*

Caso de uso	CU15 Registrar consulta médica
Paquete · Ciclo	P5 Atención médica · Ciclo 2
Propósito	Registrar la atención del paciente: signos vitales, examen físico, diagnóstico y observaciones.
Actores	Médico
Actor iniciador	Médico
Precondición	Sesión de médico con consultas.registrar. La cita es suya, está pendiente y es del día (o anterior).
Flujo principal	1. El médico abre la cita desde su agenda y elige Atender y registrar consulta. 2. El sistema muestra el formulario con los antecedentes, alergias y consultas anteriores del paciente. 3. El médico registra signos vitales, examen físico, diagnóstico (obligatorio) y observaciones; opcionalmente agrega tratamientos (CU17) y estudios (CU18). 4. El sistema valida rangos (peso, talla, temperatura, frecuencia) y el formato de presión (120/80). 5. En una transacción, el sistema guarda la consulta, los tratamientos y los estudios, y pasa la cita a ATENDIDA.
Flujos alternativos	A1. Cita de otro médico: acceso denegado (403). A2. Cita futura o ya cerrada: se informa y no se permite.
Postcondición	La consulta queda registrada y la cita queda ATENDIDA.
Requisitos / reglas	RF10, RN10

CU16 Gestionar historial clínico

Tabla 22. *Descripción del caso de uso CU16 Gestionar historial clínico*

Caso de uso	CU16 Gestionar historial clínico
Paquete · Ciclo	P5 Atención médica · Ciclo 2
Propósito	Mantener los antecedentes del paciente y consultar sus consultas anteriores.
Actores	Médico, Recepcionista, Paciente
Actor iniciador	Médico
Precondición	El paciente existe y tiene historia clínica.
Flujo principal	1. El actor abre la ficha del paciente, pestaña Historia clínica. 2. El sistema muestra antecedentes personales y familiares, alergias, enfermedades crónicas, medicación habitual y las consultas con sus tratamientos y estudios. 3. El actor autorizado actualiza los antecedentes y guarda.
Flujos alternativos	A1. El paciente actualiza su propia información médica desde el portal.
Postcondición	La historia clínica queda actualizada.
Requisitos / reglas	Sección 3.4 del enunciado

CU17 Registrar tratamiento

Tabla 23. *Descripción del caso de uso CU17 Registrar tratamiento*

Caso de uso	CU17 Registrar tratamiento
Paquete · Ciclo	P5 Atención médica · Ciclo 2
Propósito	Indicar medicamentos con dosis, frecuencia, duración e indicaciones durante la consulta.
Actores	Médico
Actor iniciador	Médico
Precondición	El médico está registrando una consulta (CU15).
Flujo principal	1. En el formulario de consulta, el médico pulsa Agregar medicamento. 2. Elige el medicamento y completa dosis, frecuencia, duración e indicaciones. Puede agregar varias filas. 3. Al guardar la consulta, el sistema valida cada fila y registra los tratamientos.
Flujos alternativos	A1. Fila incompleta: el sistema indica qué fila completar.
Postcondición	Los tratamientos quedan asociados a la consulta y visibles en el historial.
Requisitos / reglas	«extend» de CU15

CU18 Gestionar estudios médicos

Tabla 24. *Descripción del caso de uso CU18 Gestionar estudios médicos*

Caso de uso	CU18 Gestionar estudios médicos
Paquete · Ciclo	P5 Atención médica · Ciclo 2
Propósito	Solicitar estudios de laboratorio o imagen, registrar sus resultados y administrar el catálogo.
Actores	Médico (solicita), Recepcionista (registra resultado), Administrador (catálogo)
Actor iniciador	Médico
Precondición	Para solicitar: consulta en curso. Para resultados: permiso estudios.gestionar. Para el catálogo: estudios.catalogo.
Flujo principal	1. En la consulta, el médico pulsa Solicitar estudio, elige el estudio del catálogo y escribe las indicaciones. 2. Al guardar la consulta, el estudio queda SOLICITADO. 3. La recepción abre la bandeja Estudios médicos, filtra por estado y elige Registrar resultado. 4. Registra el estado (en proceso, con resultado o cancelado), el resultado y, opcionalmente, un enlace al archivo. 5. El sistema guarda el resultado y envía un aviso al portal del paciente.
Flujos alternativos	A1. Resultado vacío con estado "Con resultado": se rechaza. A2. El médico solo ve los estudios de sus propios pacientes. A3. Catálogo: agregar, editar, activar o desactivar estudios (sin nombres repetidos).
Postcondición	El estudio queda con su resultado y el paciente es notificado.
Requisitos / reglas	«extend» de CU15

CU19 Consultar historial y resultados

Tabla 25. *Descripción del caso de uso CU19 Consultar historial y resultados*

Caso de uso	CU19 Consultar historial y resultados
Paquete · Ciclo	P5 Atención médica · Ciclo 2
Propósito	Permitir que el paciente consulte sus consultas, diagnósticos, tratamientos y resultados de estudios.
Actores	Paciente
Actor iniciador	Paciente
Precondición	Sesión de paciente con portal.paciente.
Flujo principal	1. El paciente abre Historial médico en su portal. 2. El sistema muestra su resumen médico, los resultados de sus estudios y sus consultas con tratamientos. 3. Desde Inicio, el paciente ve sus recordatorios y avisos (incluido "resultado

	disponible") y los marca como leídos.
Flujos alternativos	A1. El paciente solo accede a su propia información; cualquier otra ruta interna devuelve 403.
Postcondición	El paciente conoce su historial.
Requisitos / reglas	Sección 3.4 del enunciado

CU20 Generar reportes

Tabla 26. *Descripción del caso de uso CU20 Generar reportes*

Caso de uso	CU20 Generar reportes
Paquete · Ciclo	P6 Reportes · Ciclo 2
Propósito	Obtener reportes administrativos de las citas por fecha, médico, especialidad y estado, pacientes registrados y agenda diaria.
Actores	Administrador, Recepcionista
Actor iniciador	Administrador
Precondición	Sesión iniciada con reportes.ver.
Flujo principal	1. El actor abre Reportes y elige el tipo: resumen del período, citas por fecha, por médico, por especialidad, por estado, pacientes registrados o agenda diaria. 2. Elige el rango de fechas (o un atajo: este mes, mes pasado, este año) y los filtros. 3. El sistema muestra la tabla con totales y porcentajes, además de gráficos de barras y de anillo. 4. Opcionalmente, el actor exporta a CSV o imprime a PDF.
Flujos alternativos	A1. Rango invertido: el sistema lo corrige. A2. Rango mayor a un año: se limita. A3. Cada exportación se registra en la bitácora.
Postcondición	El actor obtiene el reporte con sus totales.
Requisitos / reglas	RF13

4.4 Prototipar la interfaz de usuario

Los prototipos se elaboraron primero como bocetos de interfaz (wireframes) y luego se construyeron como pantallas funcionales. Las pantallas finales se presentan en la sección 7.3.

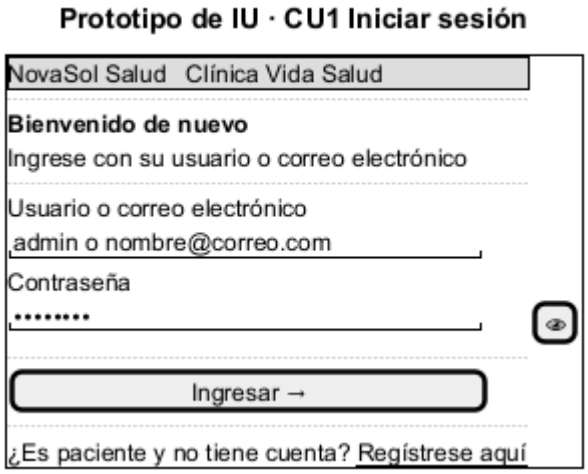


Figura 11. *Prototipo de la interfaz de CU1 Iniciar sesión*

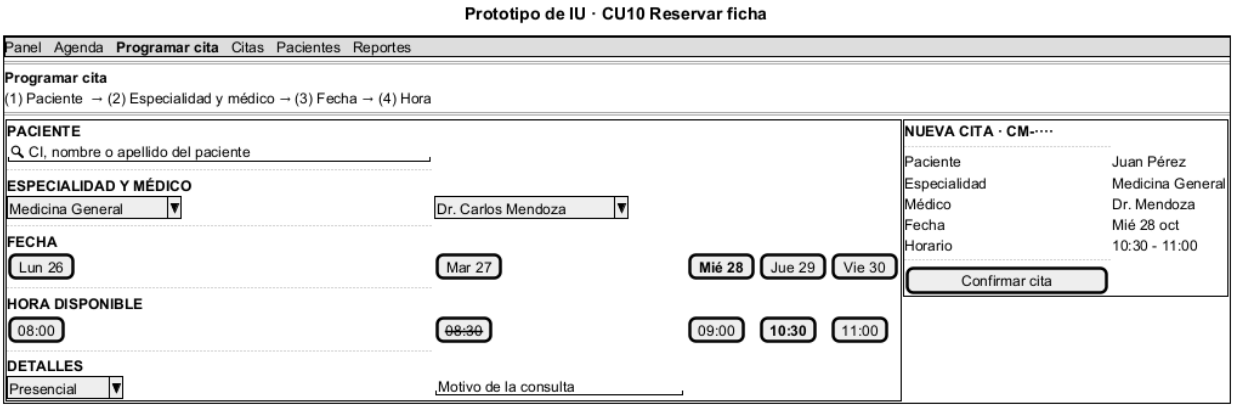


Figura 12. *Prototipo de la interfaz de CU10 Reservar ficha*

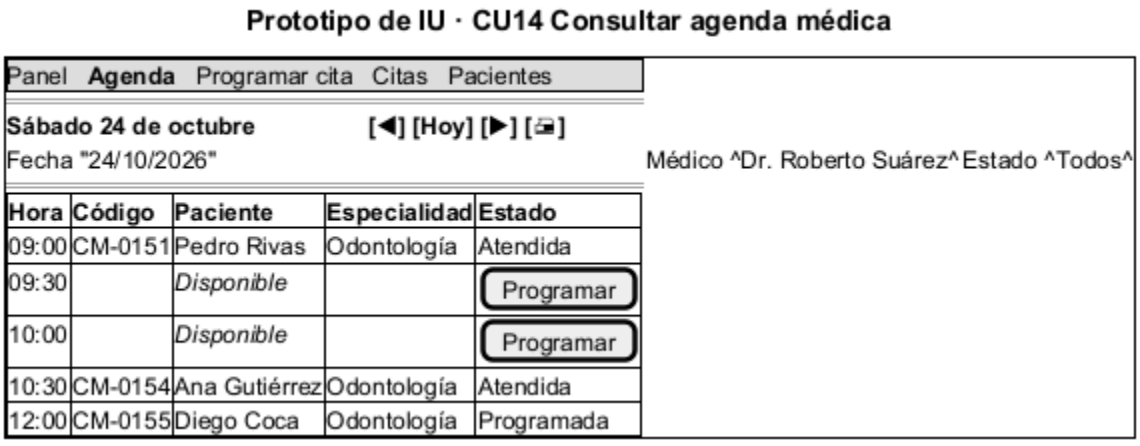


Figura 13. *Prototipo de la interfaz de CU14 Consultar agenda médica*

Tabla 27. *Interfaces del sistema por caso de uso*

Caso de uso	Interfaz (ruta en el sistema)
CU1	/login, /recuperar, /recuperar/codigo
CU2	/roles
CU3	/usuarios, /usuarios/nuevo, /configuracion, /bitacora
CU4	/especialidades
CU5	/medicos, /medicos/nuevo, /medicos/ver
CU6	/horarios
CU7	/pacientes, /pacientes/nuevo, /pacientes/ver
CU8	/registro
CU9 · CU10	/citas/nueva, /portal/reservar
CU11	/citas/reprogramar
CU12 · CU13	/citas/ver, /portal/citas
CU14	/agenda
CU15 · CU17	/atencion
CU16	/pacientes/ver?tab=clinica, /portal/perfil
CU18	/estudios, /estudios/resultado, /estudios/catalogo
CU19	/portal/historial
CU20	/reportes

4.5 Estructurar el modelo de casos de uso

El modelo se estructura con **generalización de actores** (el Administrador hereda los casos de uso del Recepcionista y todos heredan de Usuario), relaciones **«include»** (comportamiento que siempre se ejecuta, por ejemplo reservar una ficha siempre consulta la disponibilidad) y **«extend»** (comportamiento opcional, como el tratamiento y los estudios dentro de la consulta).

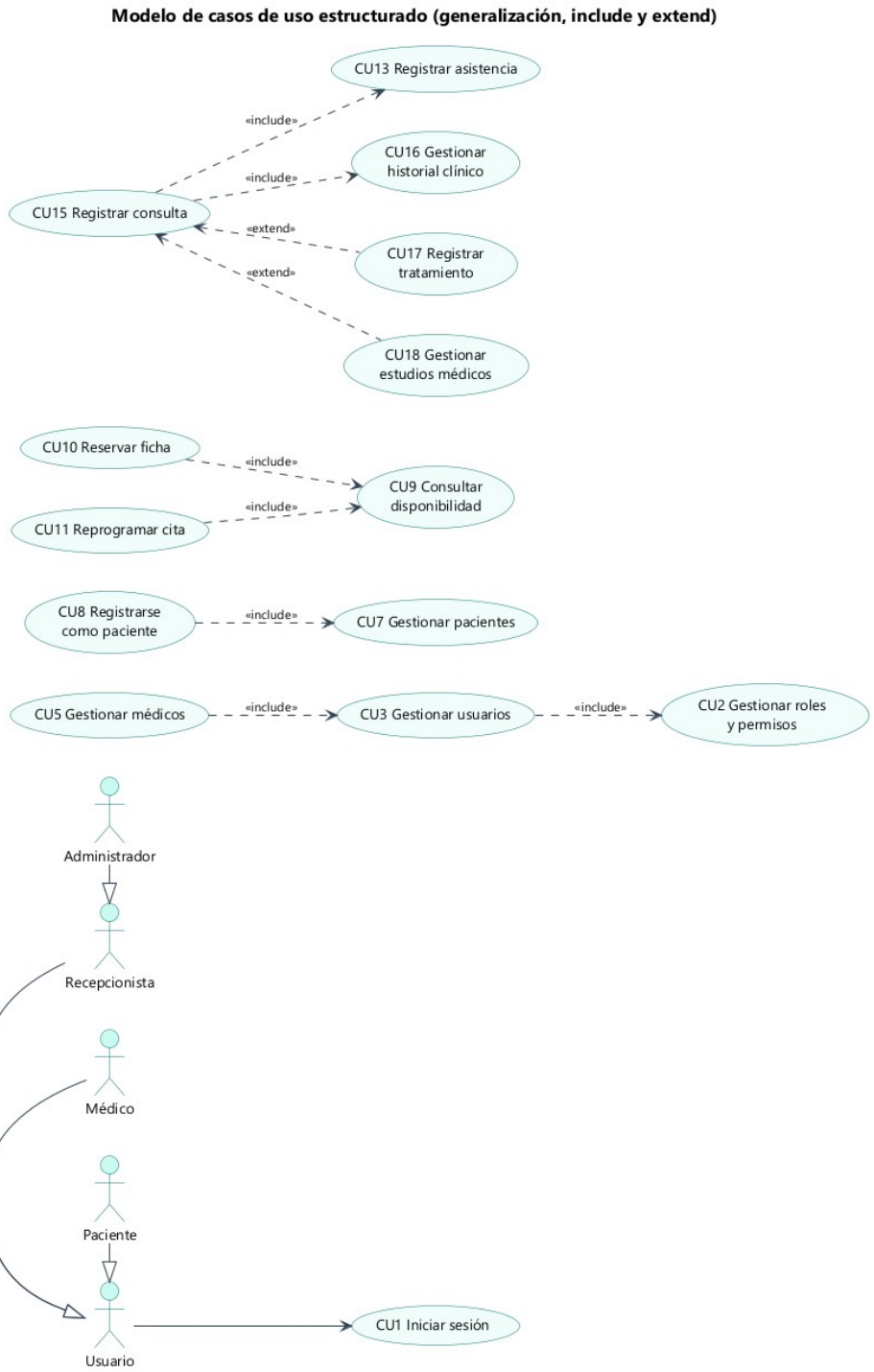


Figura 14. *Modelo de casos de uso estructurado*

4.6 Requerimientos y reglas de negocio

Tabla 28. *Requerimientos funcionales*

Código	Requerimiento funcional	Casos de uso
RF01	Inicio de sesión y control de acceso	CU1, CU2, CU3

Código	Requerimiento funcional	Casos de uso
RF02	Gestión de pacientes (CI único)	CU7, CU8
RF03	Gestión de médicos	CU5
RF04	Gestión de especialidades (nombre normalizado único)	CU4
RF05	Horarios de atención y duración de citas	CU6, CU9
RF06	Registro de citas	CU10
RF07	Prevención de citas duplicadas	CU10, CU11
RF08	Reprogramación de citas	CU11
RF09	Cancelación de citas	CU12
RF10	Control de asistencia y estados	CU13, CU15
RF11	Consulta de agenda médica	CU14
RF12	Historial de citas por paciente	CU7, CU19
RF13	Reportes	CU20

Tabla 29. *Requerimientos no funcionales*

Código	Requerimiento no funcional
RNF01 Seguridad	Contraseñas con password_hash(), consultas parametrizadas, CSRF, XSS, sesiones seguras, permisos por rol, bitácora y recuperación de cuenta con código por correo.
RNF02 Usabilidad	Interfaz en español, diseño adaptable a celulares, modo oscuro, mensajes claros y validación inmediata.
RNF03 Rendimiento	Respuestas menores a 1 segundo; índices en las columnas de búsqueda; servidor y base en la misma región.
RNF04 Disponibilidad	Publicado en la nube (Vercel y Supabase), accesible las 24 horas desde cualquier navegador.
RNF05 Mantenibilidad	Arquitectura MVC con capa de servicios, código comentado y organizado por paquetes.
RNF06 Portabilidad	Funciona en Vercel, en Apache (XAMPP/Laragon) y en el servidor integrado de PHP.

Tabla 30. *Reglas de negocio y su implementación*

Regla	Descripción	Dónde se cumple
RN01 / RN02	Paciente y médico con identificador único	PK, UNIQUE (código, CI, registro)
RN03	Cita asociada a paciente y médico válidos	FOREIGN KEY y CitaService::programar
RN04	El médico debe estar activo	Disponibilidad y trigger validar_cita
RN05	La cita pertenece al horario del médico	Disponibilidad y trigger validar_cita

Regla	Descripción	Dónde se cumple
RN06	Sin citas en fechas pasadas	Validator y trigger validar_cita
RN07	Sin citas superpuestas	Transacción + FOR UPDATE + restricción EXCLUDE
RN08	Duración según configuración	Bloques de DURACION_CITA_MINUTOS y trigger
RN09	Las canceladas no ocupan disponibilidad	estadocita.ocupahorario y EXCLUDE … WHERE
RN10	Las atendidas conservan su historial	No se borran citas; historialcita
RN11	No eliminar con historial: desactivar	Solo cambio de estado; FK sin CASCADE
RN12	Solo usuarios autorizados	Auth::requerir(permiso) en cada acción
RN13	Disponibilidad verificada en el servidor	Se repite la verificación dentro de la transacción
RN14	Persistencia en PostgreSQL	PDO con pdo_pgsql

5. FT ANÁLISIS

5.1 Análisis de la arquitectura

Los casos de uso se agrupan en paquetes de análisis según su afinidad funcional. **P1 Seguridad y acceso** es un paquete de servicio común del que dependen todos los demás; **P4 Gestión de citas** es el paquete central del negocio.

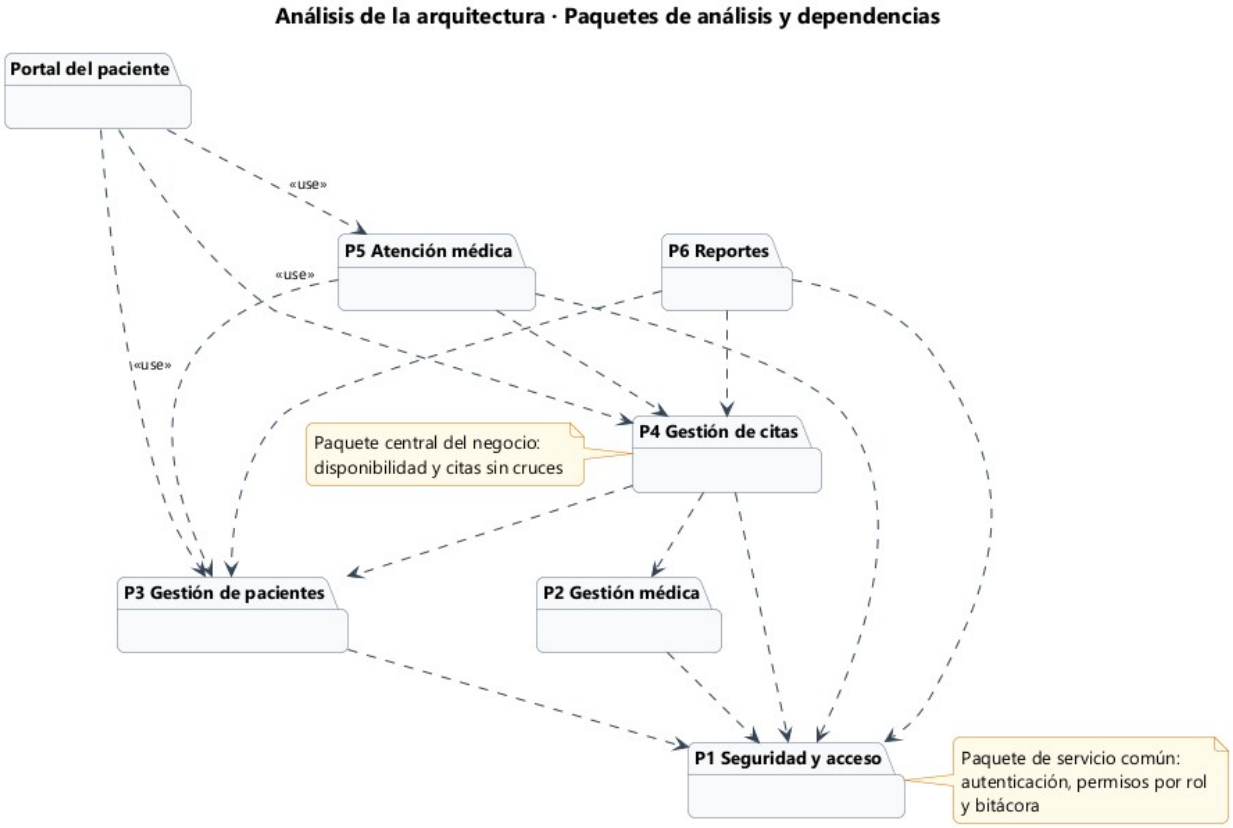


Figura 15. *Paquetes de análisis y sus dependencias*

5.2 Analizar casos de uso

Para cada caso de uso se identificaron sus clases de análisis (interfaz, control y entidad) y los mensajes que intercambian, representados en diagramas de comunicación.

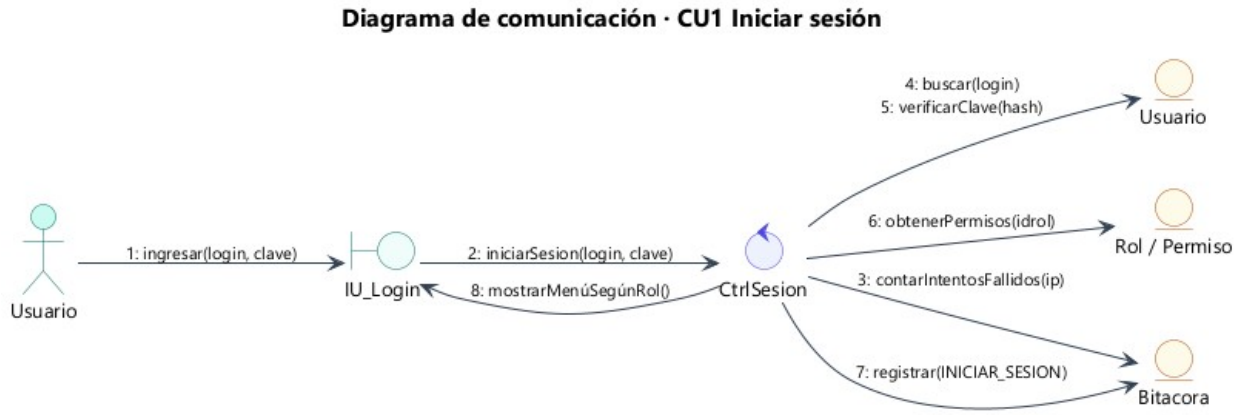


Figura 16. *Diagrama de comunicación de CU1 Iniciar sesión*

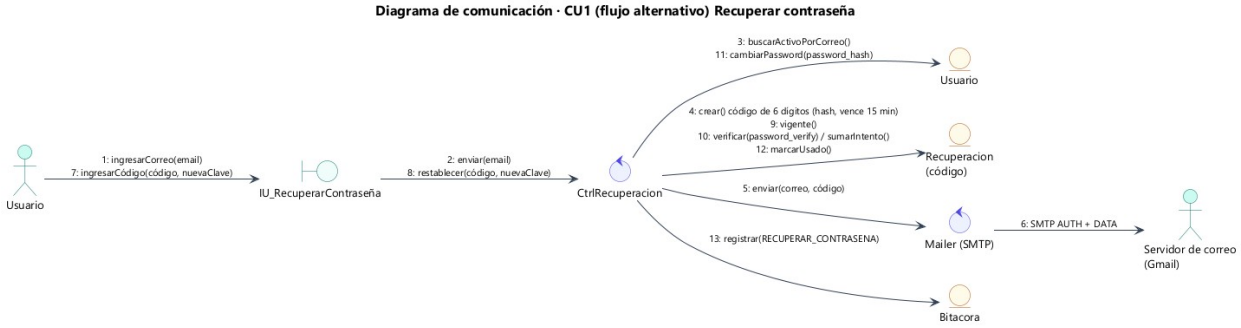


Figura 17. *Diagrama de comunicación de CU1 (flujo alternativo) Recuperar contraseña por correo*

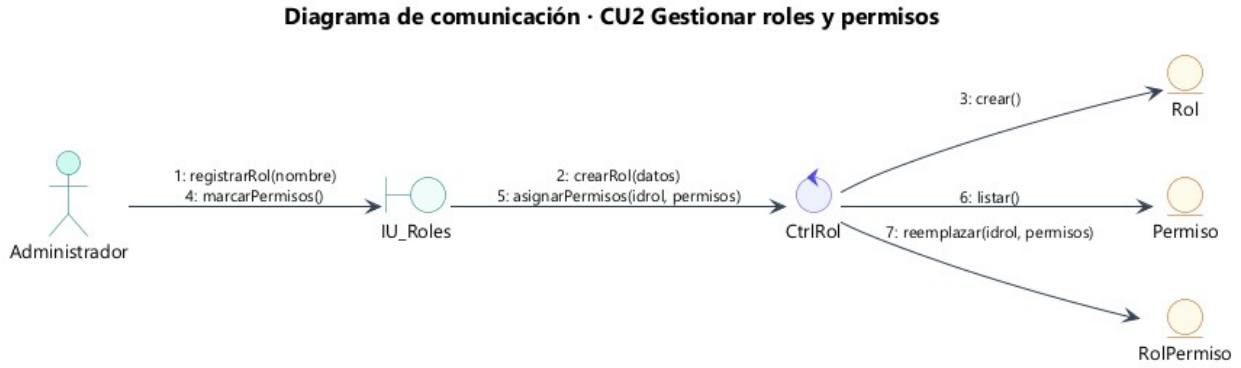


Figura 18. *Diagrama de comunicación de CU2 Gestionar roles y permisos*

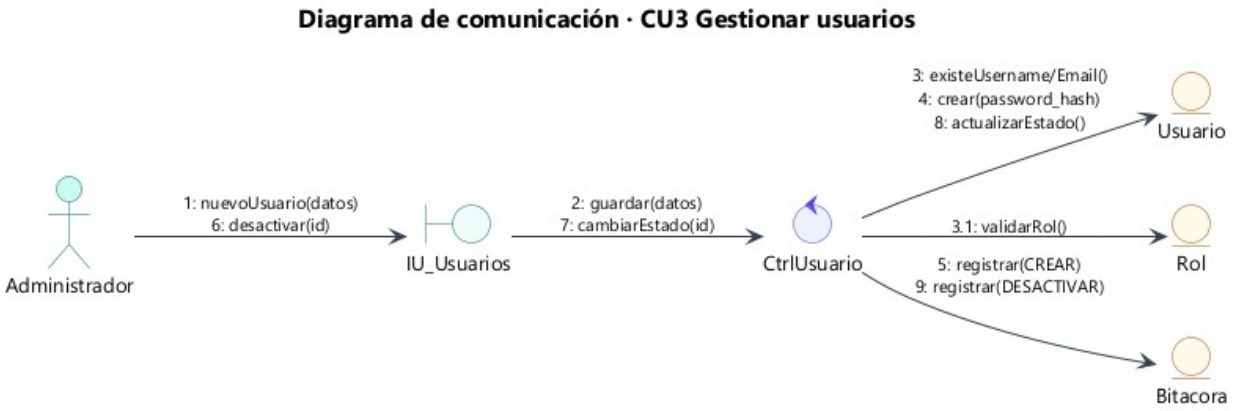


Figura 19. *Diagrama de comunicación de CU3 Gestionar usuarios*

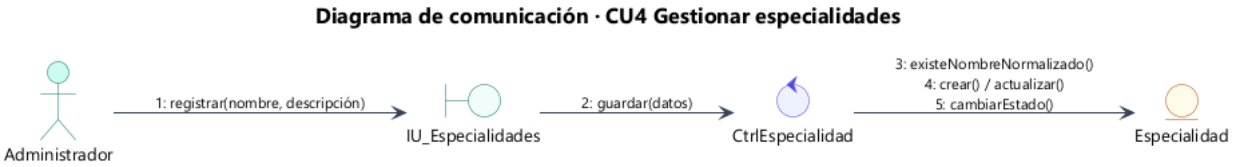


Figura 20. *Diagrama de comunicación de CU4 Gestionar especialidades*

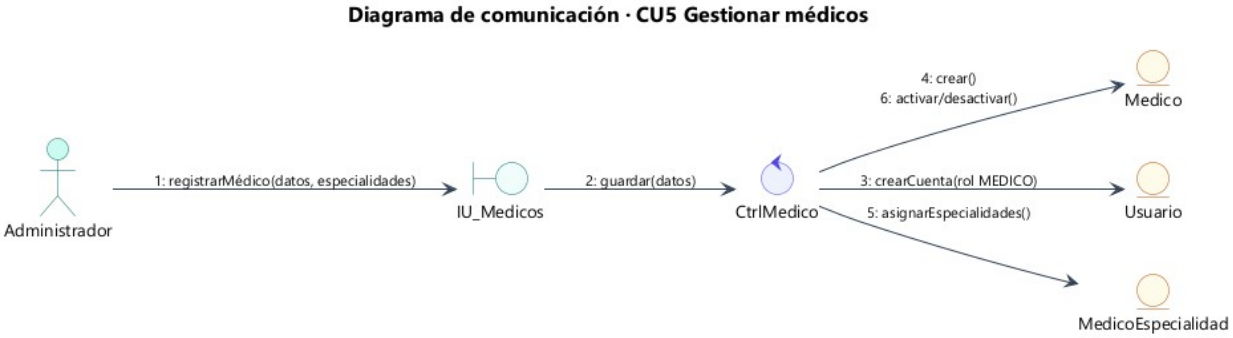


Figura 21. *Diagrama de comunicación de CU5 Gestionar médicos*



Figura 22. *Diagrama de comunicación de CU6 Gestionar horarios*

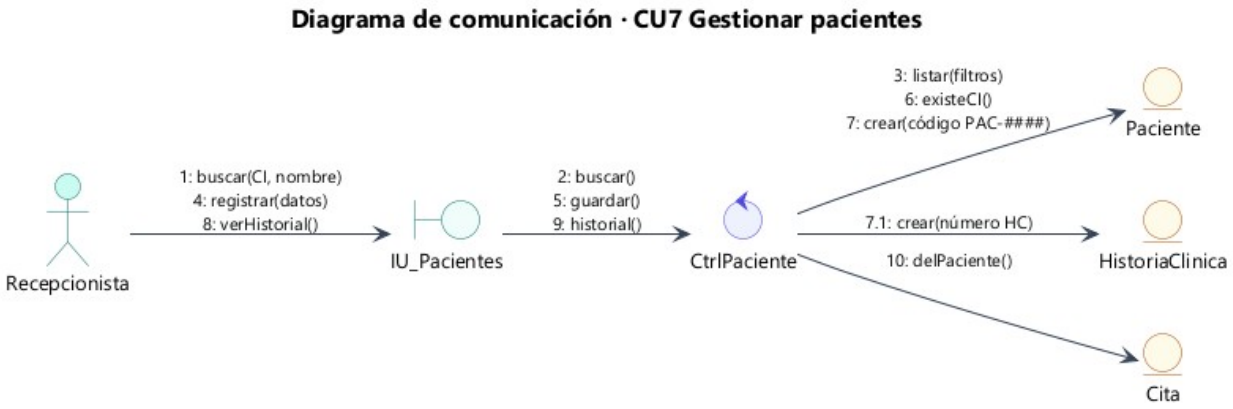


Figura 23. *Diagrama de comunicación de CU7 Gestionar pacientes*

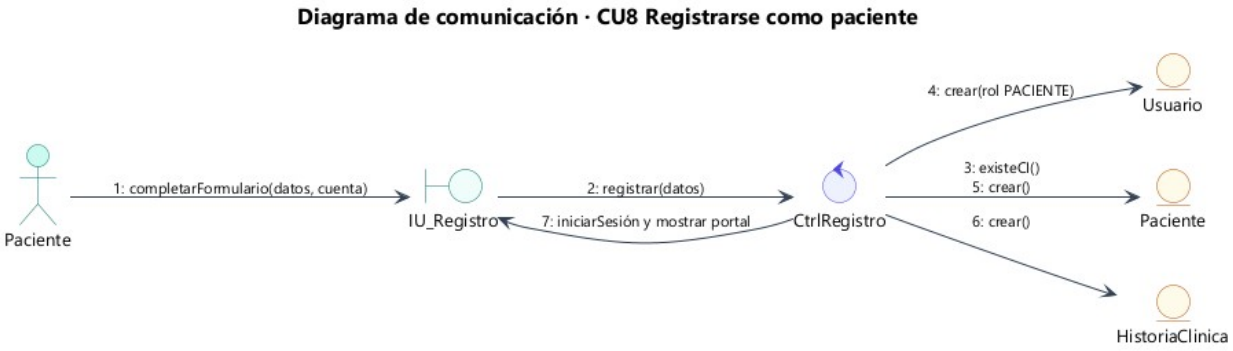


Figura 24. *Diagrama de comunicación de CU8 Registrarse como paciente*

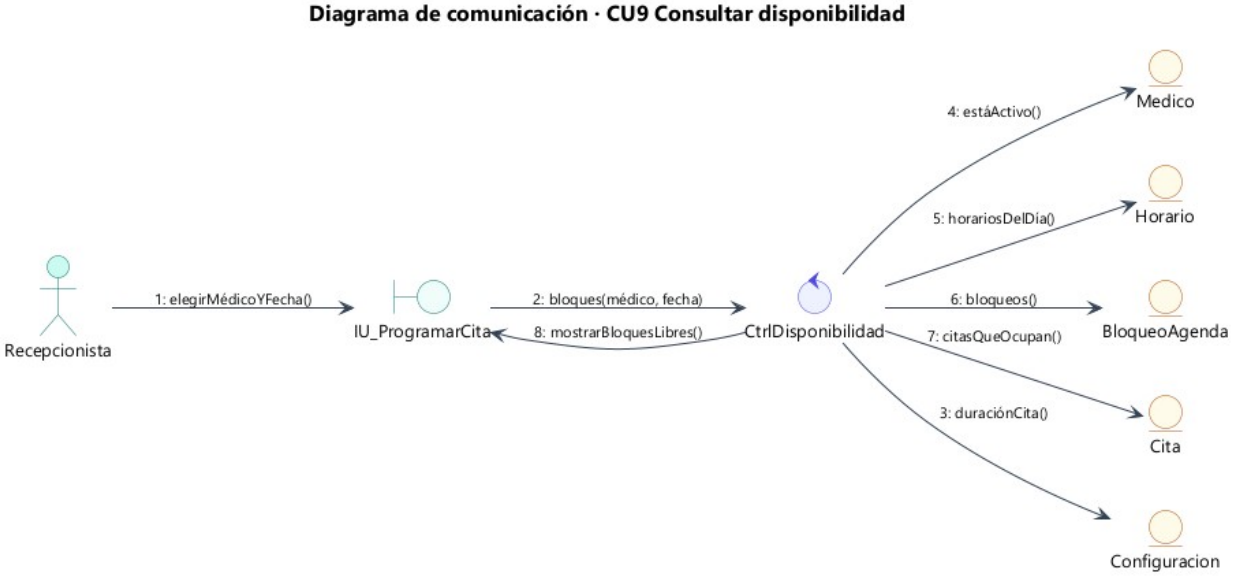


Figura 25. *Diagrama de comunicación de CU9 Consultar disponibilidad*

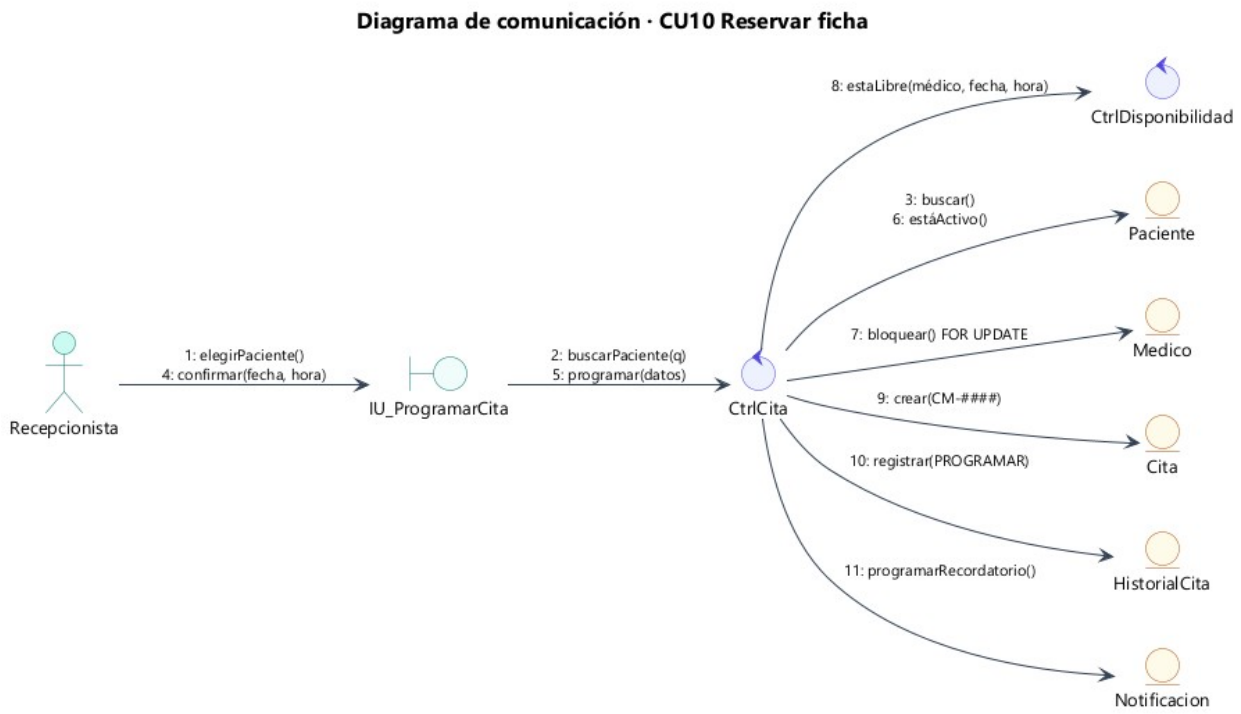


Figura 26. *Diagrama de comunicación de CU10 Reservar ficha*

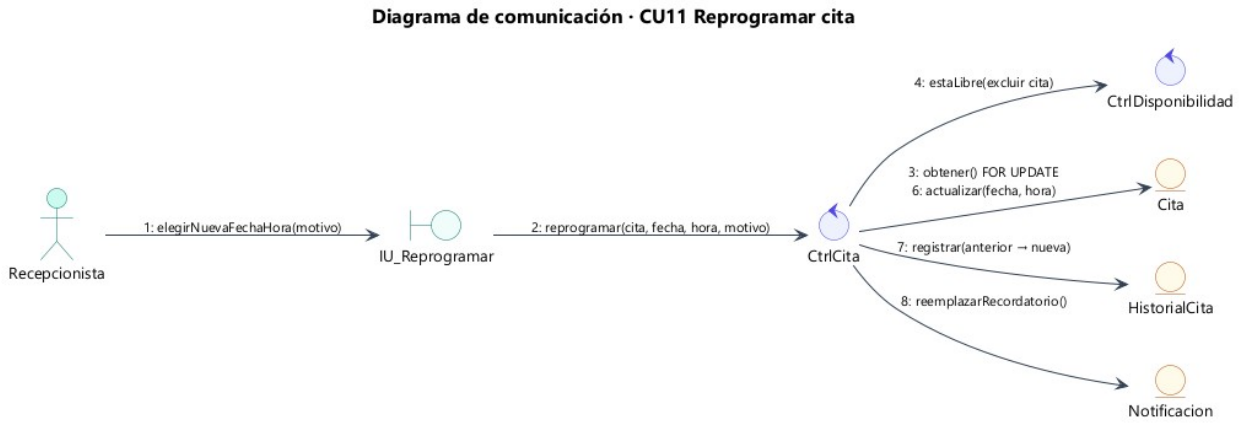


Figura 27. *Diagrama de comunicación de CU11 Reprogramar cita*



Figura 28. *Diagrama de comunicación de CU12 Cancelar cita*



Figura 29. *Diagrama de comunicación de CU13 Registrar asistencia*

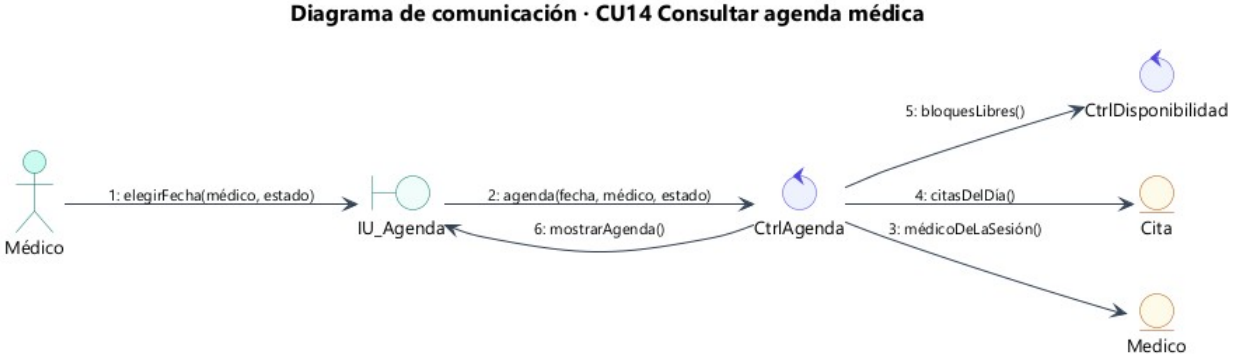


Figura 30. *Diagrama de comunicación de CU14 Consultar agenda médica*

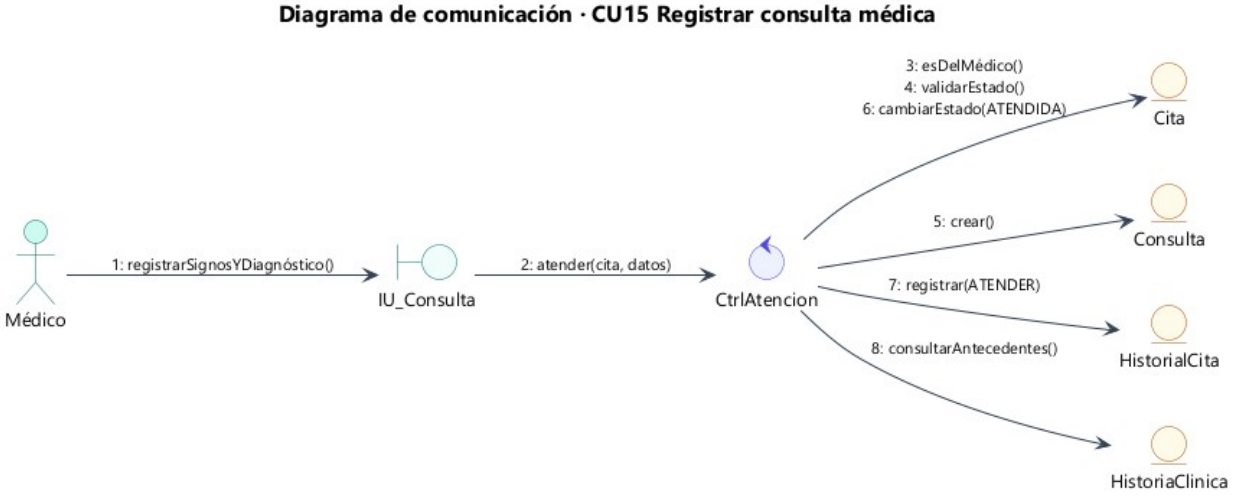


Figura 31. *Diagrama de comunicación de CU15 Registrar consulta médica*

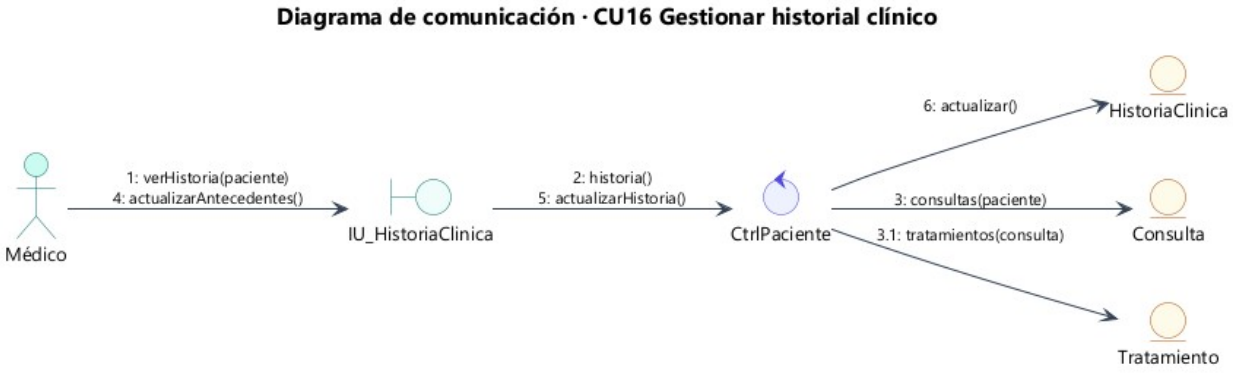


Figura 32. *Diagrama de comunicación de CU16 Gestionar historial clínico*



Figura 33. *Diagrama de comunicación de CU17 Registrar tratamiento*

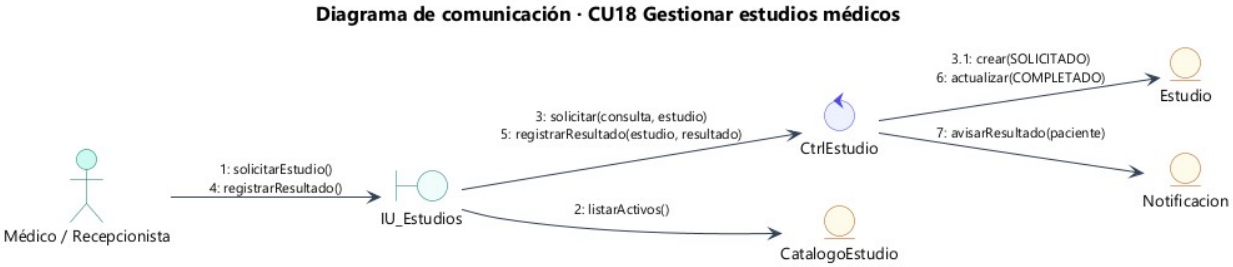


Figura 34. *Diagrama de comunicación de CU18 Gestionar estudios médicos*

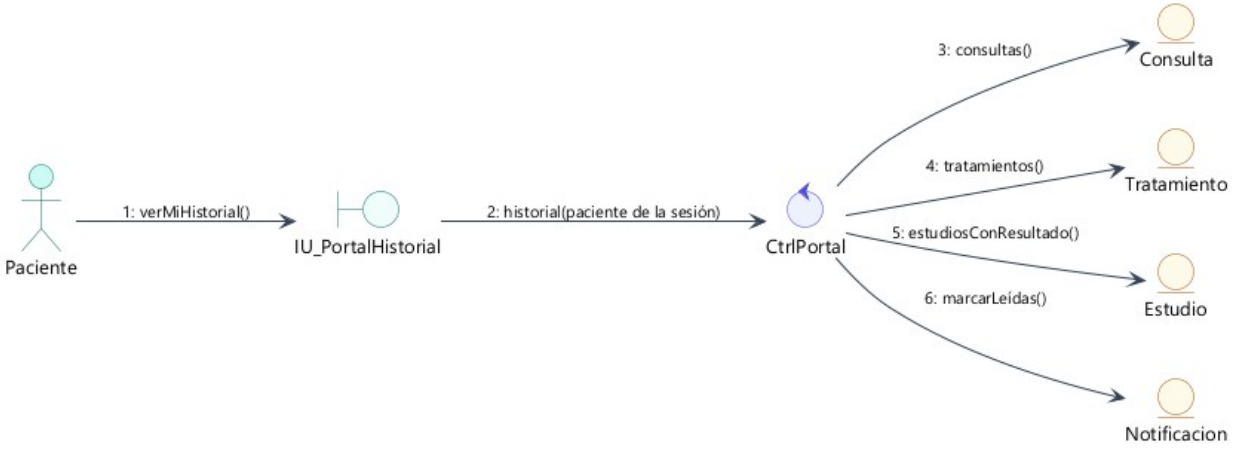


Figura 35. *Diagrama de comunicación de CU19 Consultar historial y resultados*

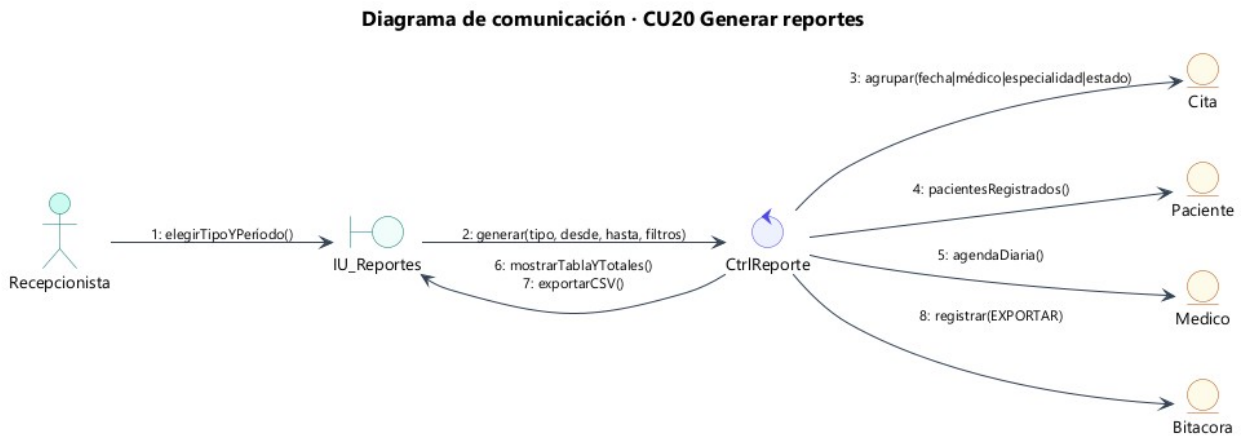


Figura 36. *Diagrama de comunicación de CU20 Generar reportes*

5.3 Análisis de clases

El siguiente diagrama reúne las clases de análisis de los casos de uso principales: las de interfaz representan las pantallas, las de control coordinan cada caso de uso y las de entidad representan la información persistente.

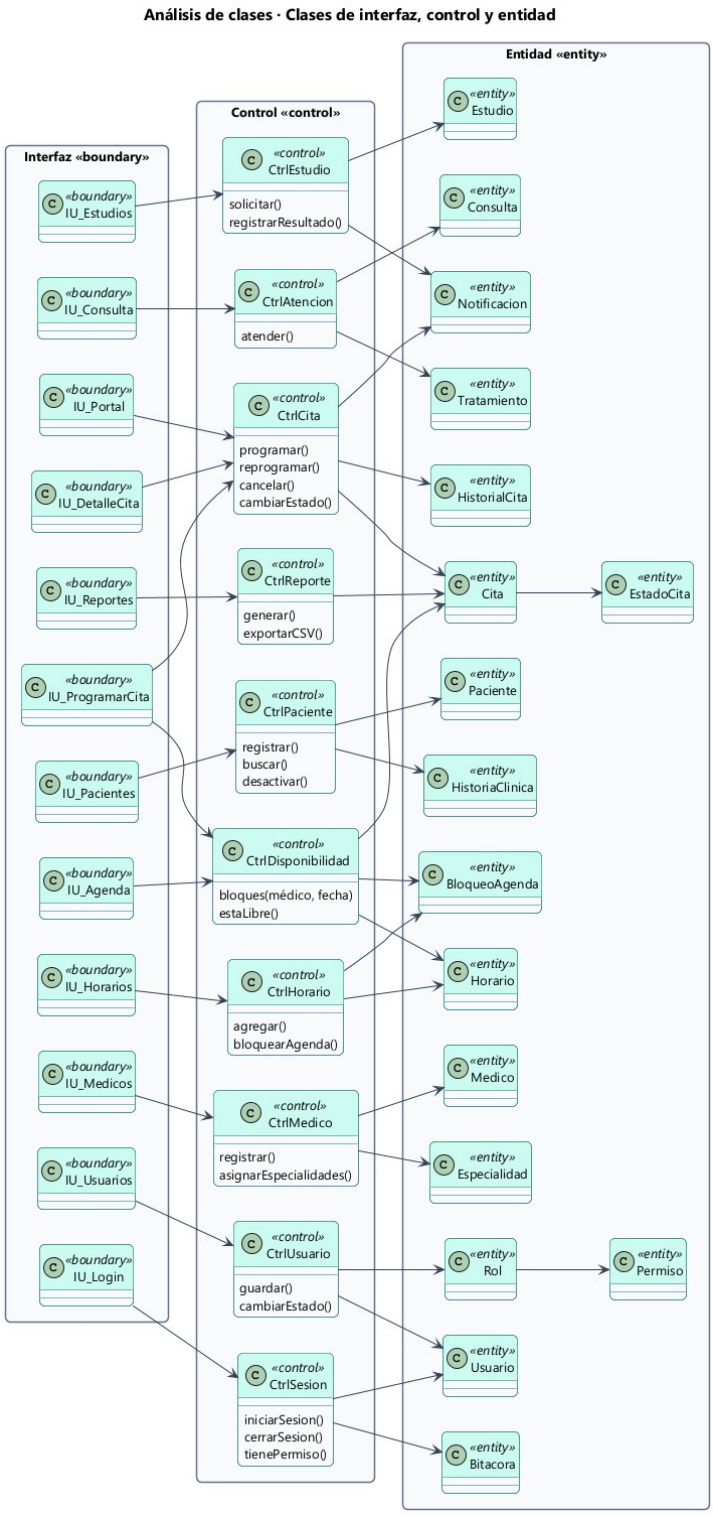


Figura 37. *Diagrama de clases de análisis*

La entidad **Cita** tiene un ciclo de vida con estados y transiciones controladas (RF10):

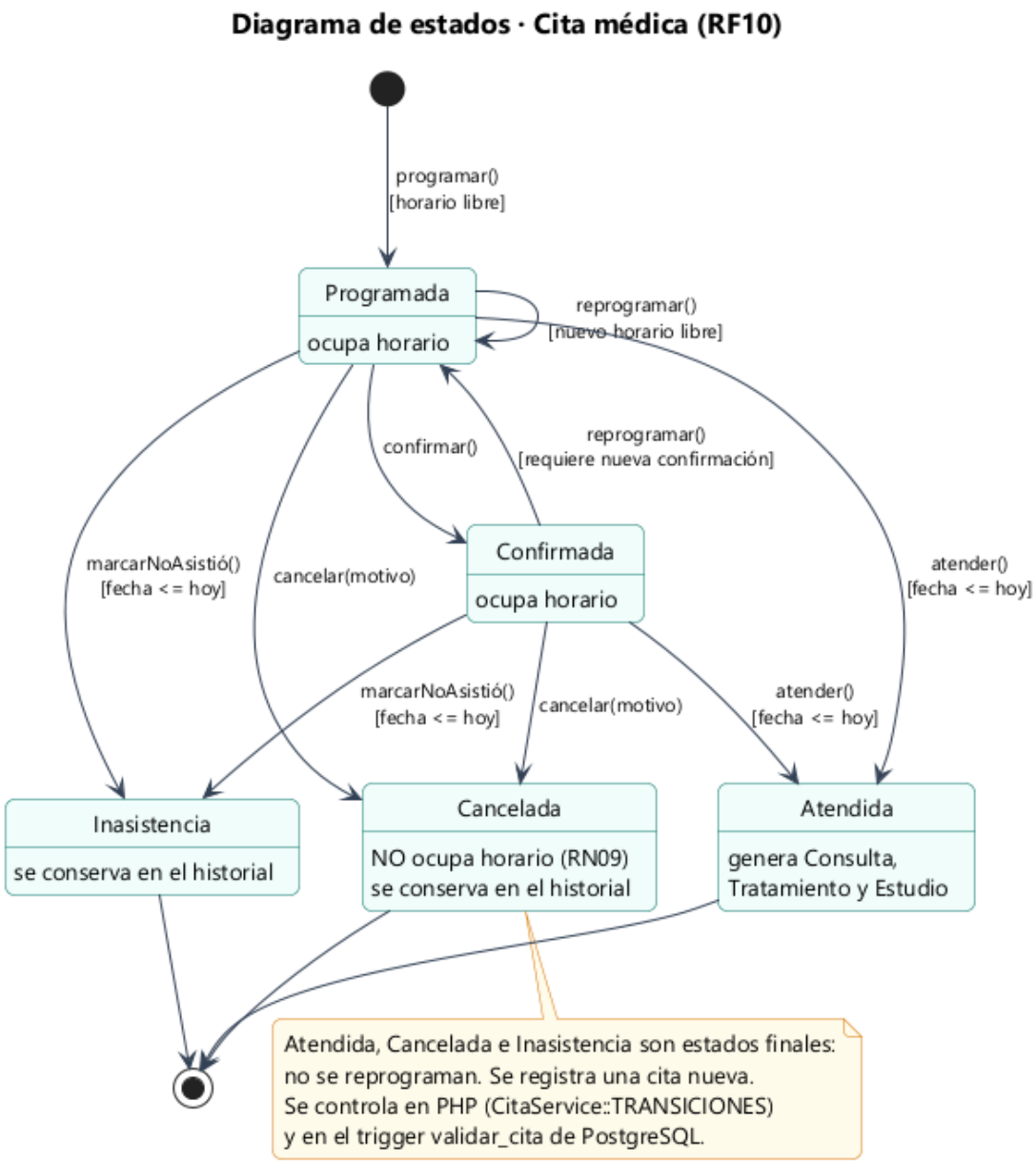


Figura 38. *Diagrama de estados de la cita médica*

5.4 Analizar paquetes

Tabla 31. *Contenido de los paquetes de análisis*

Paquete	Interfaz «boundary»	Control «control»	Entidad «entity»	CU
P1 Seguridad y acceso	IU_Login, IU_Usuarios, IU_Roles, IU_Configuración, IU_Bitácora	CtrlSesion, CtrlUsuario, CtrlRol	Usuario, Rol, Permiso, Configuracion, Bitacora	CU1–CU3
P2 Gestión médica	IU_Especialidades, IU_Medicos, IU_Horarios	CtrlEspecialidad, CtrlMedico,	Especialidad, Medico, MedicoEspecialidad, Horario,	CU4–CU6

Paquete	Interfaz «boundary»	Control «control»	Entidad «entity»	CU
		CtrlHorario	BloqueoAgenda	
P3 Gestión de pacientes	IU_Pacientes, IU_FichaPaciente, IU_Registro	CtrlPaciente, CtrlRegistro	Paciente, HistoriaClinica, ContactoEmergencia	CU7–CU8
P4 Gestión de citas	IU_ProgramarCita, IU_DetalleCita, IU_Reprogramar, IU_Agenda	CtrlCita, CtrlDisponibilidad, CtrlAgenda	Cita, EstadoCita, HistorialCita, Notificacion	CU9–CU14
P5 Atención médica	IU_Consulta, IU_Estudios, IU_PortalHistorial	CtrlAtencion, CtrlEstudio, CtrlPortal	Consulta, Tratamiento, Medicamento, Estudio, CatalogoEstudio	CU15–CU19
P6 Reportes	IU_Reportes	CtrlReporte	Cita, Paciente, Medico (consulta)	CU20

6. FT DISEÑO

6.1 Diseño de la arquitectura

6.1.1 Diseño físico de la arquitectura · Diagrama de despliegue

El sistema se despliega en la nube. El navegador se comunica por HTTPS con Vercel, que sirve los archivos estáticos desde su CDN y ejecuta PHP 8.3 como función serverless en São Paulo. La función se conecta por TLS, mediante PDO, al pooler de Supabase, que da acceso a PostgreSQL 17 en la misma región.

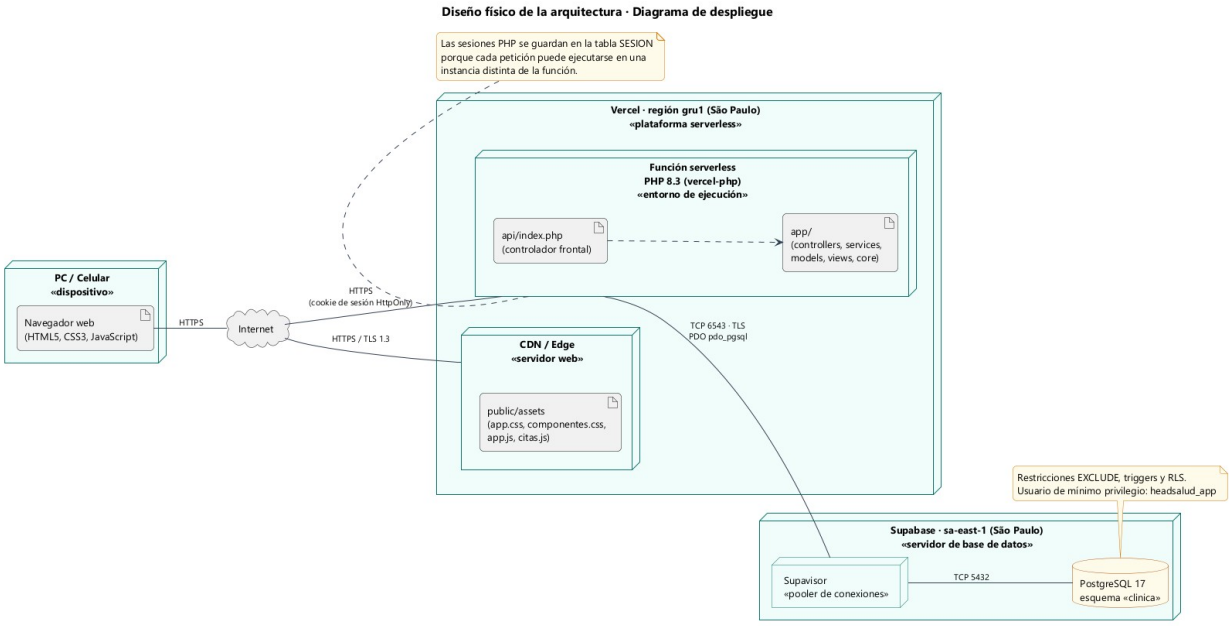


Figura 39. *Diagrama de despliegue en producción*

El sistema también puede ejecutarse en un servidor local con Apache (XAMPP o Laragon) y PostgreSQL:

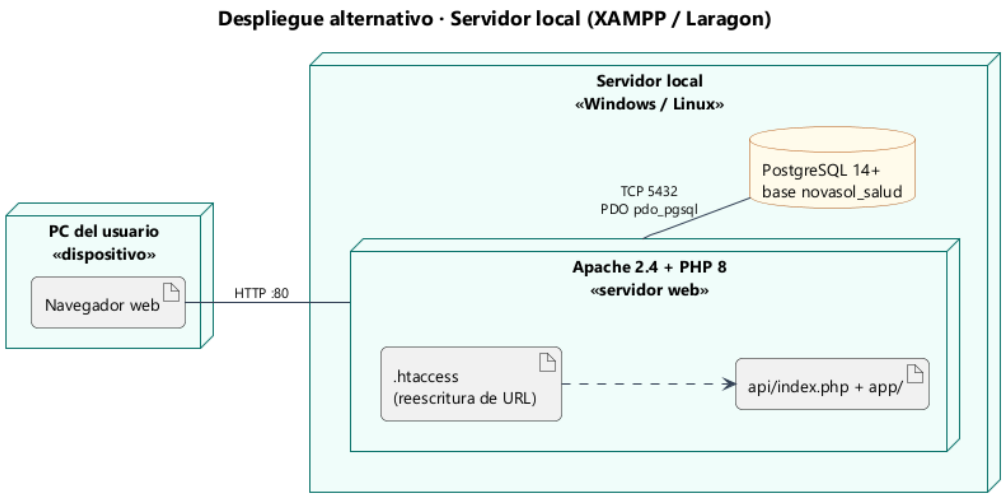


Figura 40. *Diagrama de despliegue alternativo en servidor local*

6.1.2 Diseño lógico de la arquitectura · Diagrama de paquetes

La aplicación sigue una arquitectura en capas basada en MVC: presentación (vistas y recursos estáticos), control (controlador frontal y controladores), lógica de negocio (servicios), acceso a datos (modelos) y un núcleo propio (enrutador, autenticación, sesiones, CSRF, validación, vistas y base de datos).

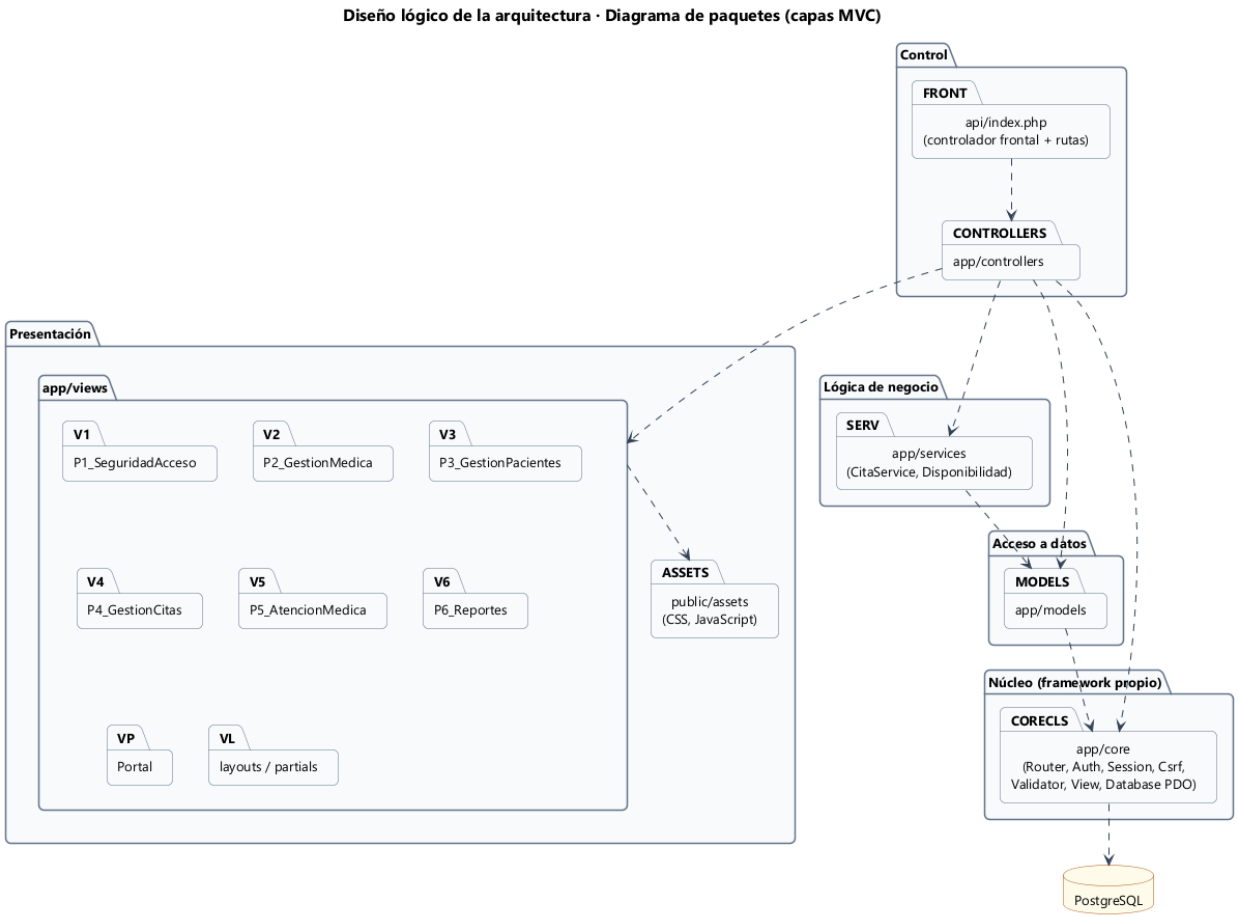


Figura 41. *Diagrama de paquetes de diseño (capas)*

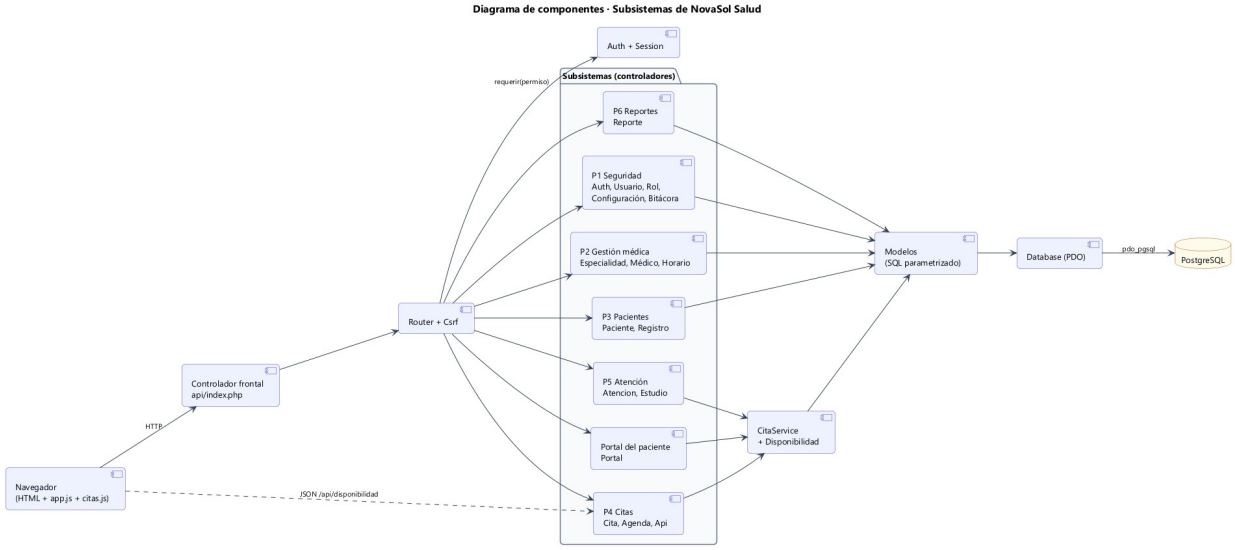


Figura 42. *Diagrama de componentes y subsistemas*

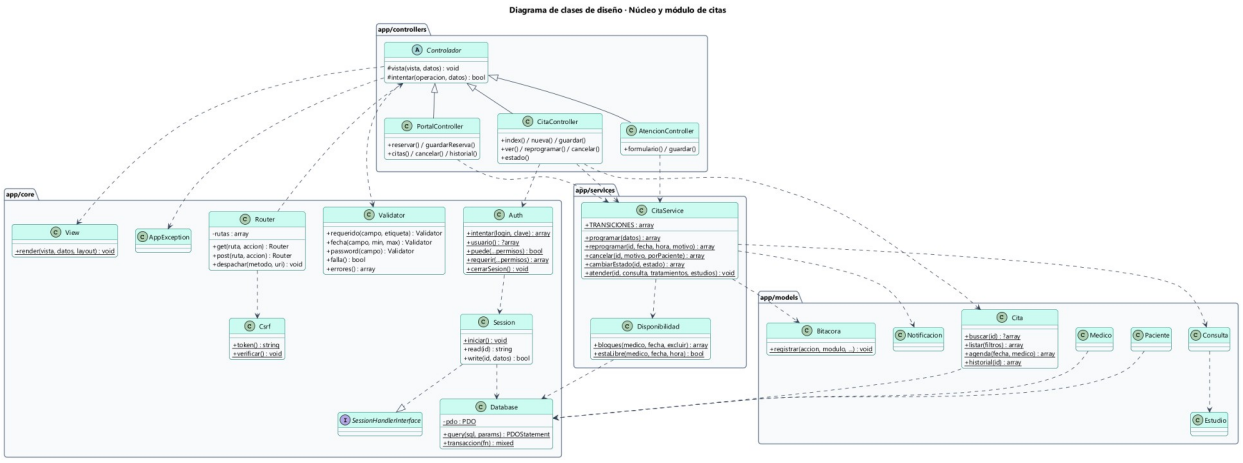


Figura 43. *Diagrama de clases de diseño del núcleo y del módulo de citas*

6.2 Diseñar casos de uso · Diagramas de secuencia

Los diagramas de secuencia muestran, para cada caso de uso, la interacción entre las vistas, los controladores, los servicios, los modelos y la base de datos tal como están implementados.

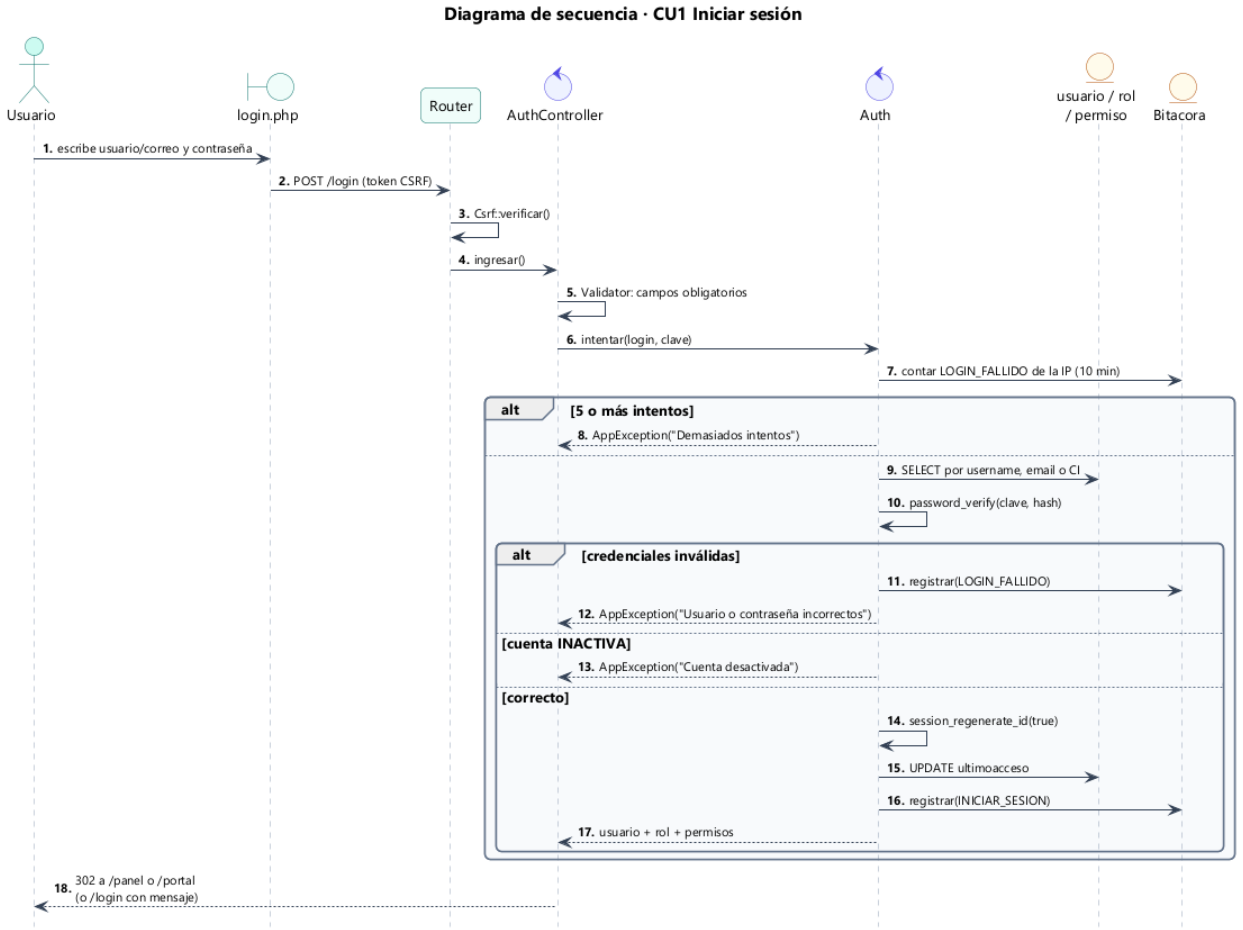


Figura 44. *Diagrama de secuencia de CU1 Iniciar sesión*

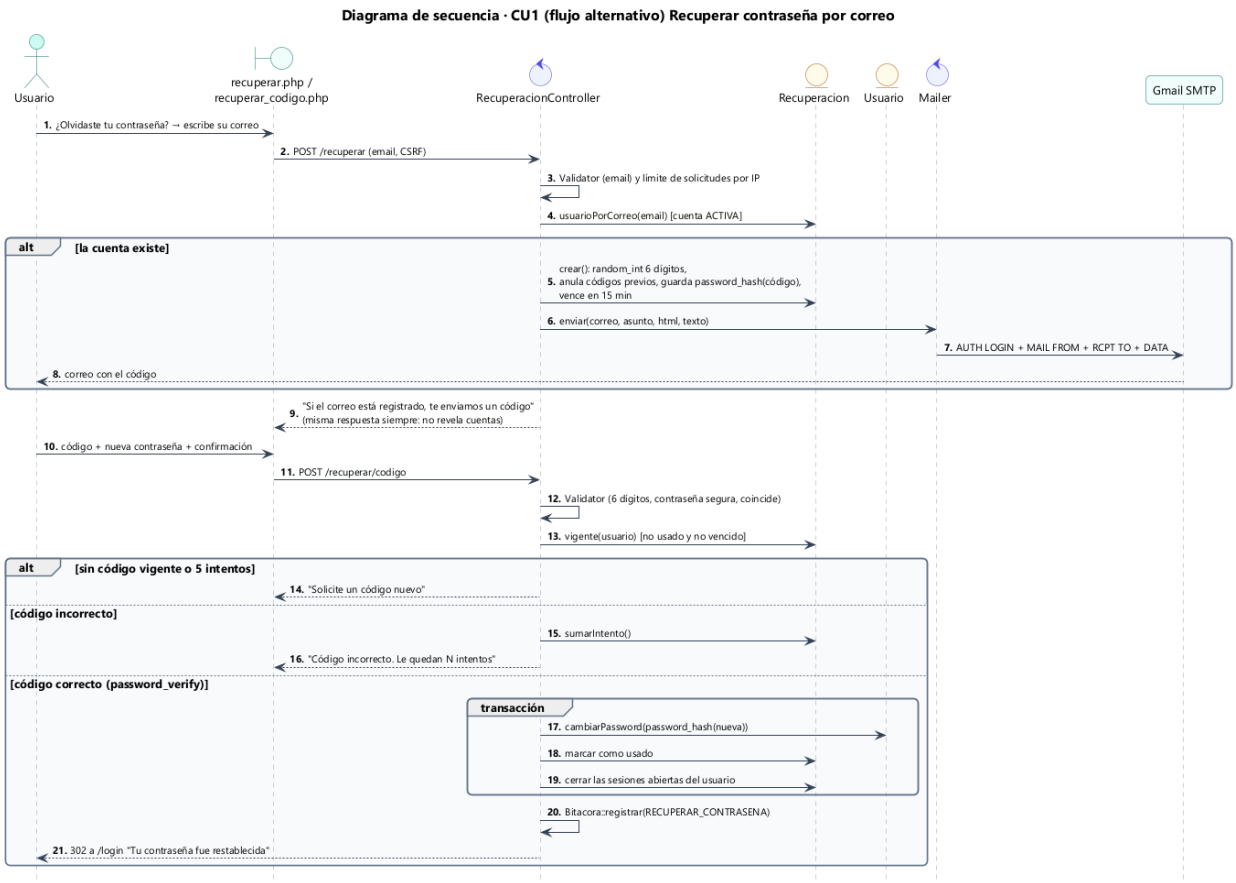


Figura 45. *Diagrama de secuencia de CU1 (flujo alternativo) Recuperar contraseña por correo*

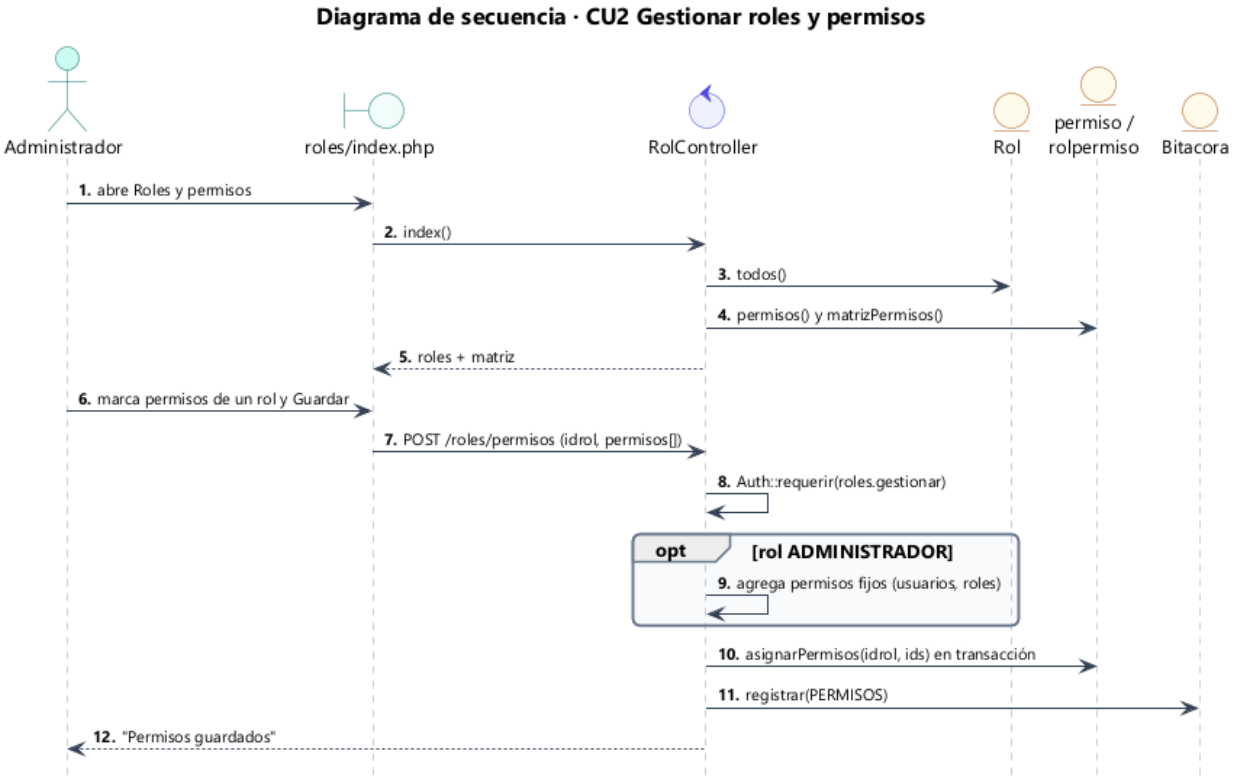


Figura 46. *Diagrama de secuencia de CU2 Gestionar roles y permisos*

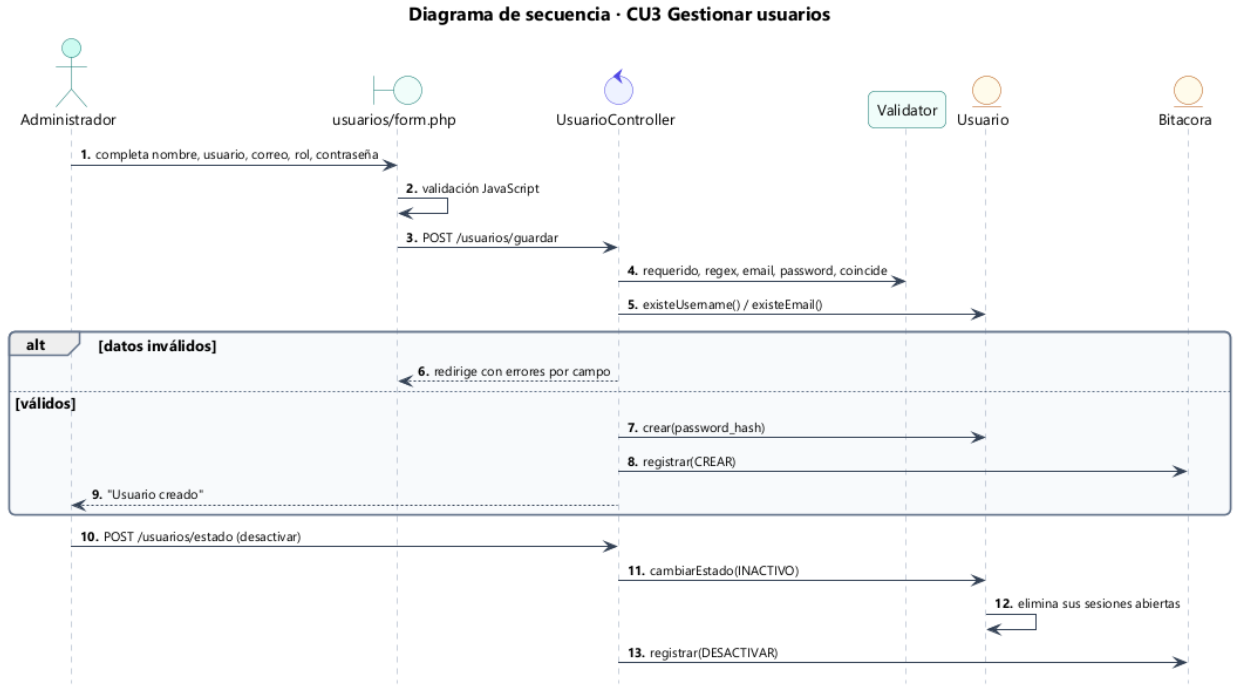


Figura 47. *Diagrama de secuencia de CU3 Gestionar usuarios*

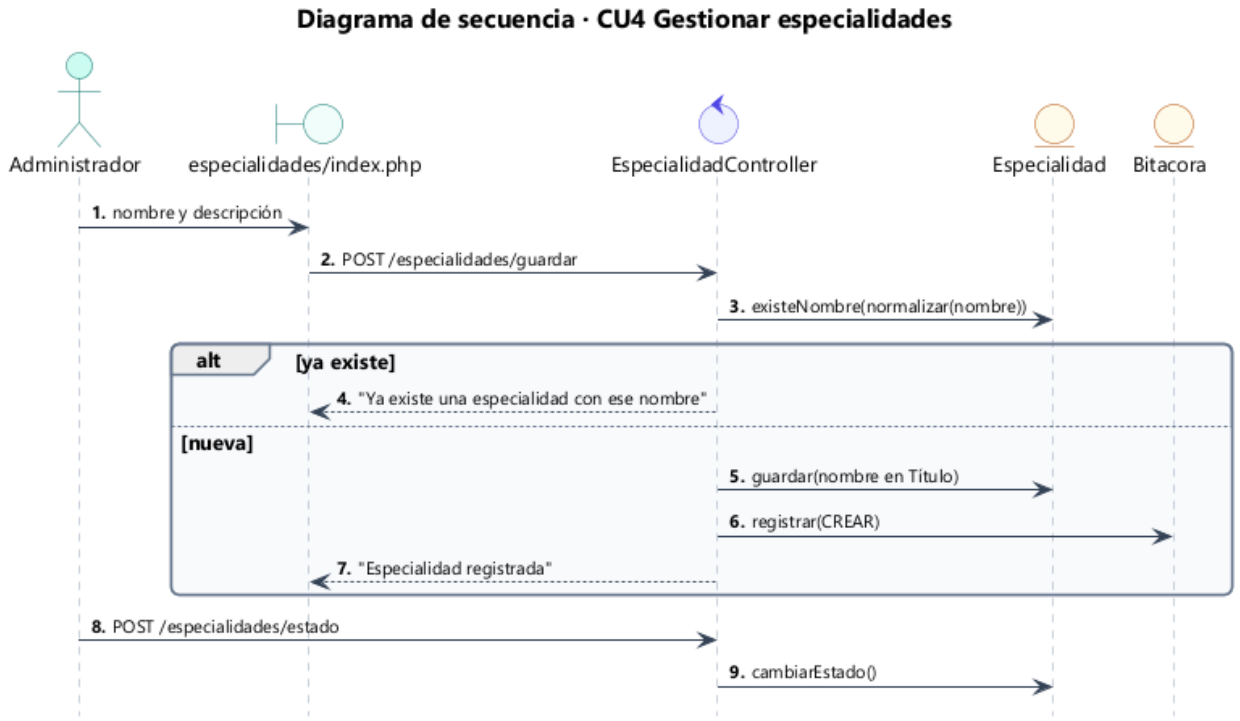


Figura 48. *Diagrama de secuencia de CU4 Gestionar especialidades*

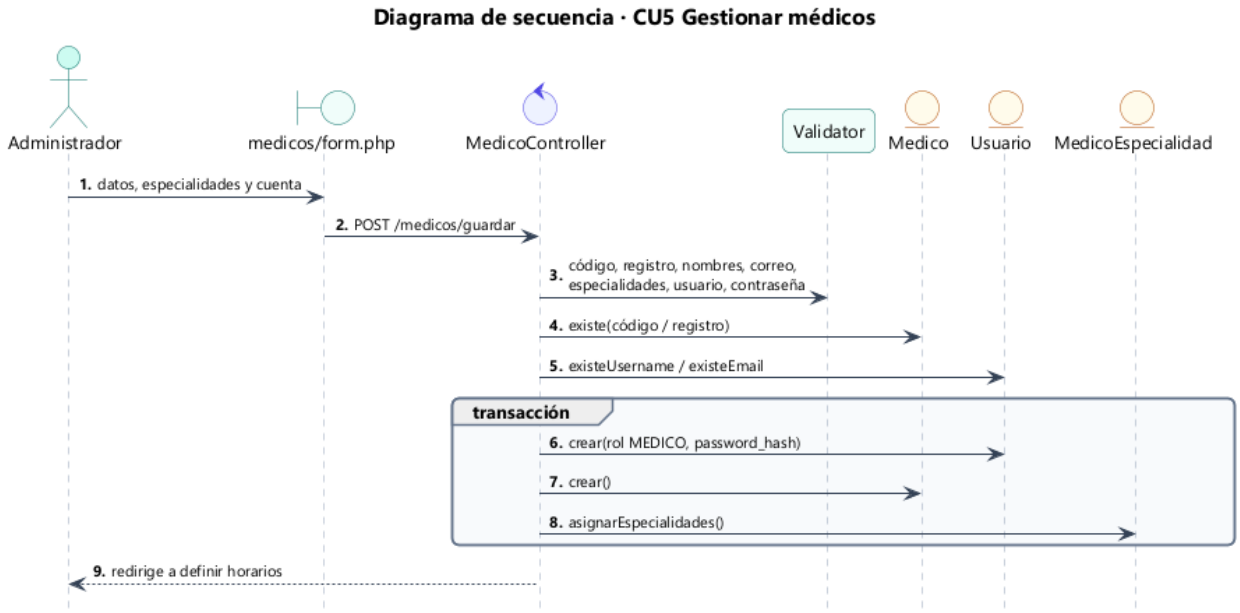


Figura 49. *Diagrama de secuencia de CU5 Gestionar médicos*

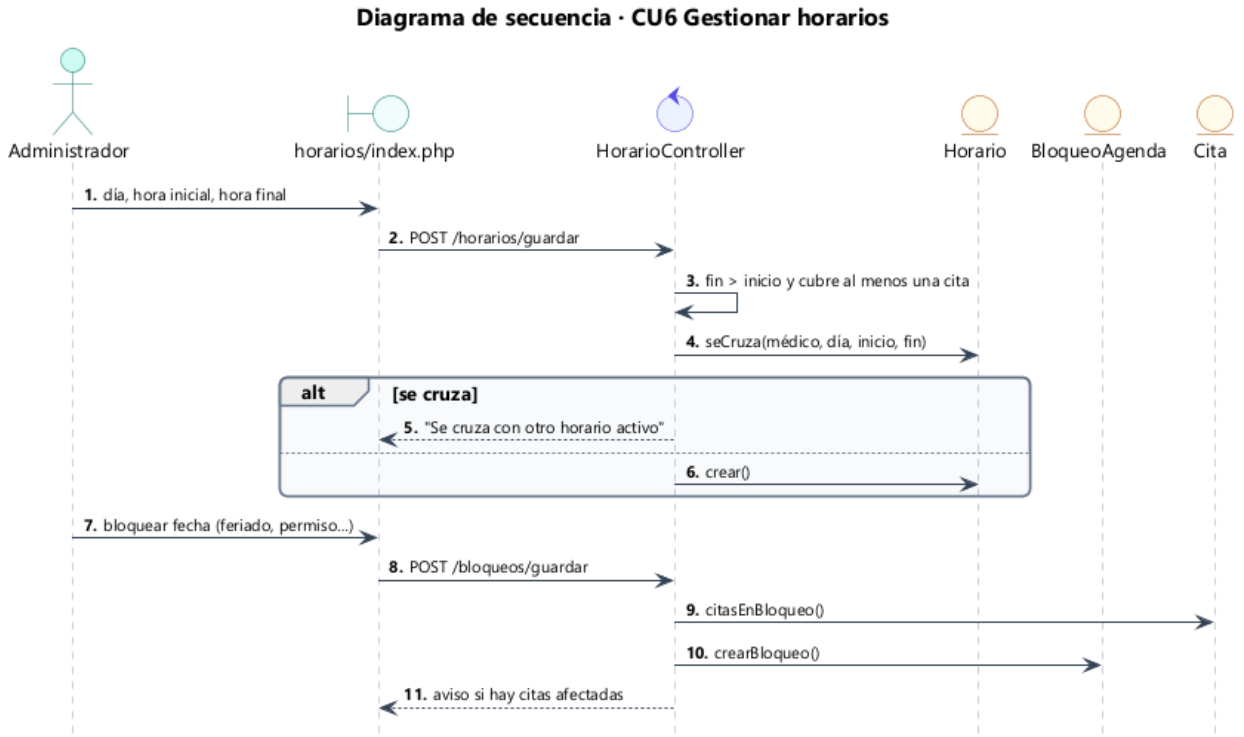


Figura 50. *Diagrama de secuencia de CU6 Gestionar horarios*

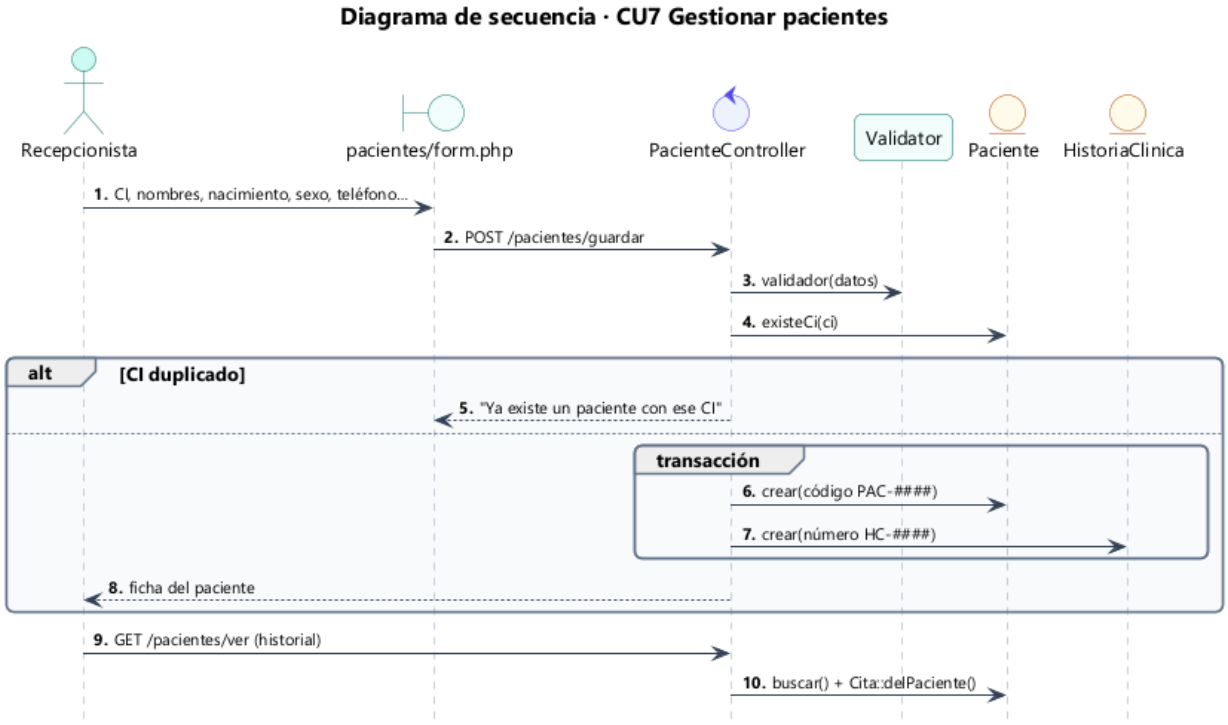


Figura 51. *Diagrama de secuencia de CU7 Gestionar pacientes*

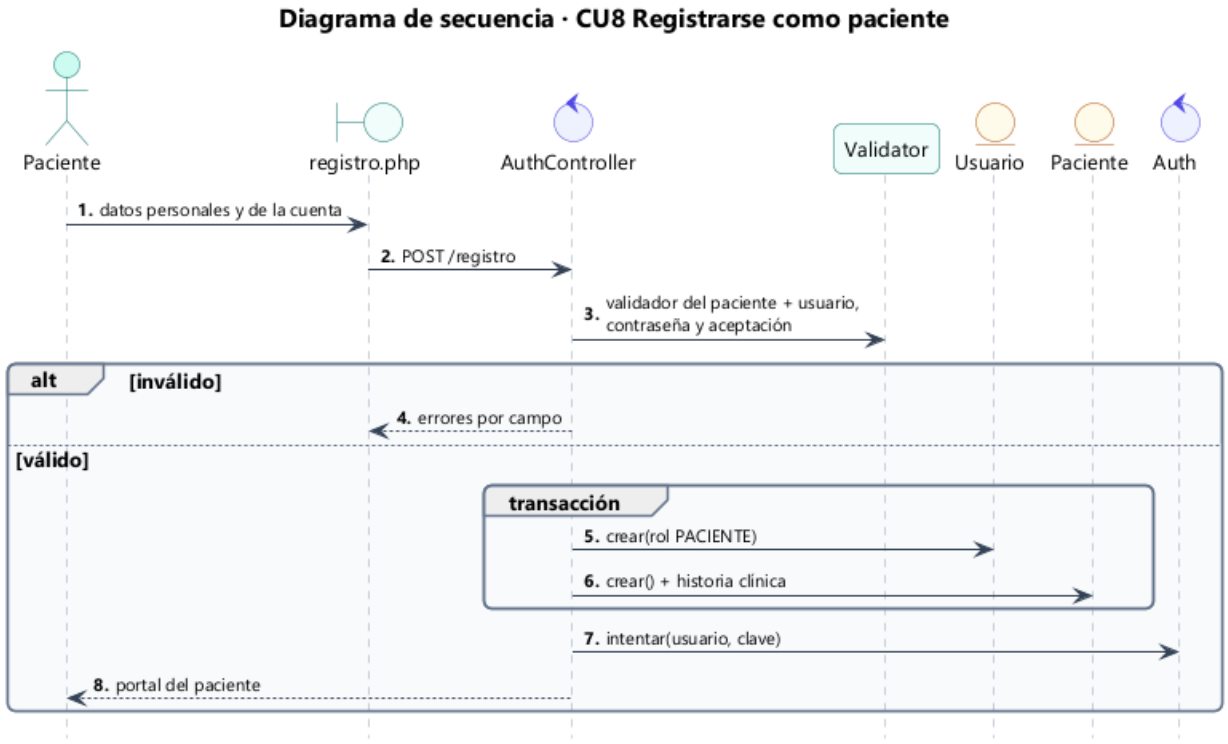


Figura 52. *Diagrama de secuencia de CU8 Registrarse como paciente*

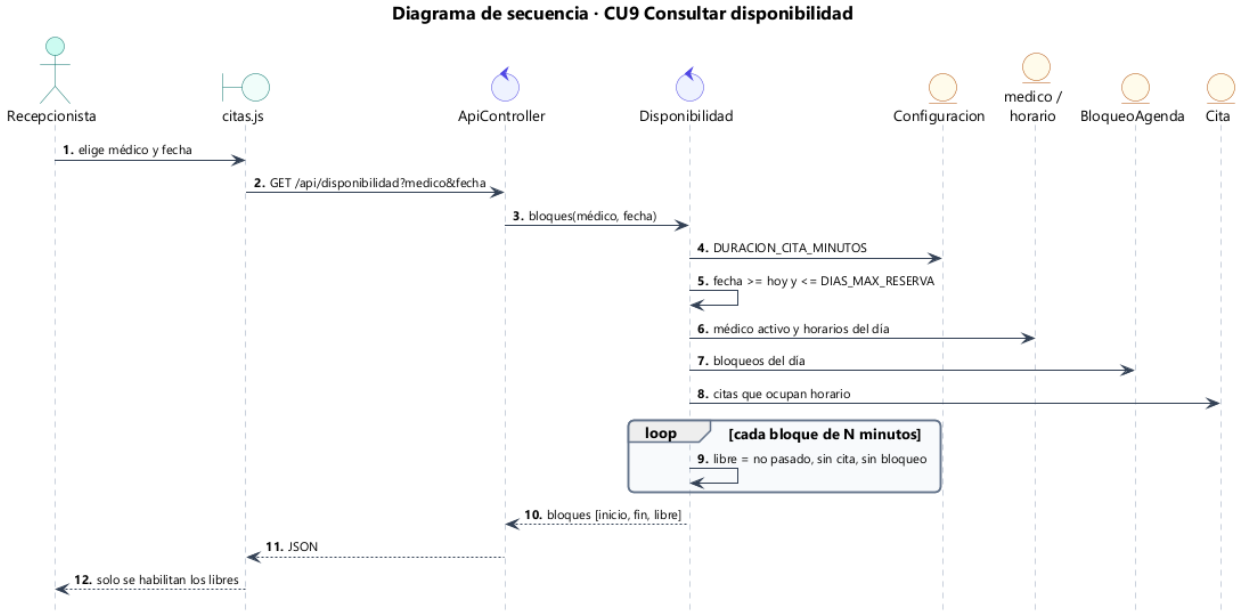


Figura 53. *Diagrama de secuencia de CU9 Consultar disponibilidad*

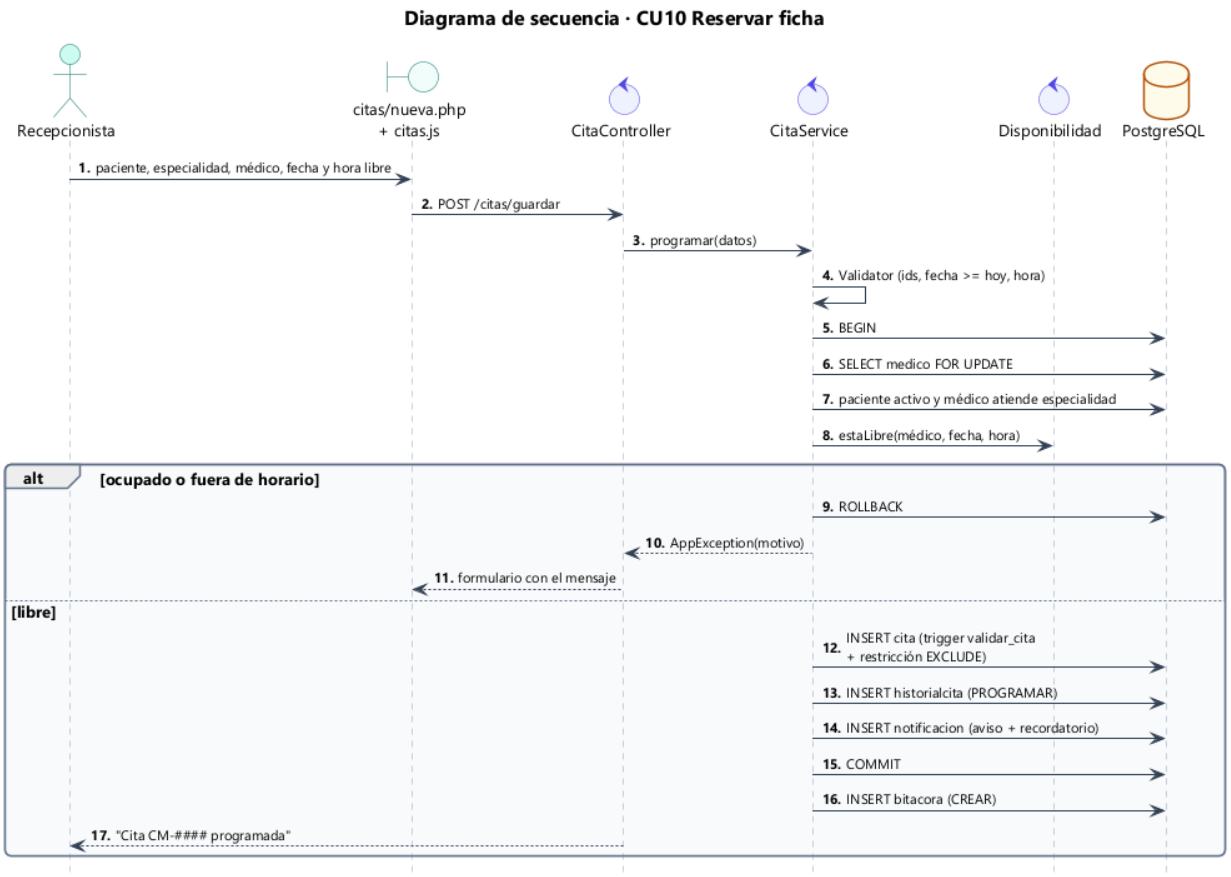


Figura 54. *Diagrama de secuencia de CU10 Reservar ficha*

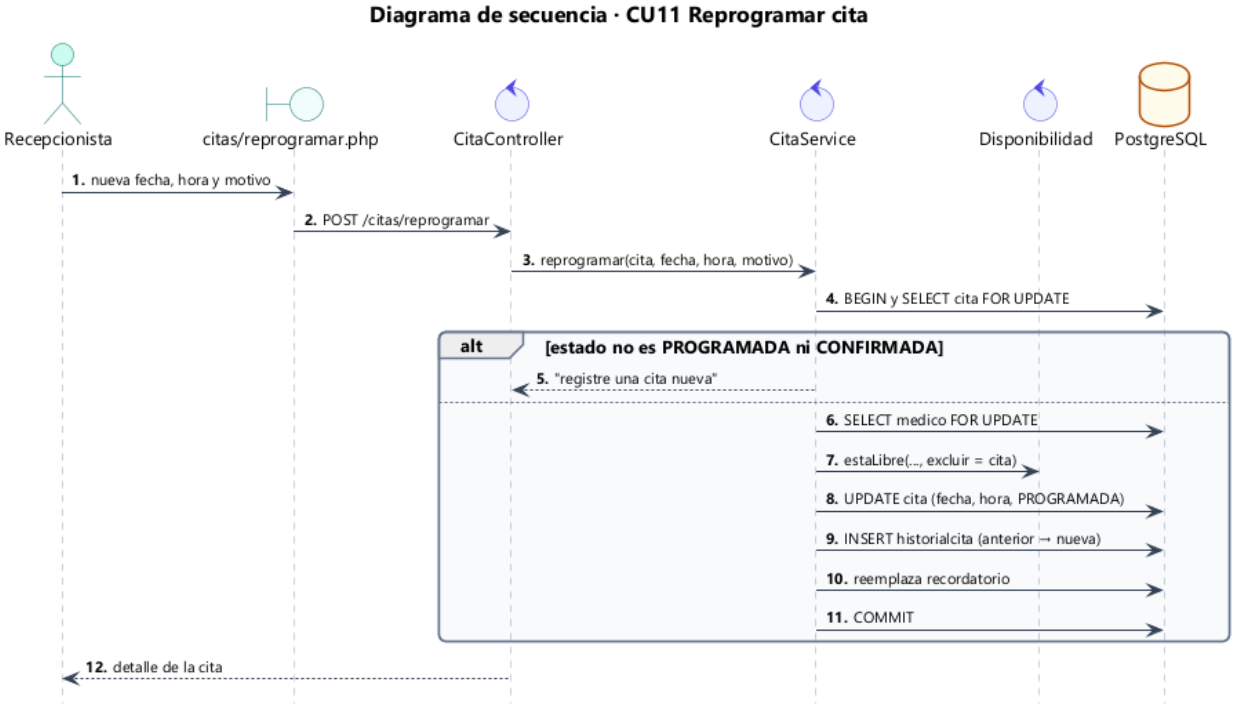


Figura 55. *Diagrama de secuencia de CU11 Reprogramar cita*

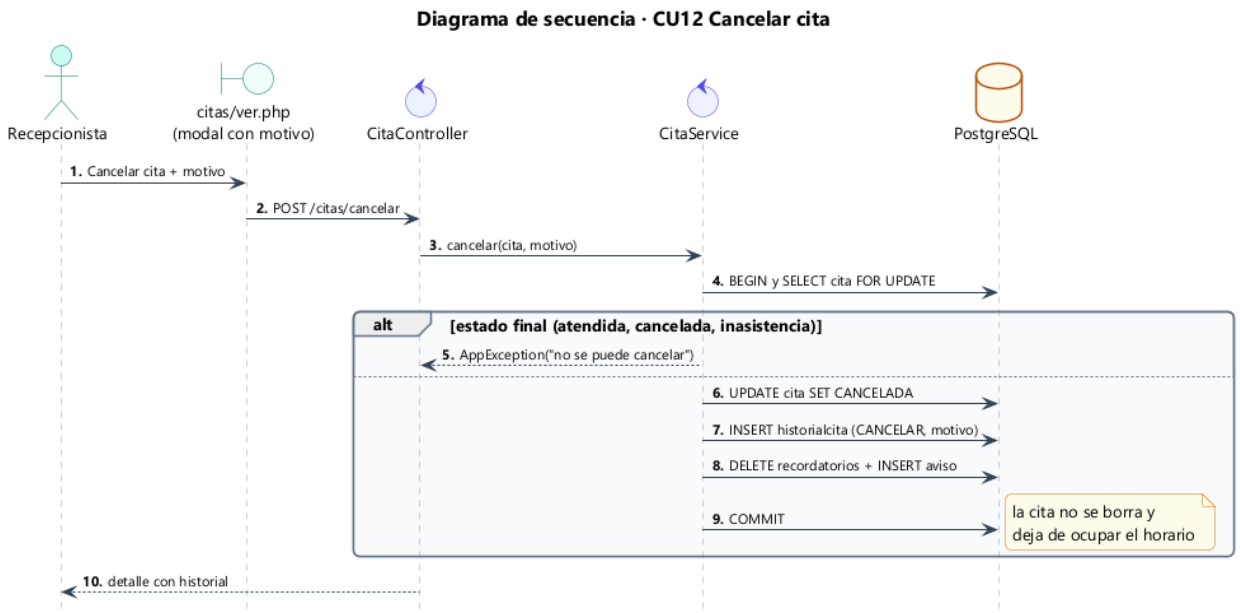


Figura 56. *Diagrama de secuencia de CU12 Cancelar cita*

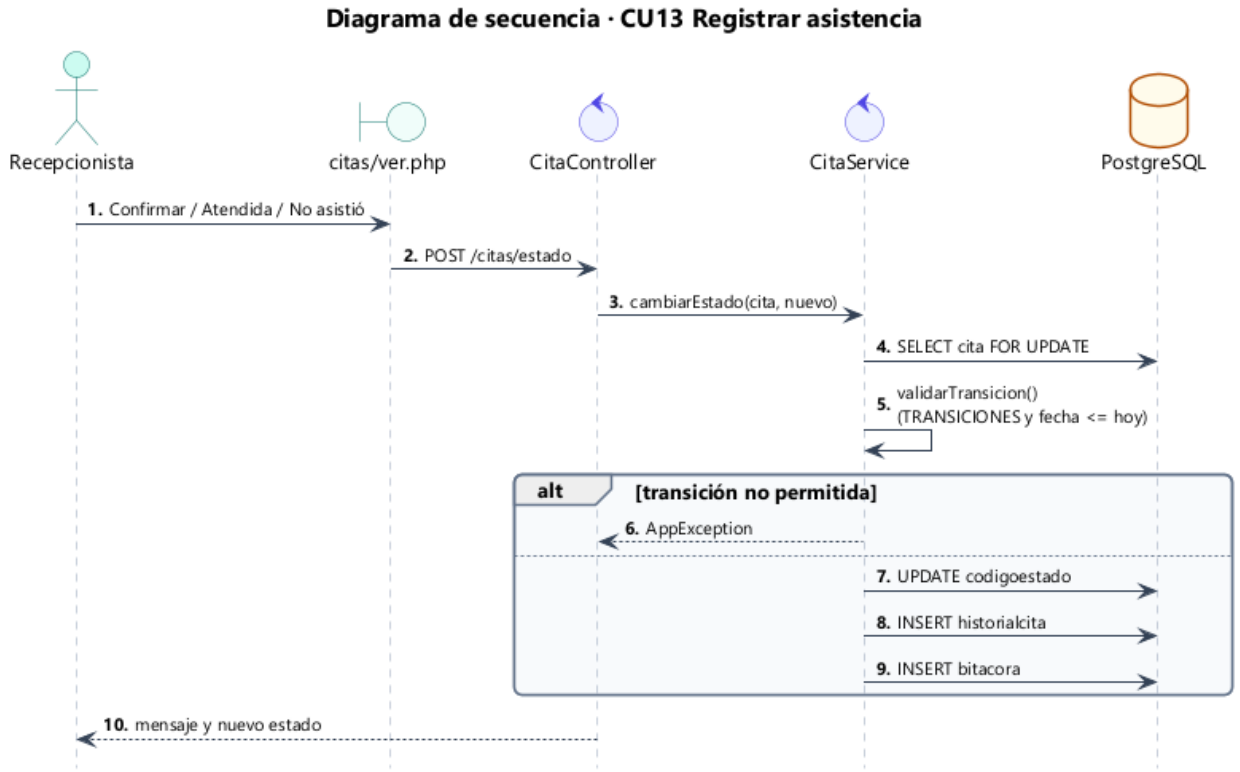


Figura 57. *Diagrama de secuencia de CU13 Registrar asistencia*

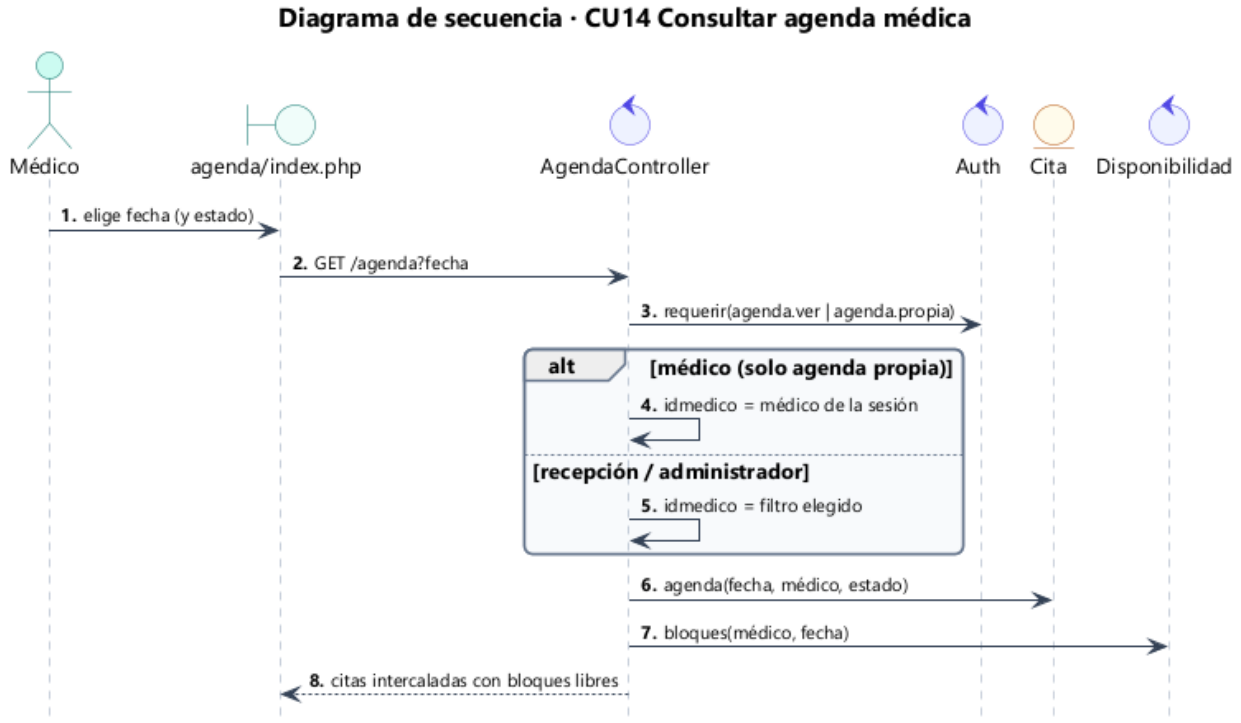


Figura 58. *Diagrama de secuencia de CU14 Consultar agenda médica*

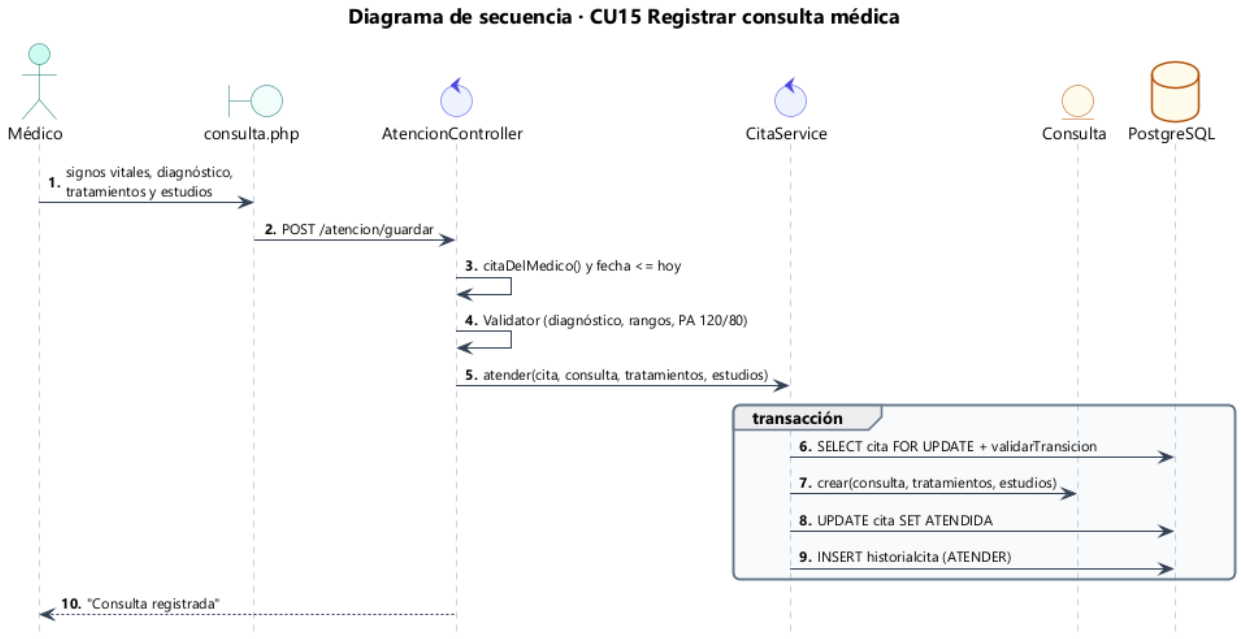


Figura 59. *Diagrama de secuencia de CU15 Registrar consulta médica*

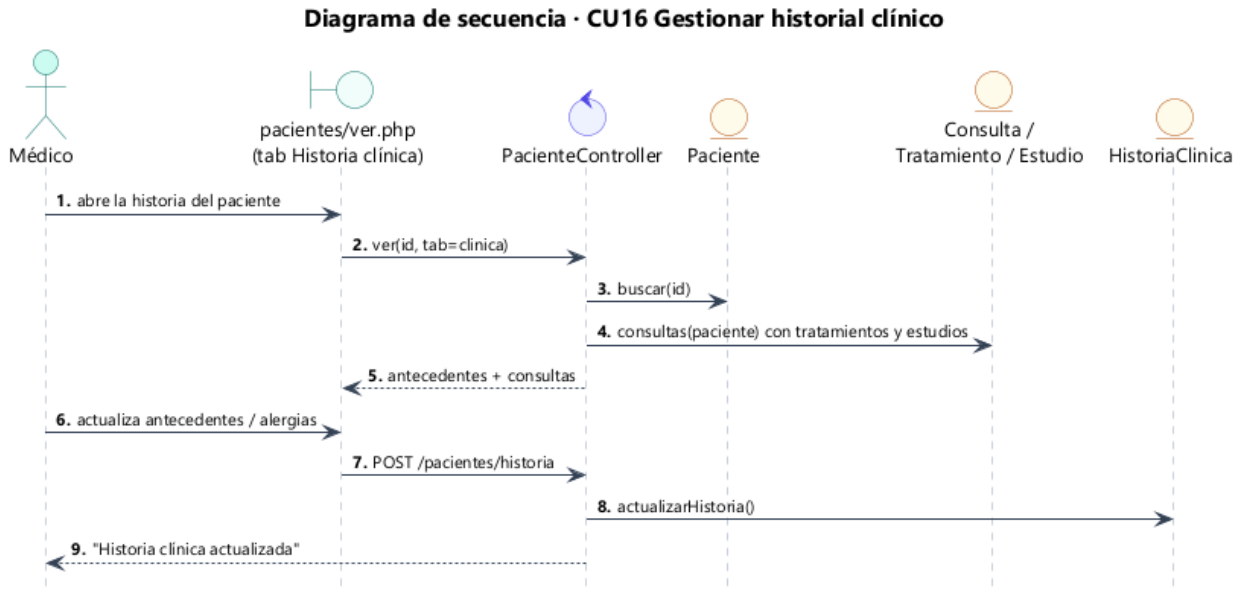


Figura 60. *Diagrama de secuencia de CU16 Gestionar historial clínico*

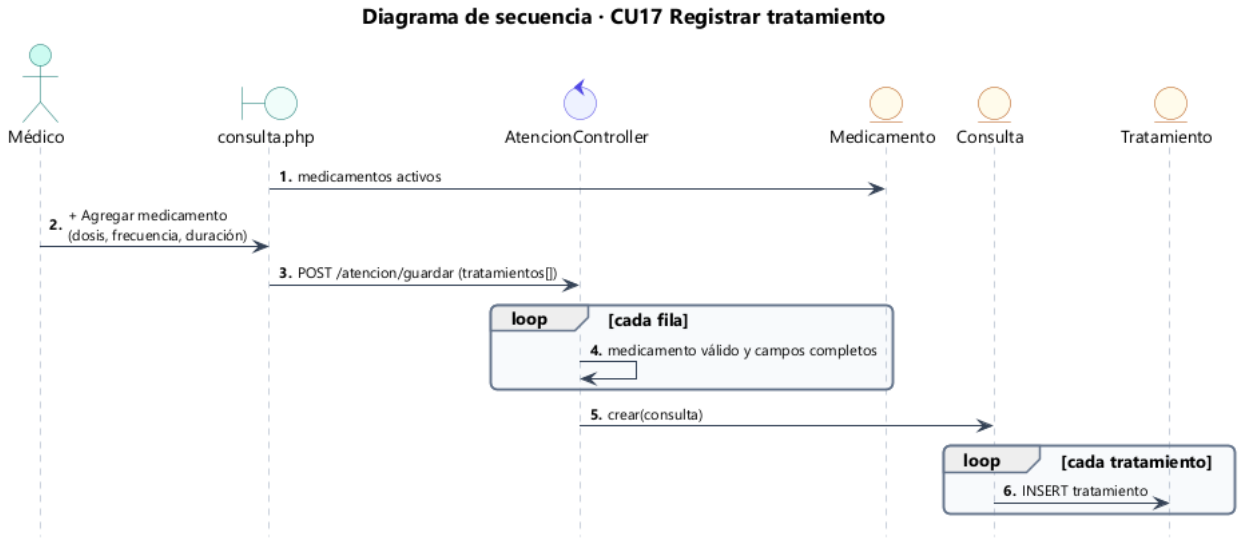


Figura 61. *Diagrama de secuencia de CU17 Registrar tratamiento*

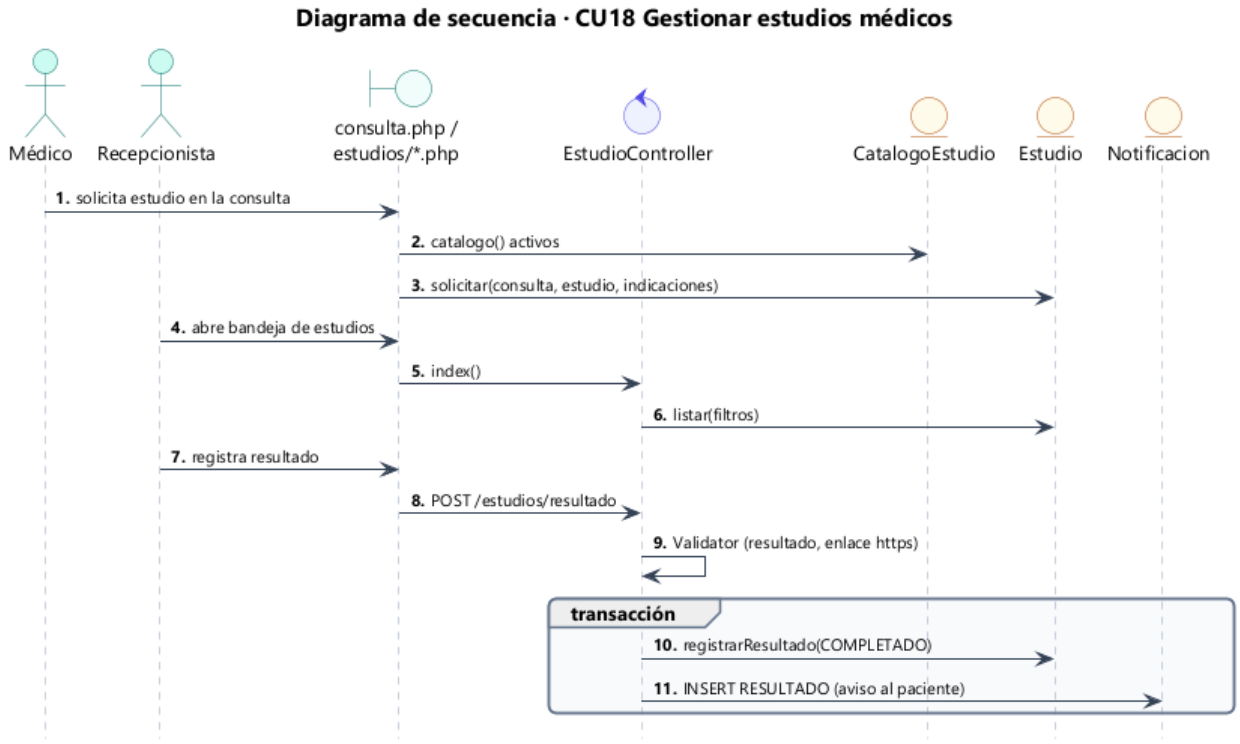


Figura 62. *Diagrama de secuencia de CU18 Gestionar estudios médicos*

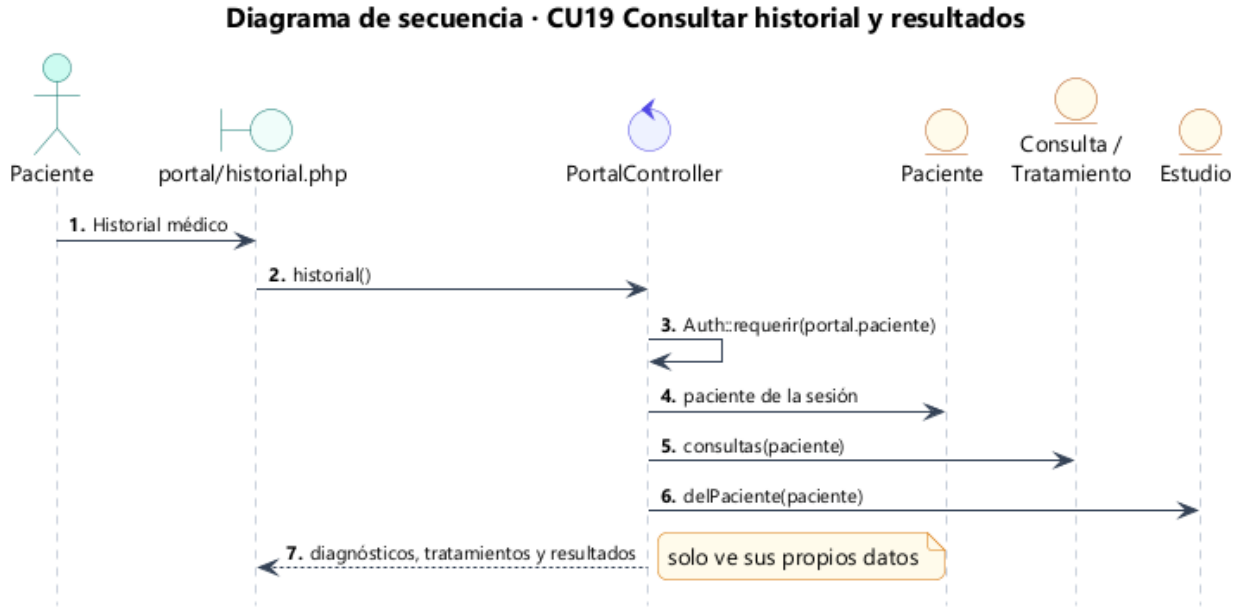


Figura 63. *Diagrama de secuencia de CU19 Consultar historial y resultados*

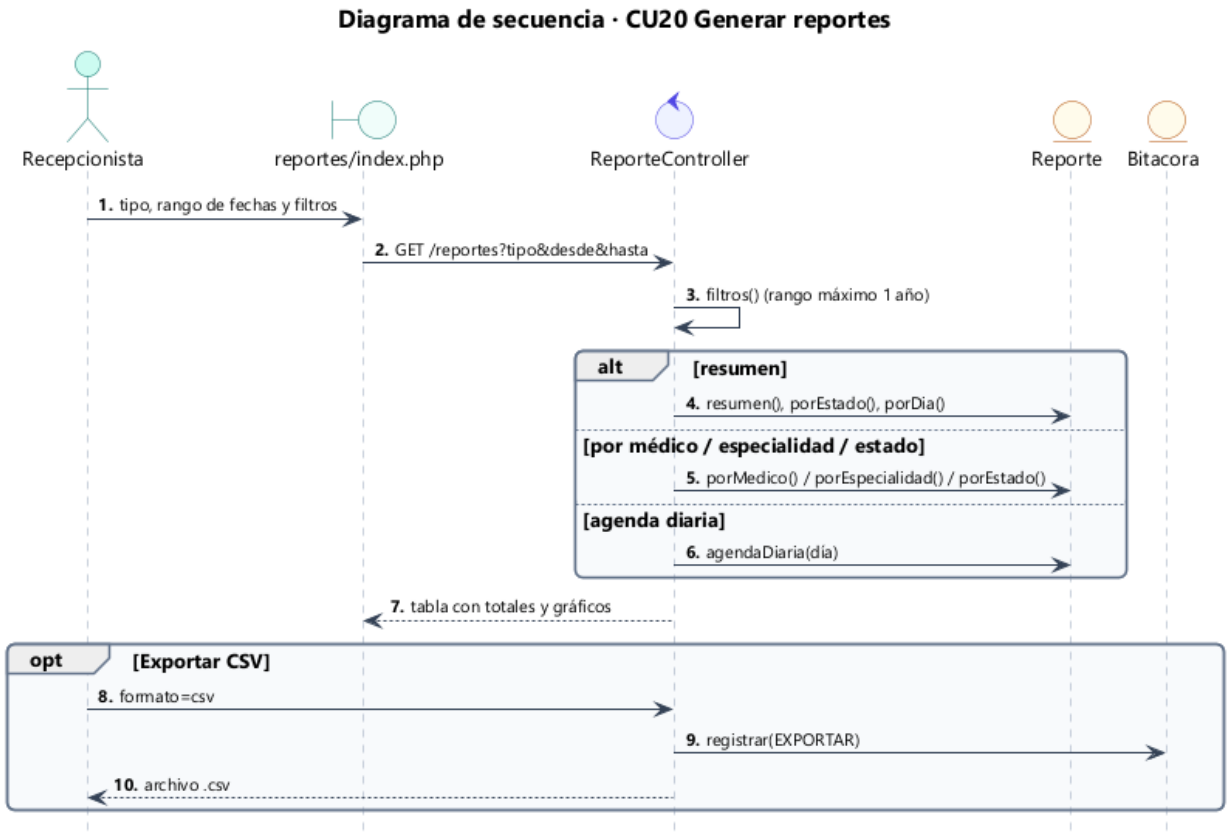


Figura 64. *Diagrama de secuencia de CU20 Generar reportes*

6.3 Diseño de datos

6.3.1 Diseño lógico

El modelo lógico parte del diagrama de clases del sistema y se normalizó hasta la tercera forma normal: cada tabla describe una sola entidad, las relaciones muchos a muchos usan tablas intermedias (MEDICOESPECIALIDAD, ROLPERMISO) y los catálogos (ESTADOCITA, ESPECIALIDAD, MEDICAMENTO, CATALOGOESTUDIO) evitan la repetición de datos.

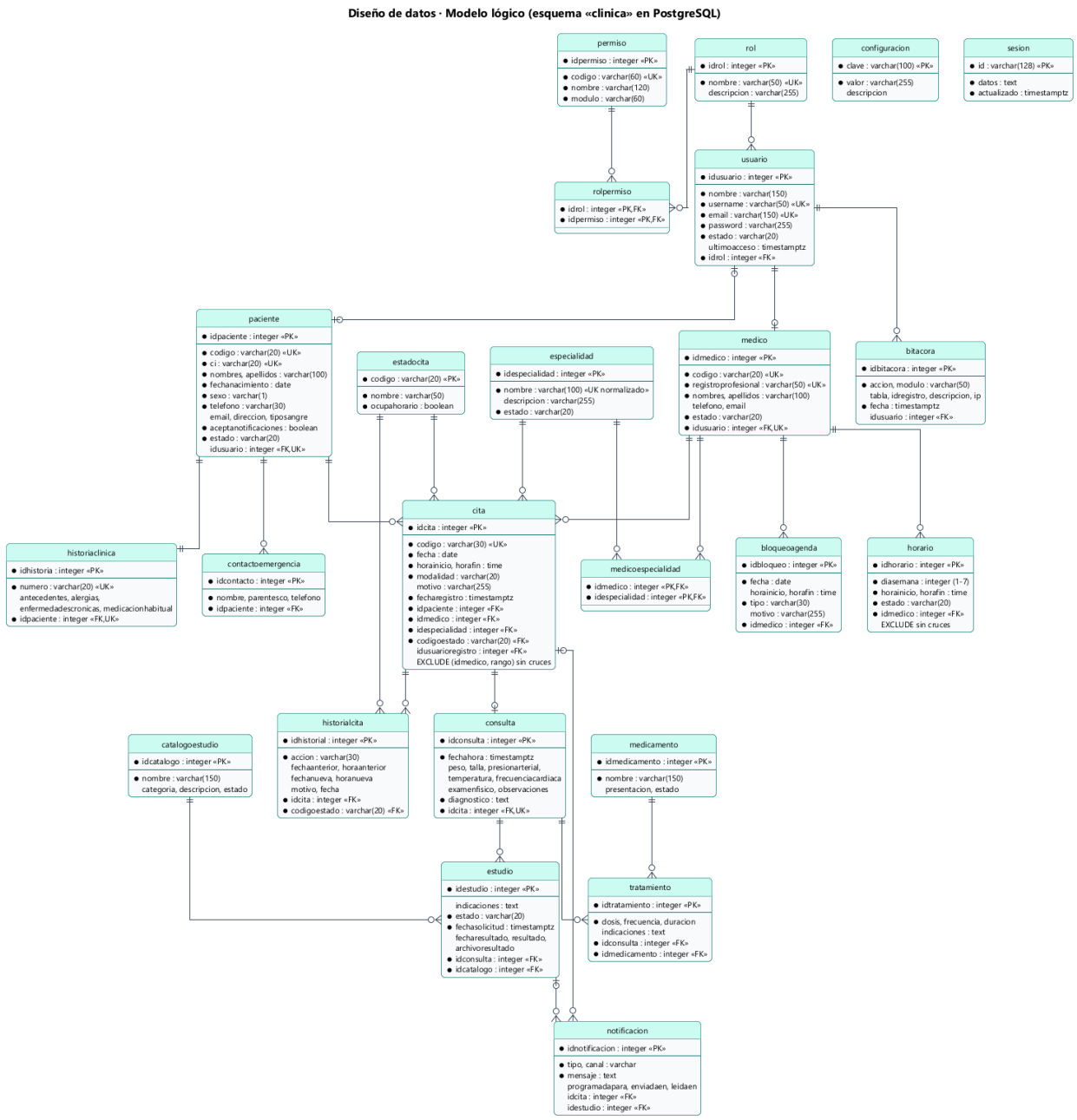


Figura 65. *Modelo lógico de datos (esquema clinica)*

6.3.2 Diseño físico

La base de datos se implementó en PostgreSQL 17, en el esquema `clinica`, con 24 tablas. Los scripts se encuentran en la carpeta `Bakend/database` del repositorio.

Tabla 32. *Scripts de la base de datos*

Script	Contenido
01_esquema.sql	Tablas, claves primarias y foráneas, UNIQUE, CHECK, índices y restricciones EXCLUDE.

Script	Contenido
02_reglas_negocio.sql	Trigger validar_cita (RN04 a RN08, RF08, RF10), protección de roles del sistema y seguridad a nivel de fila (RLS).
03_datos_iniciales.sql	Roles, permisos, configuración, estados, especialidades, médicos, horarios, pacientes, citas, consultas y estudios de demostración.
04_usuario_app.sql	Usuario de base de datos headsalud_app con privilegios mínimos para la aplicación.

Tabla 33. *Tablas del diseño físico*

Tabla	Descripción	Restricciones destacadas
rol, permiso, rolpermiso	Roles y permisos asignados	nombre y código únicos; PK compuesta
usuario	Cuentas de acceso	username y email únicos (sin distinguir mayúsculas); estado ACTIVO/INACTIVO
bitacora	Auditoría de operaciones	Índice por fecha
configuracion	Parámetros del sistema	clave como PK
sesion	Sesiones PHP persistentes	id como PK
especialidad	Especialidades	UNIQUE sobre normalizar(nombre)
medico, medicoespecialidad	Médicos y sus especialidades	código, registro e idusuario únicos
horario	Horarios semanales	CHECK horafin > horainicio; EXCLUDE sin cruces por médico y día
bloqueoagenda	Feriados, vacaciones y permisos	CHECK de rango de horas
paciente, historiaclinica, contactoemergencia	Pacientes y su información médica	CI y código únicos; sexo y tipo de sangre con CHECK
estadocita	Catálogo de estados	ocupahorario indica si el estado ocupa el horario
cita	Citas médicas	código único; EXCLUDE (idmedico, rango) y (idpaciente, rango) excepto canceladas
historialcita	Cambios de cada cita	FK a cita y estado
consulta, tratamiento, medicamento	Atención médica	una consulta por cita; CHECK de signos vitales
catalogoestudio, estudio	Estudios y resultados	CHECK del estado del estudio
notificacion	Avisos y recordatorios	FK a cita y estudio

Fragmento del diseño físico de la tabla CITA, con la restricción que impide citas superpuestas para el mismo médico:

```
create table cita (
```

```
    idcita          integer generated by default as identity primary key,
    codigo          varchar(30) not null unique default 'CM-' ||
lpad(nextval('cita_codigo_seq')::text, 4, '0'),
    fecha           date not null,
    horainicio      time not null,
    horafin         time not null,
    idpaciente      integer not null references paciente(idpaciente),
    idmedico        integer not null references medico(idmedico),
    codigoestado    varchar(20) not null default 'PROGRAMADA' references estadocita(codigo),
    constraint cita_sin_cruce_medico exclude using gist (
        idmedico with =, tsrange(fecha + horainicio, fecha + horafin) with &&
    ) where (codigoestado <> 'CANCELADA')
);
```

6.4 Trazabilidad

Cada modelo se deriva del anterior y mantiene una relación de trazabilidad «trace»: el caso de uso se realiza en clases de análisis, estas en clases de diseño, que se despliegan en los nodos y se implementan en el código.

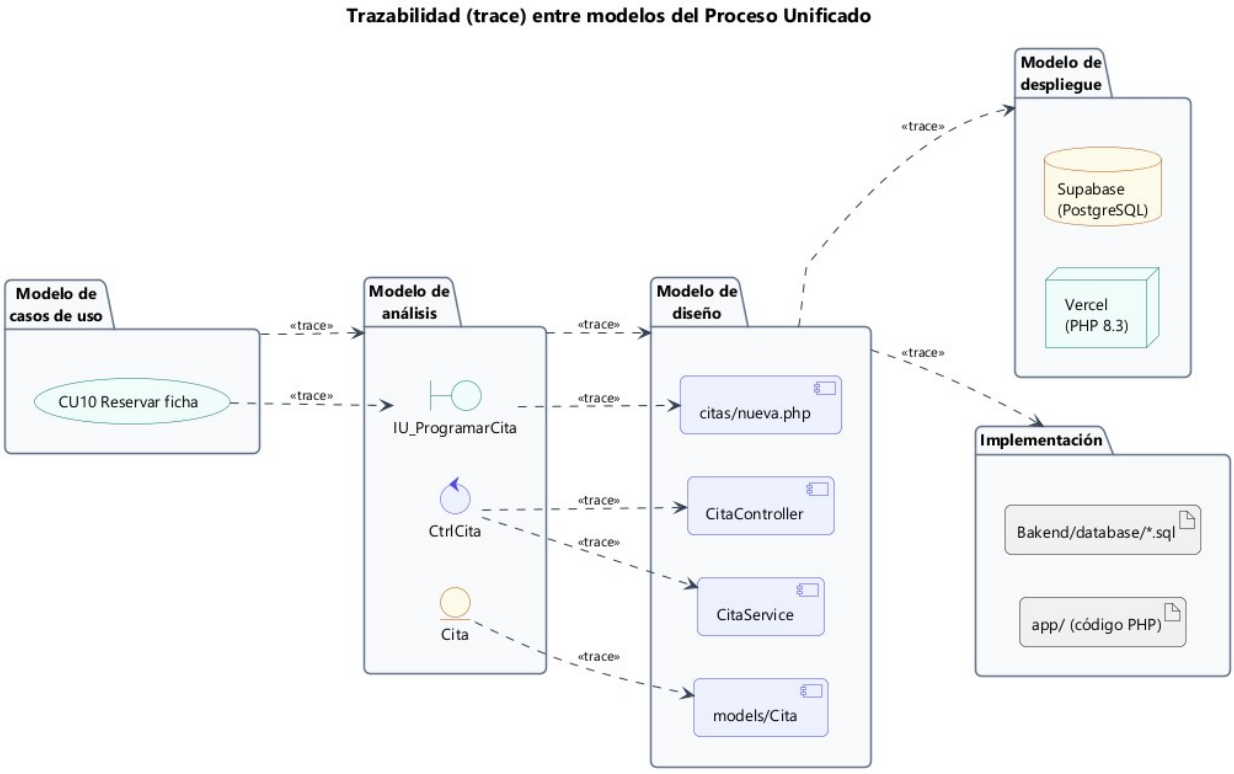


Figura 66. *Trazabilidad entre los modelos del Proceso Unificado*

Tabla 34. *Matriz de trazabilidad*

Caso de uso	Clases de análisis	Clases de diseño (archivos)	Nodo
CU1 Iniciar sesión	IU_Login, CtrlSesion, Usuario	login.php, AuthController, core/Auth, core/Session	Vercel · Supabase
CU7 Gestionar	IU_Pacientes, CtrlPaciente,	pacientes/*.php, PacienteController,	Vercel ·

Caso de uso	Clases de análisis	Clases de diseño (archivos)	Nodo
pacientes	Paciente	models/Paciente	Supabase
CU9 Consultar disponibilidad	IU_ProgramarCita, CtrlDisponibilidad	services/Disponibilidad, ApiController, citas.js	Vercel · Supabase
CU10 Reservar ficha	IU_ProgramarCita, CtrlCita, Cita	citas/nueva.php, CitaController, CitaService, trigger validar_cita	Vercel · Supabase
CU12 Cancelar cita	IU_DetalleCita, CtrlCita	citas/ver.php, CitaService::cancelar	Vercel · Supabase
CU15 Registrar consulta	IU_Consulta, CtrlAtencion, Consulta	consulta.php, AtencionController, models/Consulta	Vercel · Supabase
CU18 Gestionar estudios	IU_Estudios, CtrlEstudio, Estudio	estudios/*.php, EstudioController, models/Estudio	Vercel · Supabase
CU20 Generar reportes	IU_Reportes, CtrlReporte	reportes/index.php, ReporteController, models/Reporte	Vercel · Supabase

7. FT IMPLEMENTACIÓN

7.1 Herramientas de desarrollo de la aplicación web

Tabla 35. *Herramientas de desarrollo*

Herramienta	Versión	Uso en el proyecto
PHP	8.3	Lenguaje del servidor (MVC propio, sin framework).
PostgreSQL	17	Sistema gestor de base de datos.
PDO (pdo_pgsql)	—	Acceso a datos con consultas parametrizadas.
HTML5 · CSS3 · JavaScript	—	Interfaz, diseño adaptable, animaciones y validaciones.
Supabase	—	PostgreSQL administrado en la nube (región São Paulo).
Vercel + vercel-php	0.7.4	Publicación del sistema como función serverless de PHP 8.3.
Gmail SMTP	—	Envío del código de verificación para recuperar la contraseña.
Laragon / XAMPP	—	Servidor local para desarrollo y pruebas.
Visual Studio Code	—	Editor de código.
Git y GitHub	—	Control de versiones y publicación del código fuente.
PlantUML y draw.io	1.2024.8	Elaboración de los diagramas UML.
Google Chrome	—	Navegador de pruebas.

7.2 Implementación de la arquitectura del sistema

El código se organiza en carpetas que reflejan la arquitectura lógica:

```
headsalud/
├── api/index.php           Controlador frontal y tabla de rutas
├── app/
│   ├── bootstrap.php       Arranque: entorno, errores, autoload, sesión
│   ├── core/               Router, Auth, Csrf, Session, Validator, View, Database
│   ├── controllers/        Un controlador por caso de uso o módulo
│   ├── models/             Acceso a datos (SQL parametrizado)
│   ├── services/           CitaService y Disponibilidad (reglas de negocio)
│   └── views/              P1_SeguridadAcceso … P6_Reportes, Portal, layouts
├── public/assets/          CSS y JavaScript
├── Bakend/database/        Scripts SQL de PostgreSQL
├── Docs/                   Diagramas, capturas y documentación
├── router.php · .htaccess  Ejecución local (PHP integrado / Apache)
└── vercel.json             Despliegue en Vercel
```

7.2.1 Flujo de una petición

1. Toda petición llega a `api/index.php` (en Vercel por `vercel.json`, en Apache por `.htaccess`).

2. `bootstrap.php` carga la configuración desde variables de entorno, registra el manejador de errores, la carga automática de clases y la sesión persistida en la tabla SESION.
3. El `Router` busca la ruta y, si es POST, verifica el token CSRF.
4. El controlador exige el permiso con `Auth::requerir()`, valida los datos con `Validator` y llama a los servicios o modelos.
5. Los modelos ejecutan SQL parametrizado mediante `Database` (PDO).
6. El controlador responde con una vista (patrón Post/Redirect/Get con mensajes flash) o con JSON.

7.2.2 Seguridad implementada

Las contraseñas se guardan con `password_hash()` y se verifican con `password_verify()`:

```
// app/models/Usuario.php
'password' => password_hash($d['password'], PASSWORD_DEFAULT),

// app/core/Auth.php
if (!$u || !password_verify($password, $u['password'])) {
    Bitacora::registrar('LOGIN_FALLIDO', 'SEGURIDAD', 'usuario', ...);
    throw new AppException('Usuario o contraseña incorrectos.');
}
session_regenerate_id(true);   // evita la fijación de sesión
```

Todas las consultas son parametrizadas; los datos del usuario nunca se concatenan al SQL:

```
// app/models/Paciente.php · búsqueda por CI, nombre o apellido (RF02)
$w[] = "(p.ci ilike :q or p.nombres || ' ' || p.apellidos ilike :q)";
$p['q'] = '%' . $f['q'] . '%';
Database::filas("select ... from paciente p $where ...", $p);
```

La programación de una cita se ejecuta en una transacción que bloquea al médico y vuelve a verificar la disponibilidad (RF07, RN13):

```
// app/services/CitaService.php
Database::transaccion(function () use ($d, $idmedico) {
    self::bloquearMedico($idmedico);              // SELECT ... FOR UPDATE
    self::exigirDisponible($idmedico, $d['fecha'], $d['hora']);
    $cita = Database::fila('insert into cita (...) values (...) returning ...', [...]);
    self::historial($cita['idcita'], 'PROGRAMAR', 'PROGRAMADA', ...);
    Notificacion::recordatorio($cita);
});
```

Tabla 36. *Mecanismos de seguridad*

Amenaza	Mecanismo
Inyección SQL	PDO con parámetros en el 100% de las consultas.
XSS	Toda salida escapada con e() (htmlspecialchars).
CSRF	Token por sesión en cada formulario POST (hash_equals).

Amenaza	Mecanismo
Robo de sesión	Cookies HttpOnly, SameSite=Lax, Secure; regeneración del ID; expiración por inactividad.
Fuerza bruta	Bloqueo tras 5 intentos fallidos por IP en 10 minutos.
Recuperación de cuenta	Código de 6 dígitos por correo, guardado con password_hash, de un solo uso, vence en 15 minutos, 5 intentos, límite de solicitudes y sin revelar si el correo existe.
Acceso no autorizado	Permisos por rol verificados en el servidor en cada petición (RN12).
Exposición de errores	En producción se registran en el log y el usuario ve un mensaje genérico.
Exposición de credenciales	Variables de entorno; nunca se suben al repositorio.

7.3 Implementación de la arquitectura de los subsistemas

A continuación se muestran las pantallas implementadas de cada subsistema (paquete), con su correspondencia a los casos de uso.

P1 · Seguridad y acceso



Figura 67. *CU1 · Inicio de sesión*



Figura 68. *CU1 · Credenciales incorrectas*

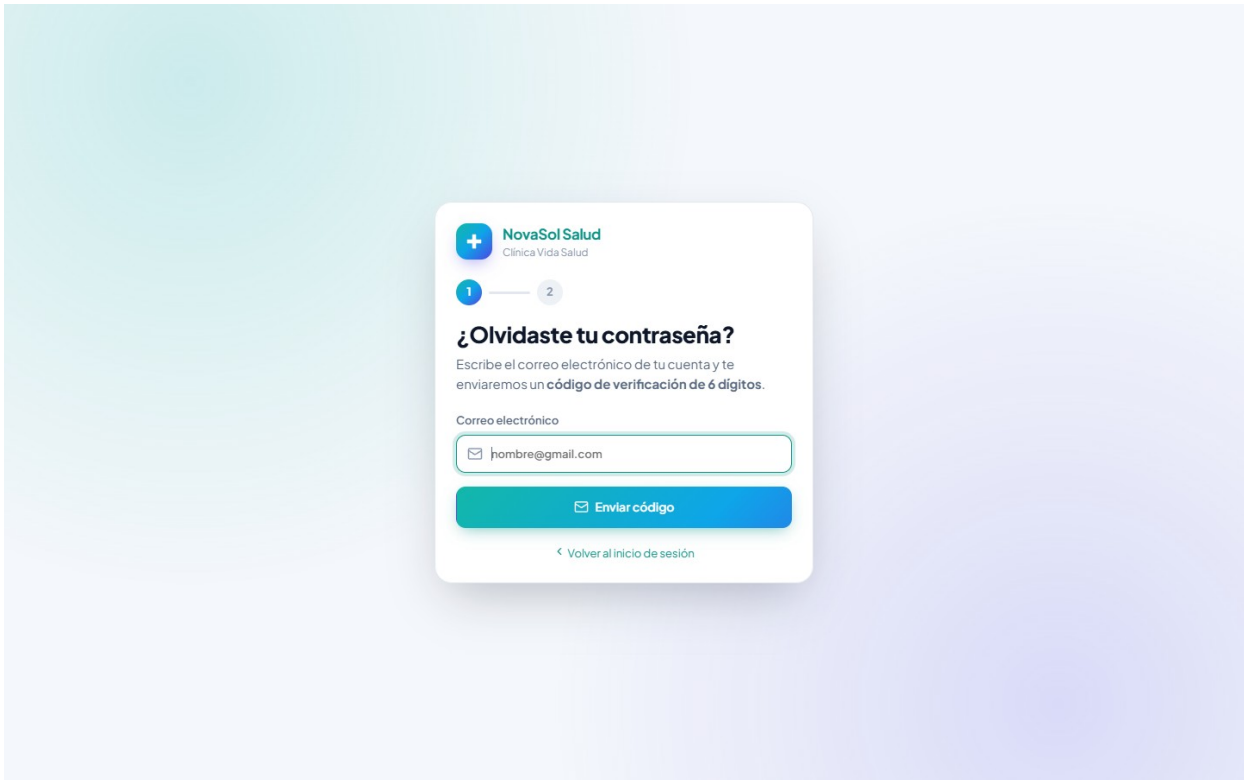


Figura 69. *CU1 · Recuperar contraseña: solicitud del código*

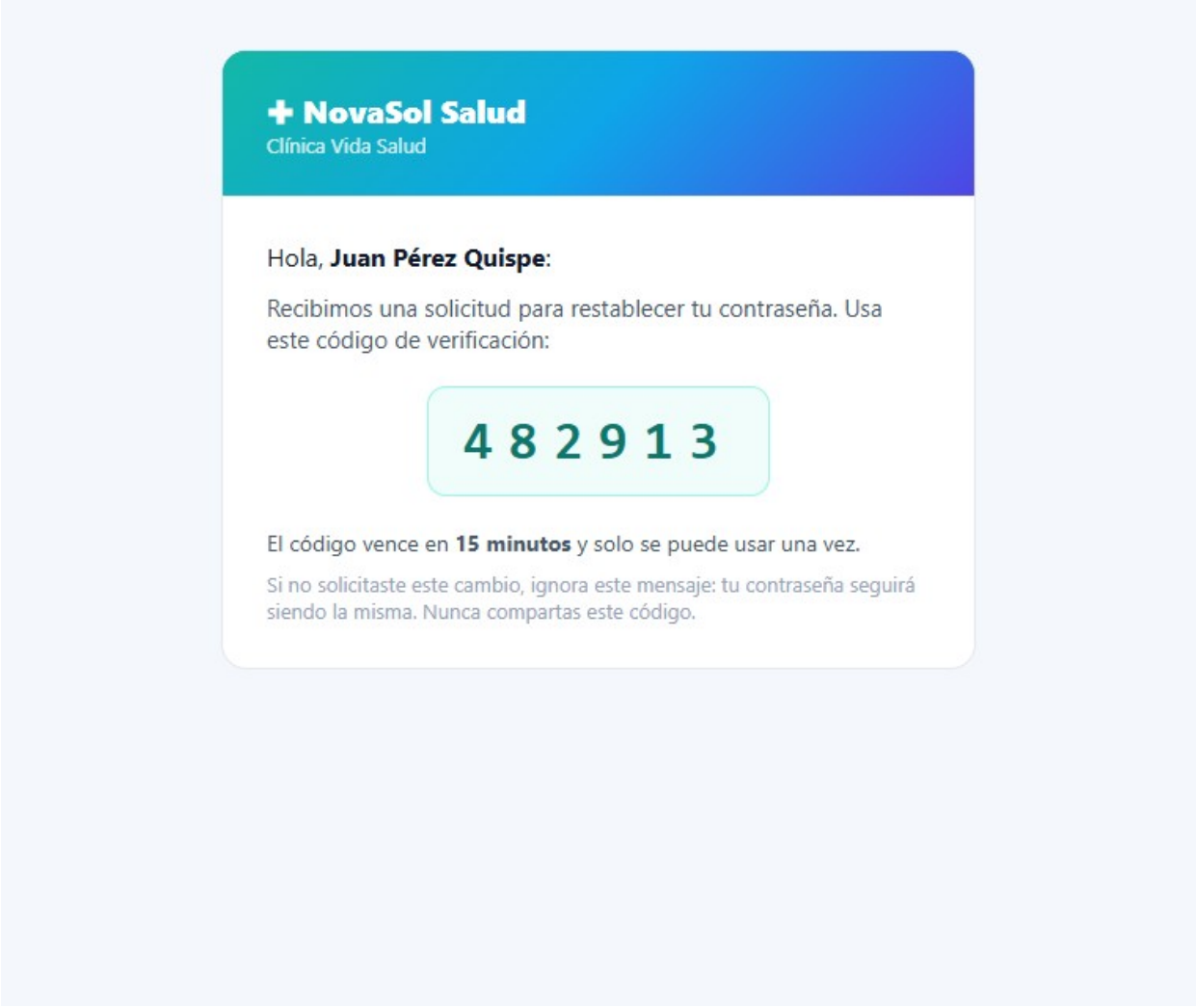


Figura 70. *CU1 · Correo con el código de verificación*

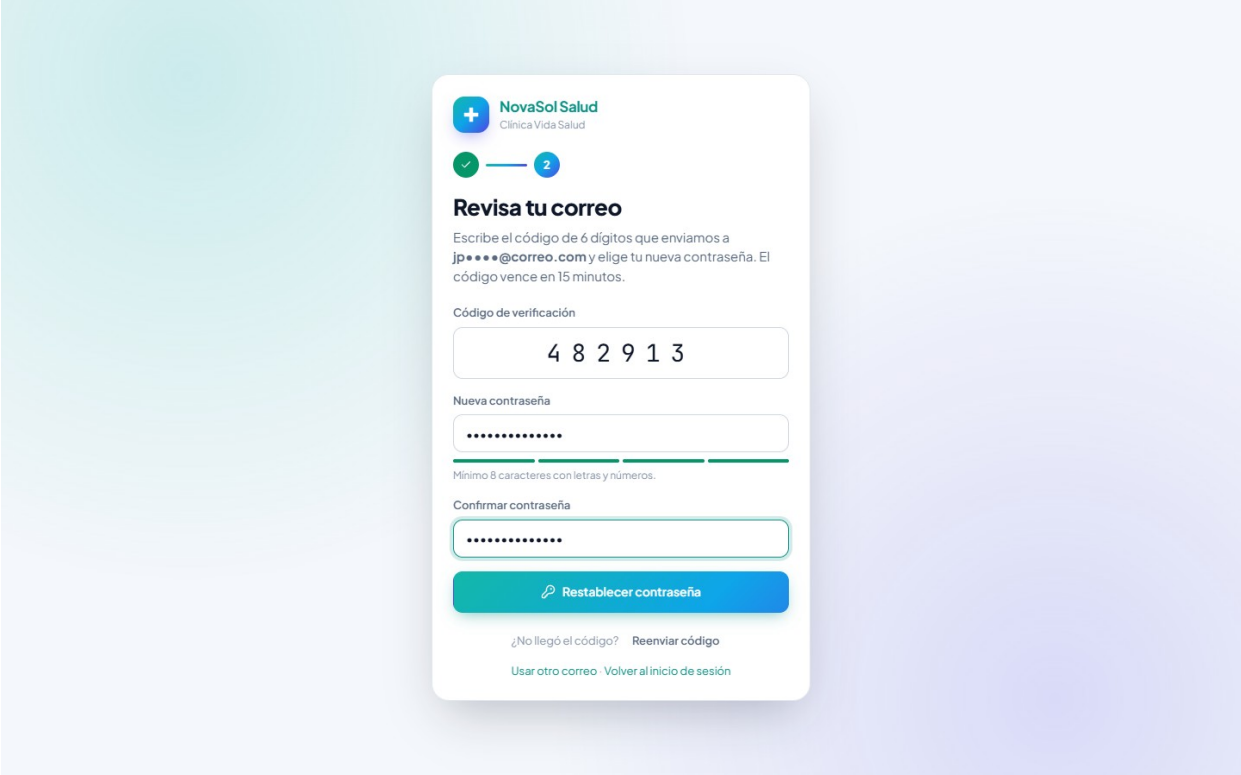


Figura 71. *CU1 · Recuperar contraseña: código y nueva contraseña*

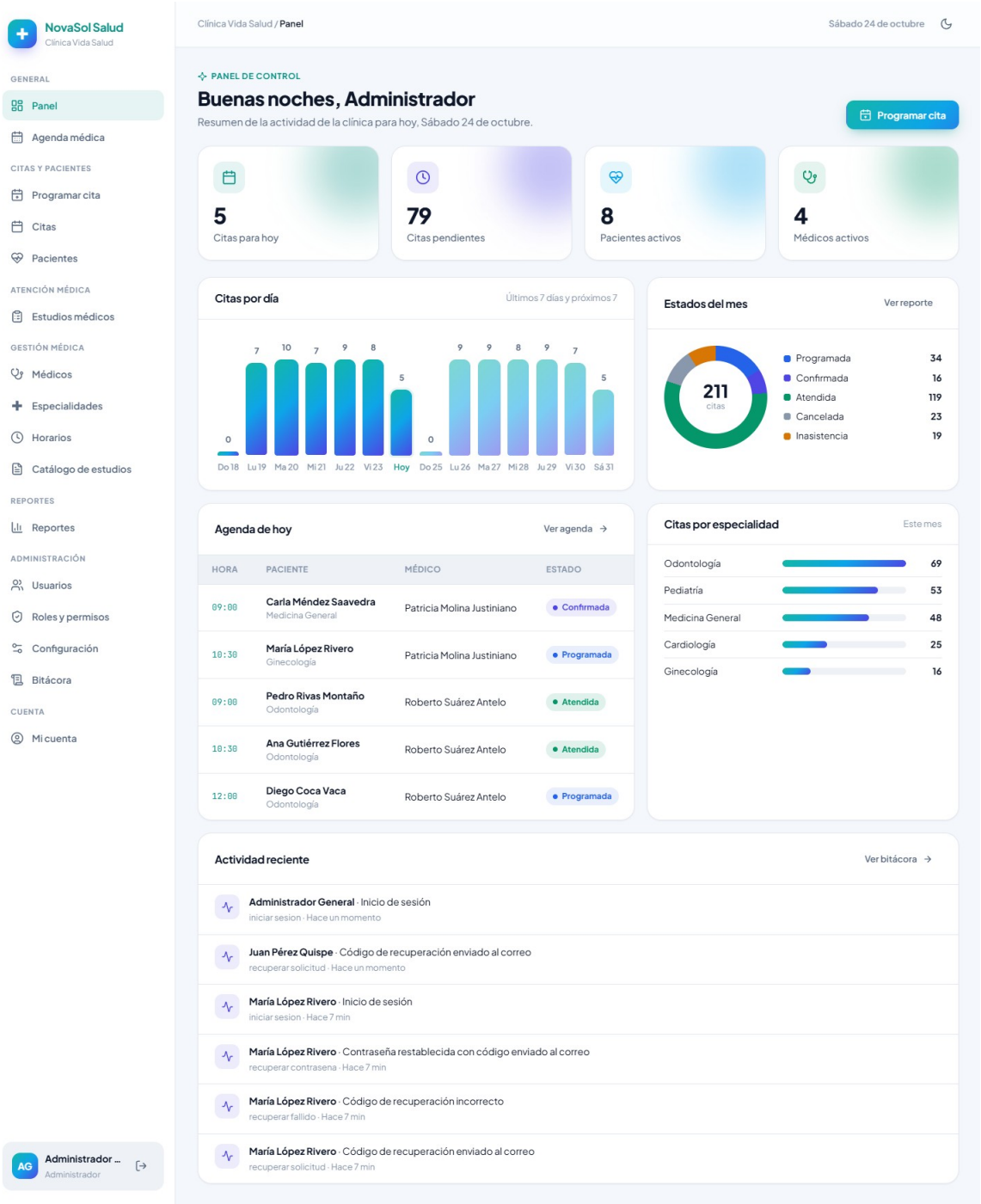


Figura 72. *Panel principal del administrador*

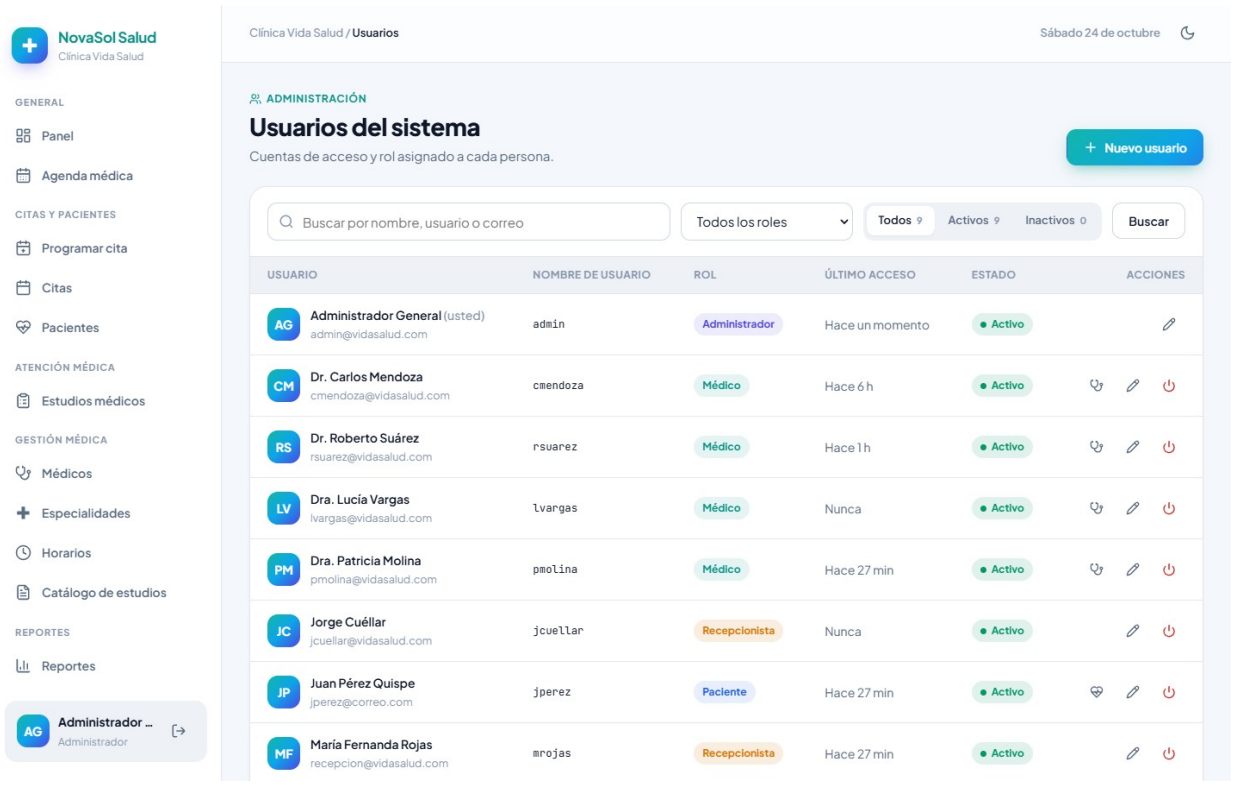


Figura 73. *CU3 · Listado de usuarios*

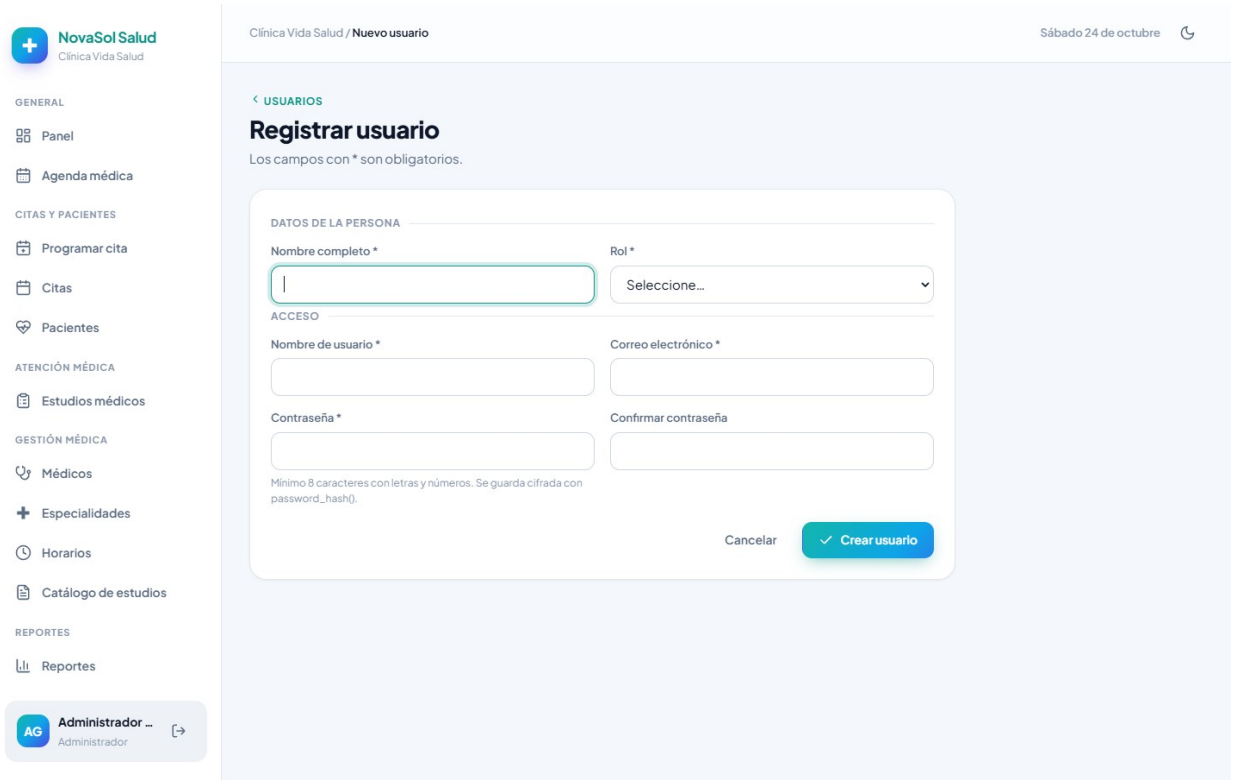


Figura 74. *CU3 · Registro de usuario*

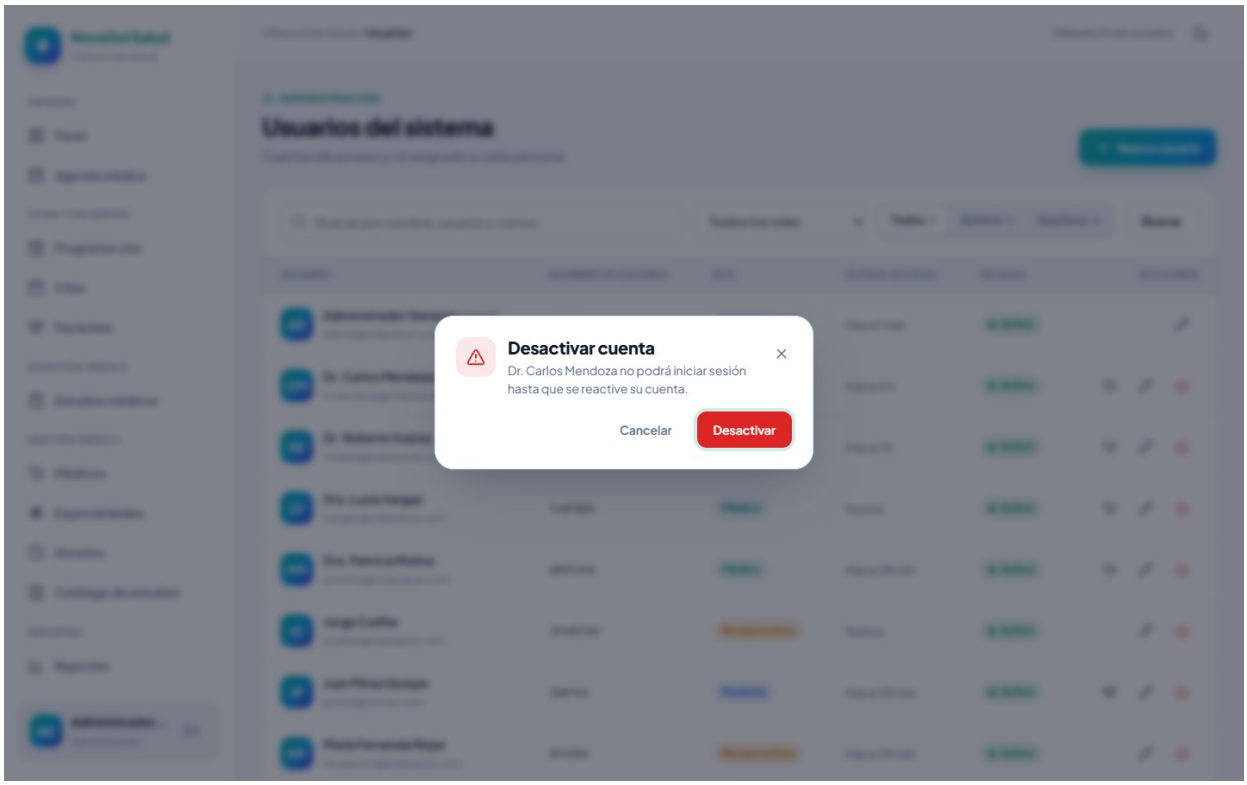


Figura 75. *CU3 · Confirmación para desactivar una cuenta*

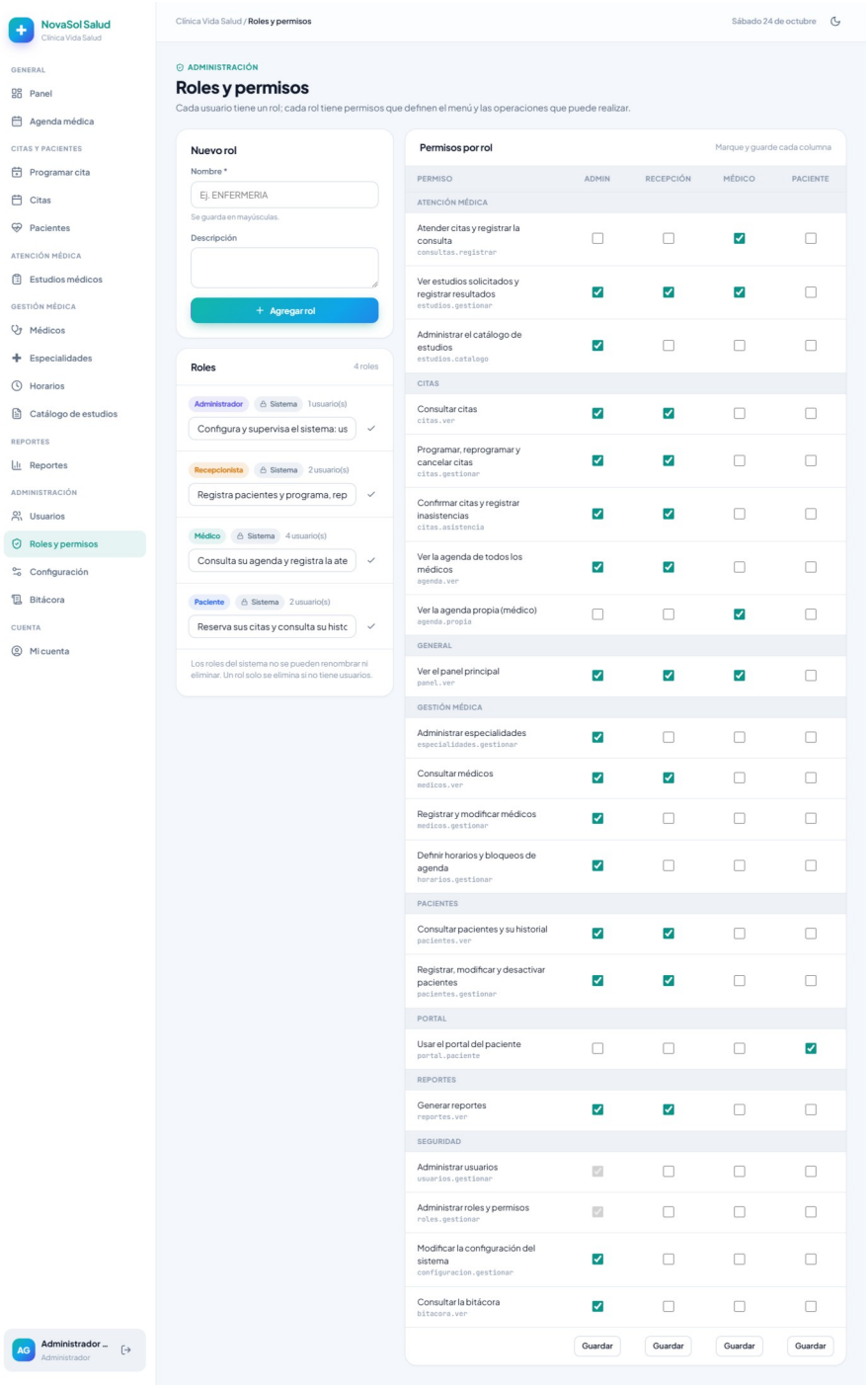


Figura 76. *CU2 · Roles y matriz de permisos*

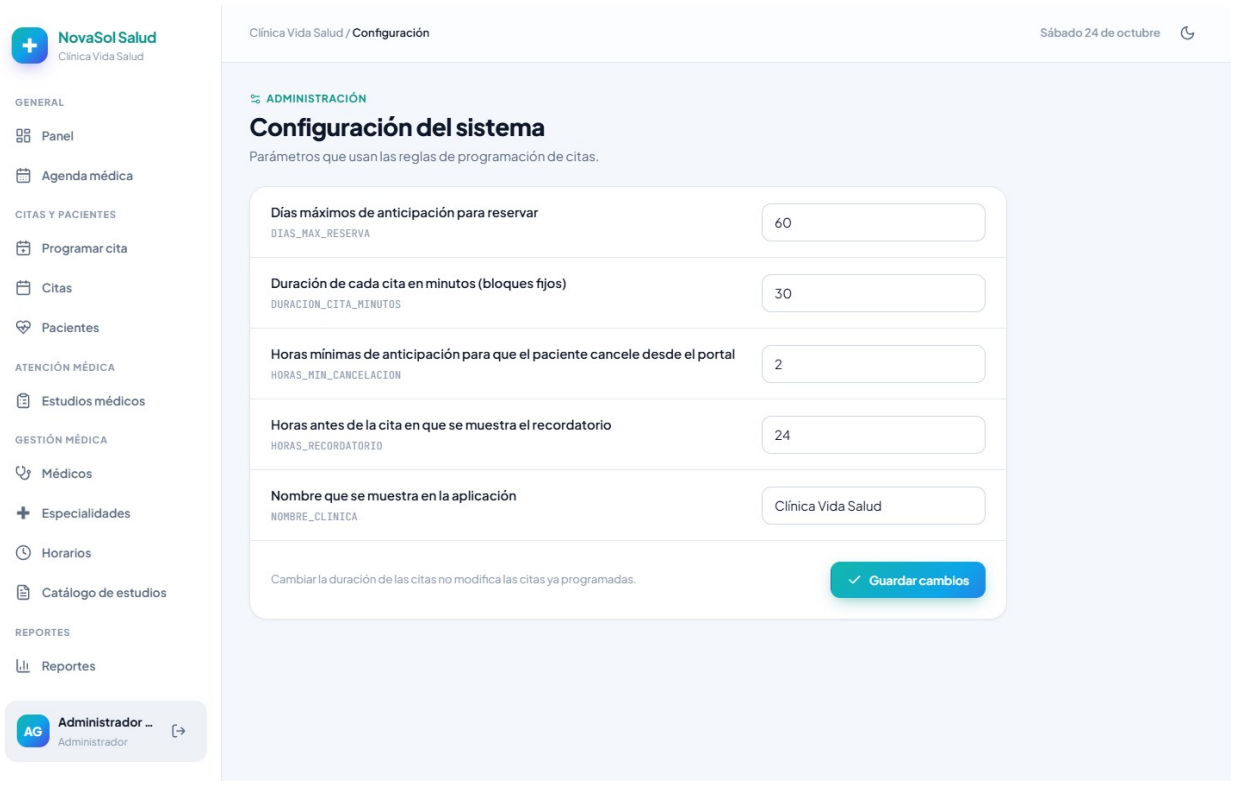


Figura 77. *CU3 · Configuración del sistema*

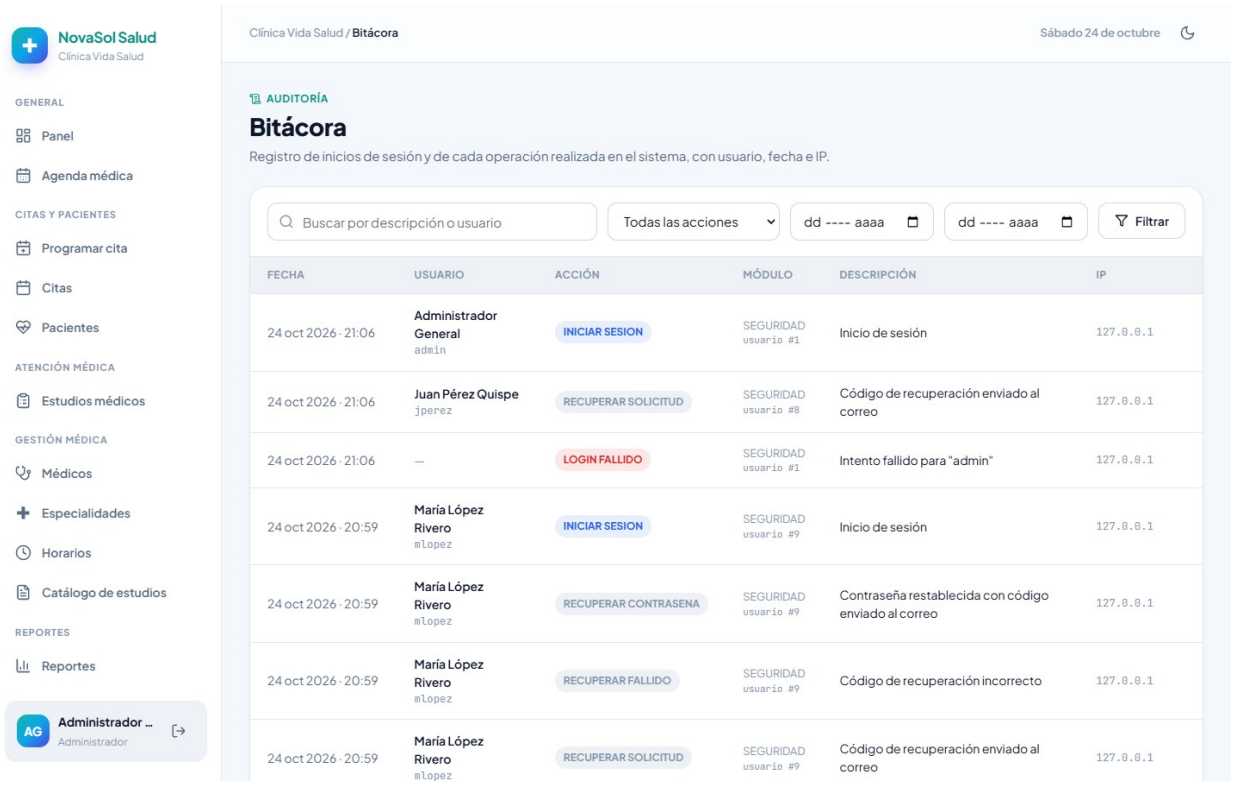


Figura 78. *CU3 · Bitácora de auditoría*

P2 · Gestión médica

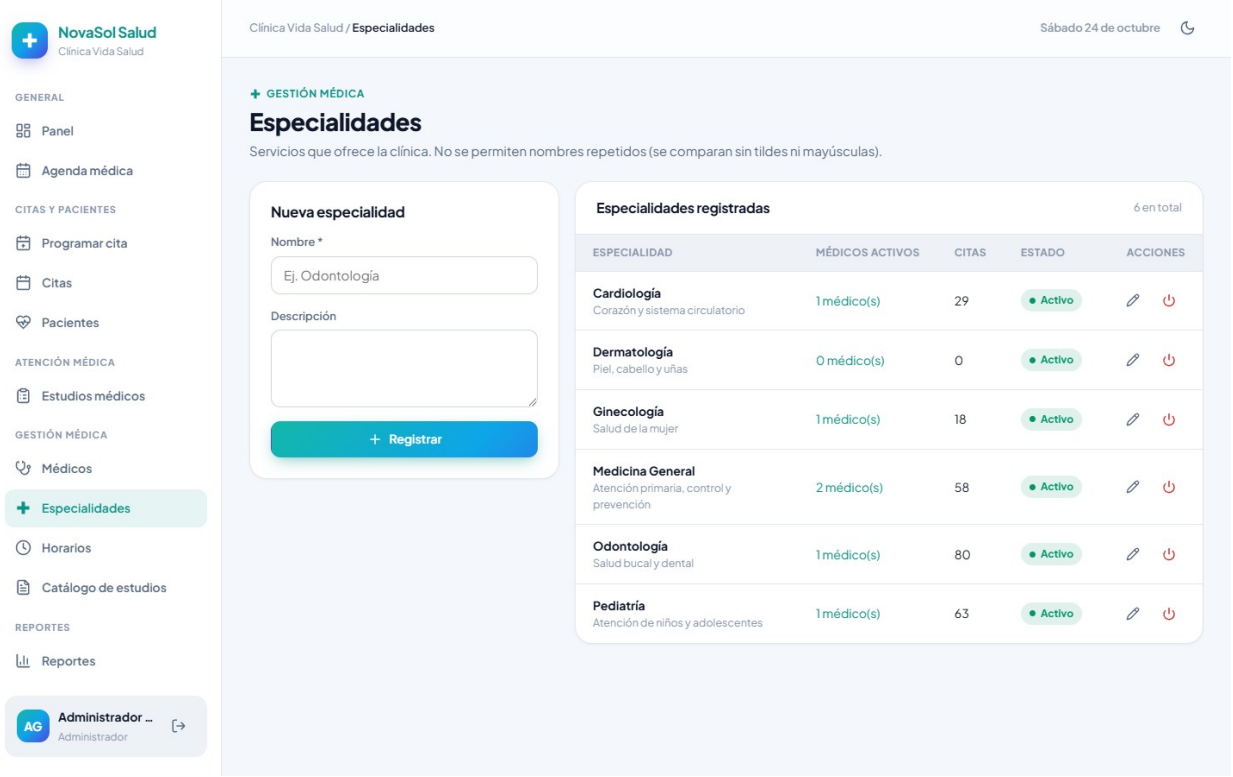


Figura 79. *CU4 · Especialidades*

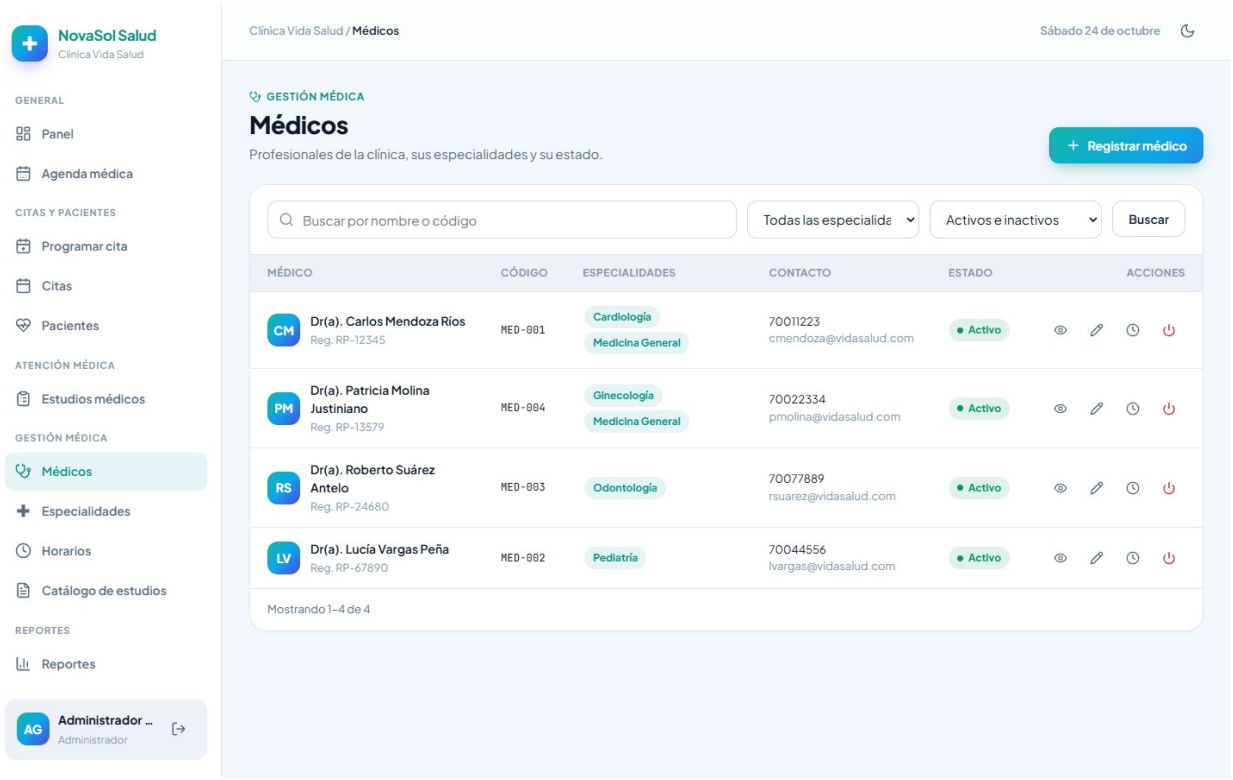


Figura 80. *CU5 · Listado de médicos*

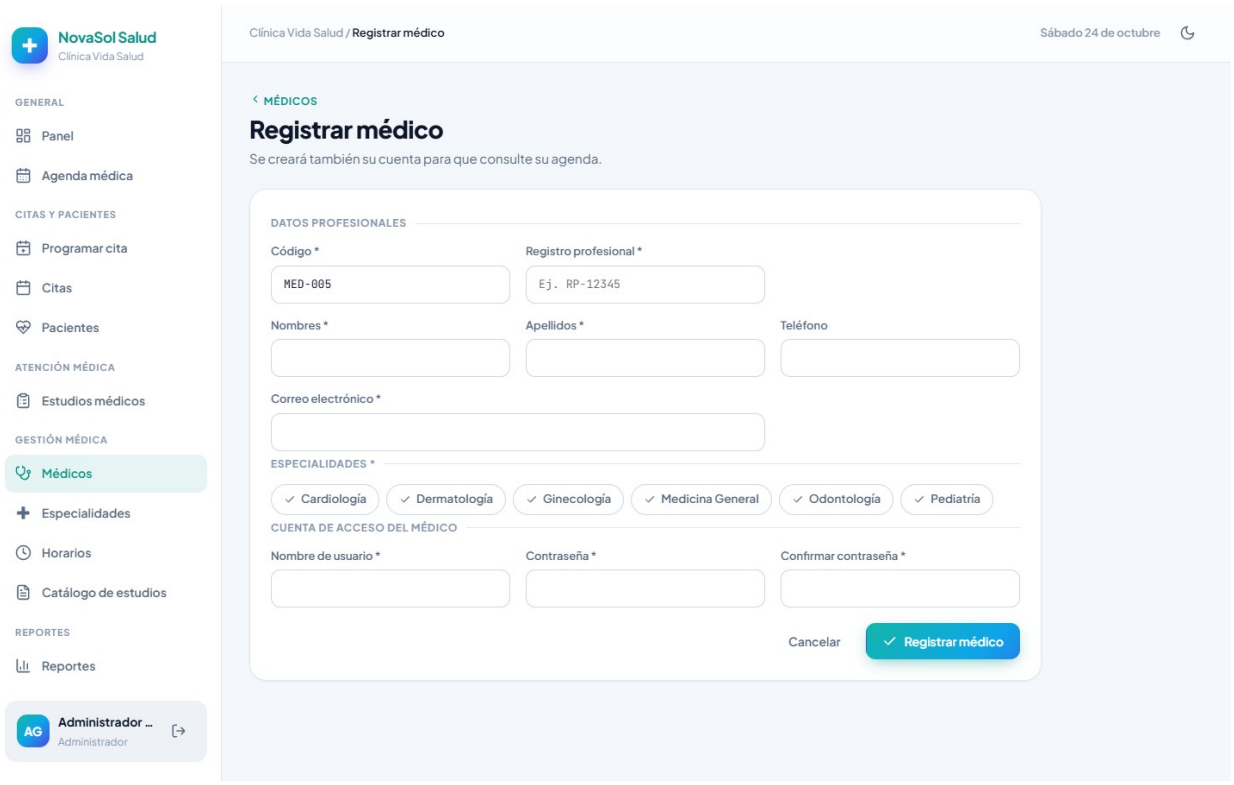


Figura 81. *CU5 · Registro de médico*

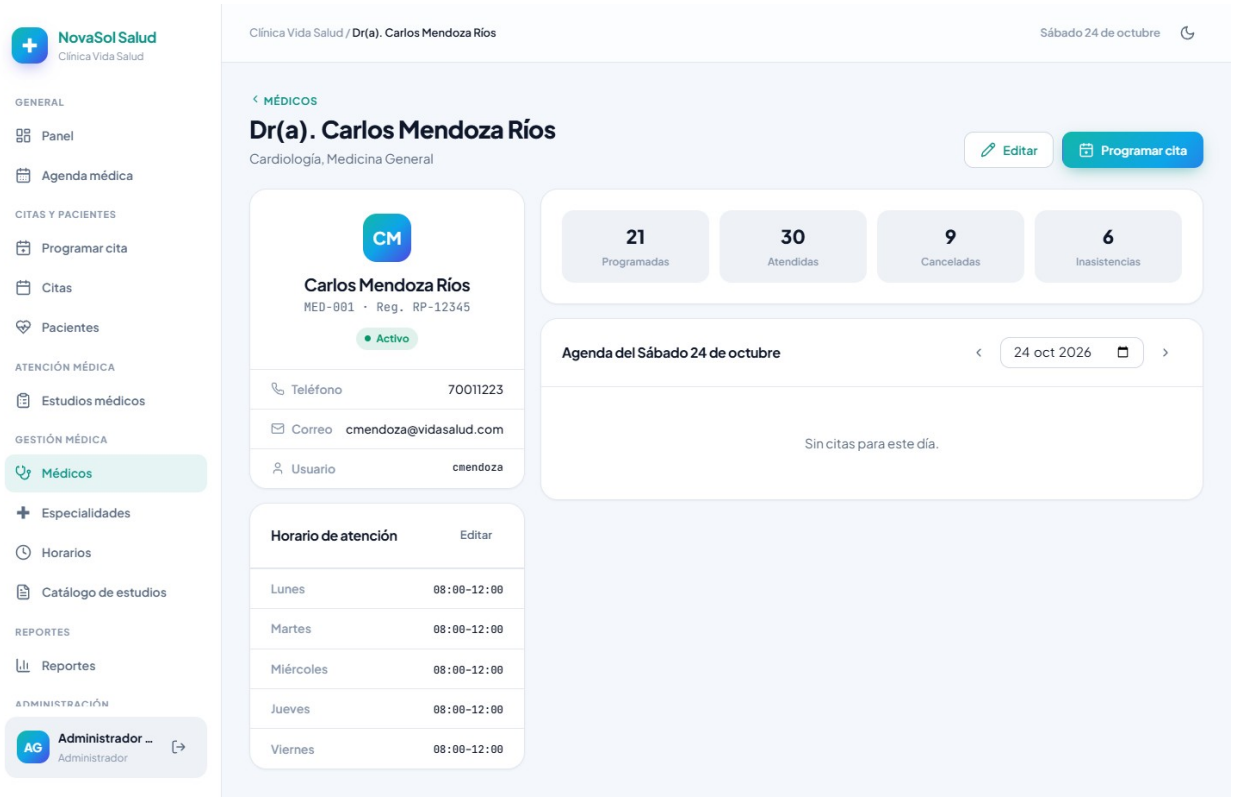


Figura 82. *CU5 · Ficha del médico con su agenda*

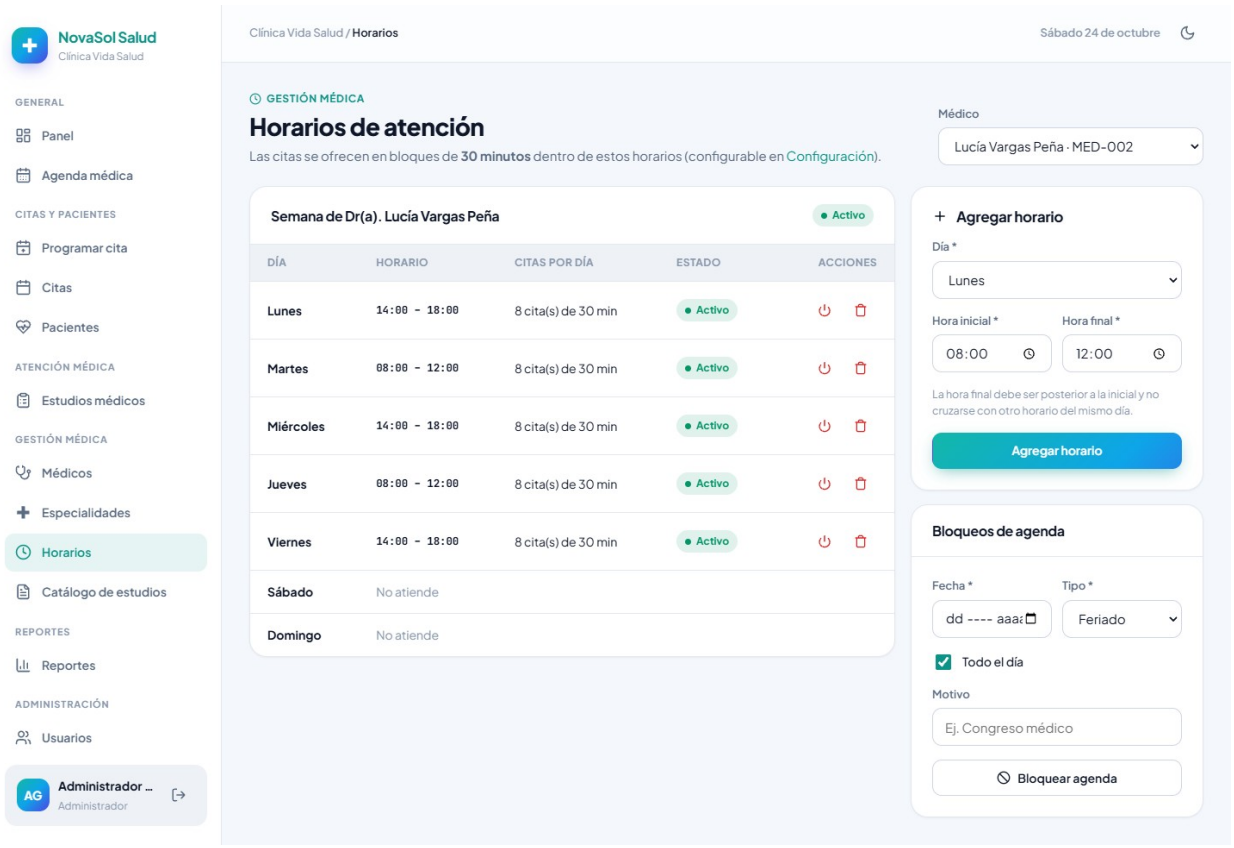


Figura 83. *CU6 · Horarios de atención y bloqueos*

P3 · Gestión de pacientes

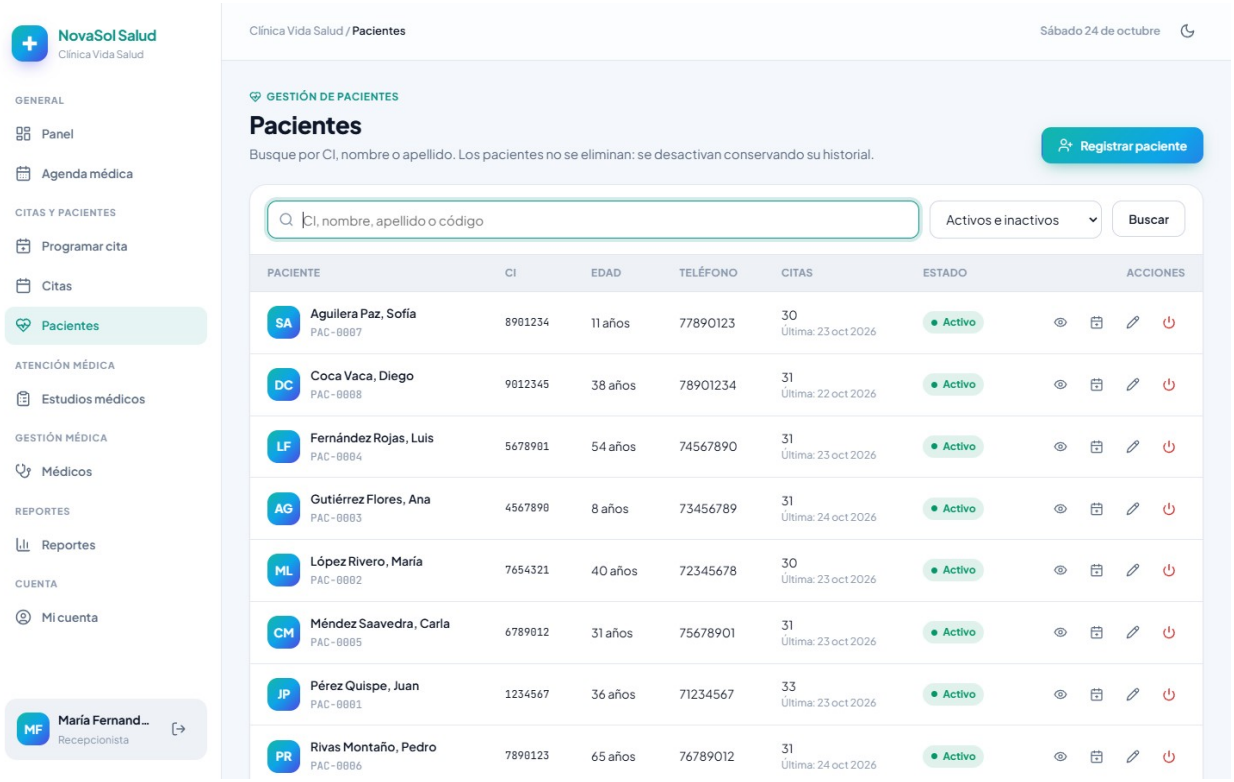


Figura 84. *CU7 · Búsqueda de pacientes*

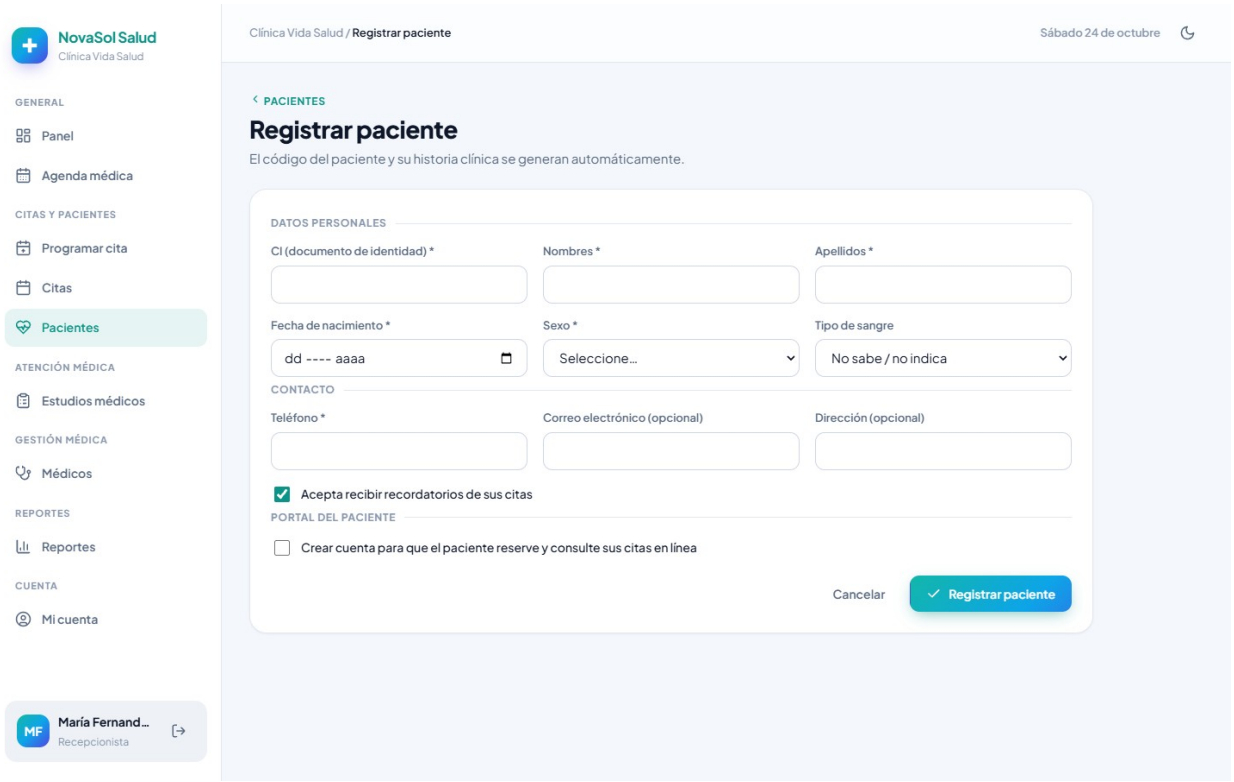


Figura 85. *CU7 · Registro de paciente*

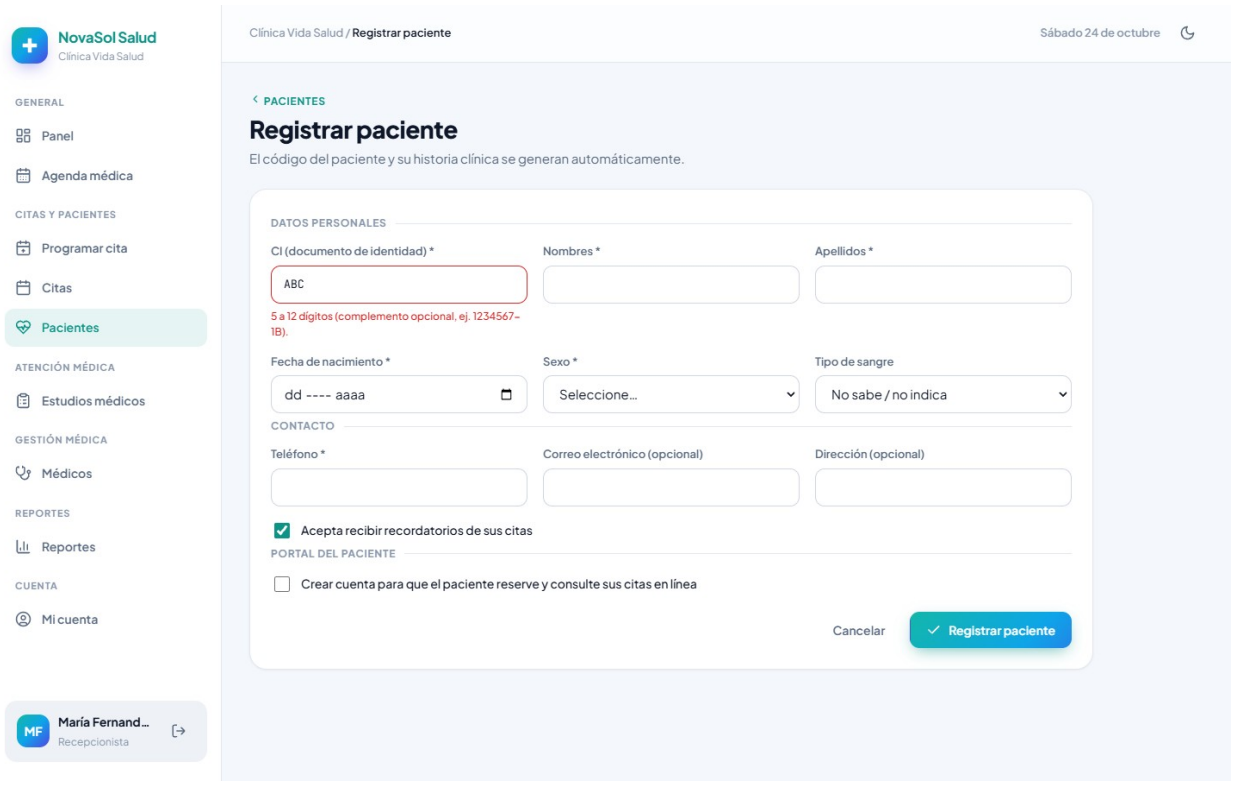


Figura 86. *CU7 · Validación de datos en el navegador*

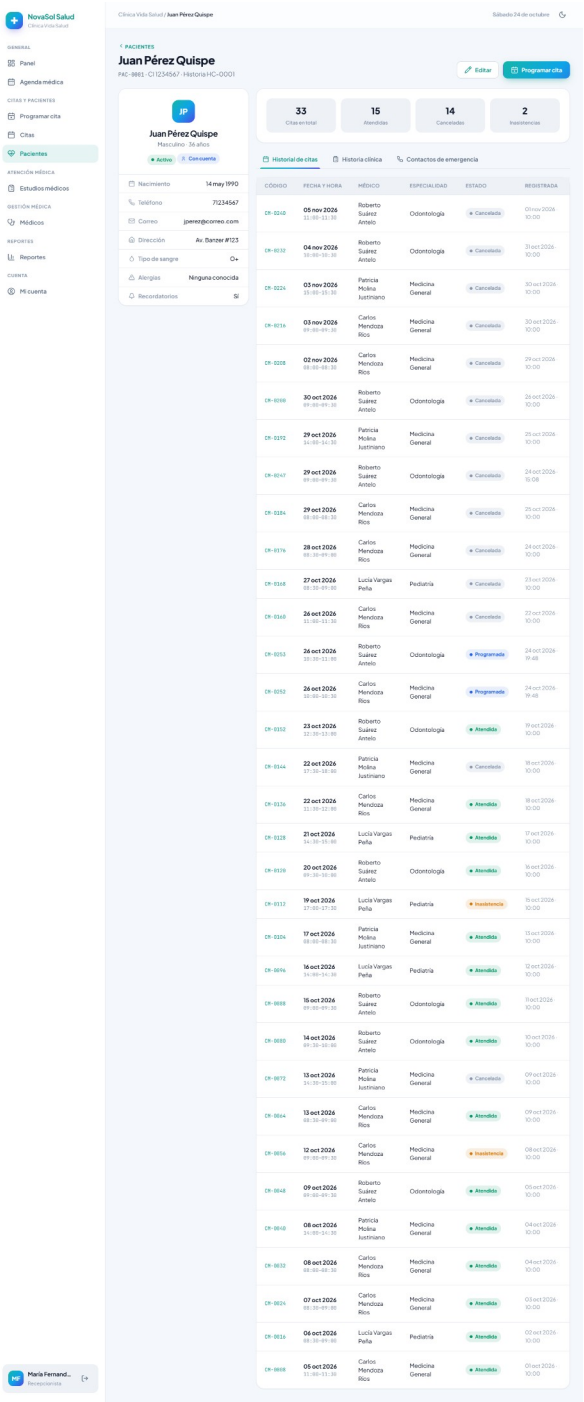


Figura 87. *CU7 · Ficha e historial de citas del paciente*

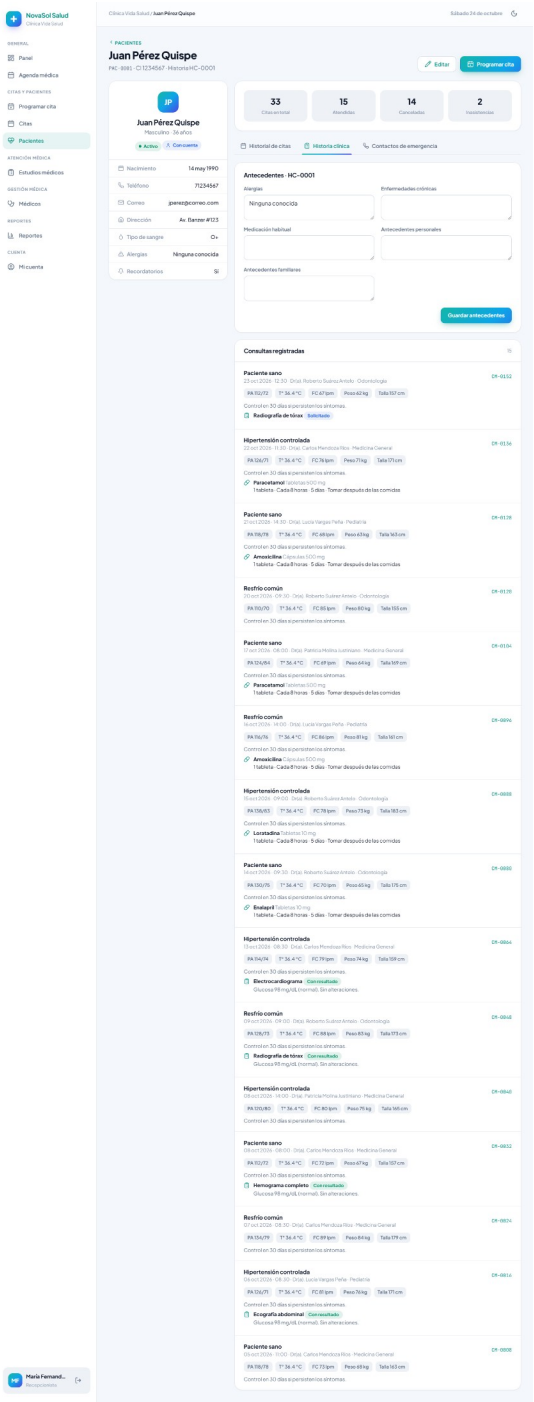


Figura 88. *CU16 · Historia clínica del paciente*

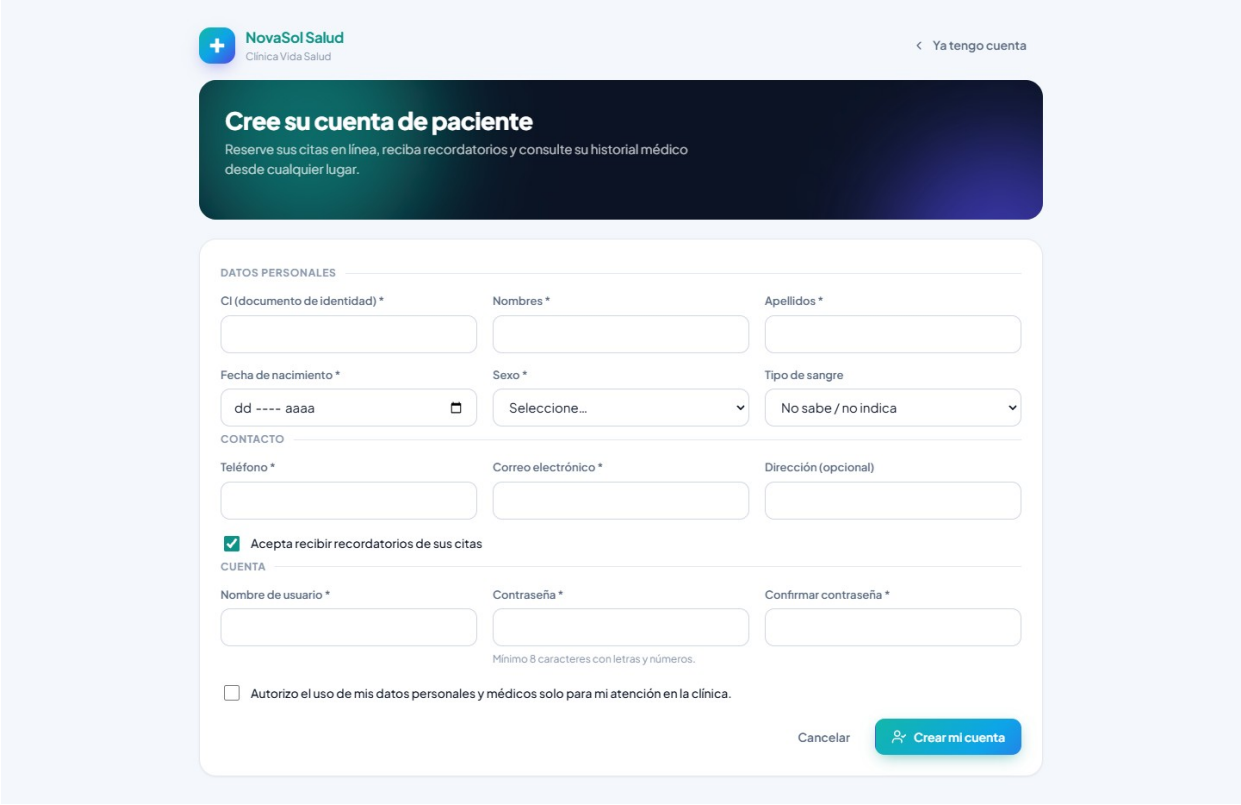


Figura 89. *CU8 · Autoregistro del paciente*

P4 · Gestión de citas

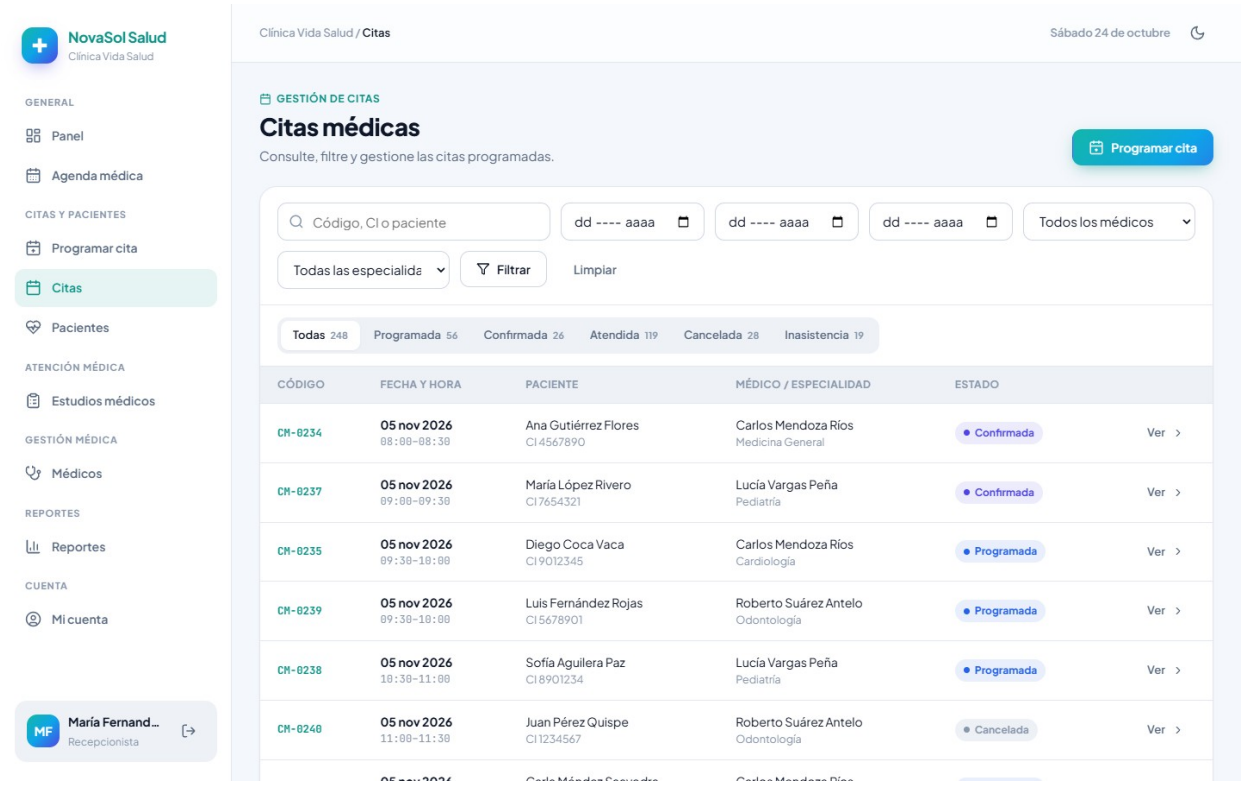


Figura 90. *Consulta de citas con filtros*

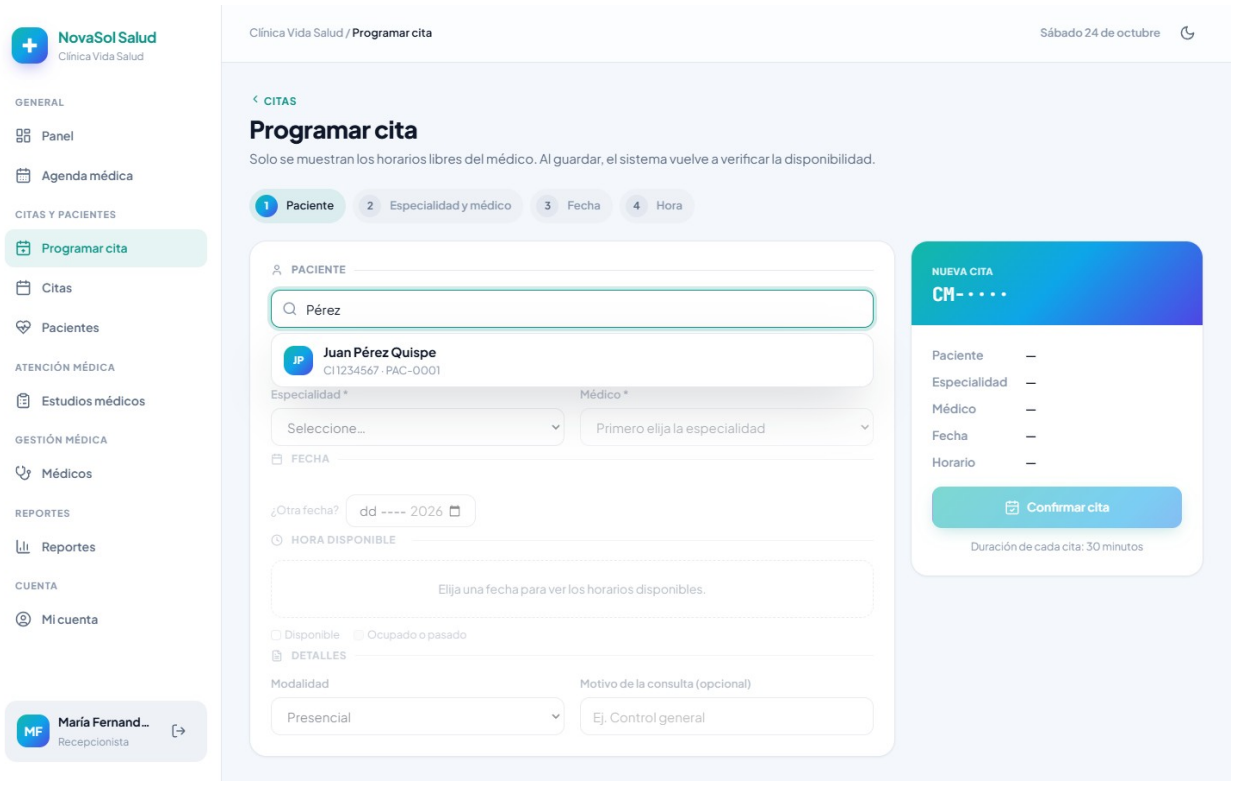


Figura 91. *CU10 · Búsqueda del paciente*

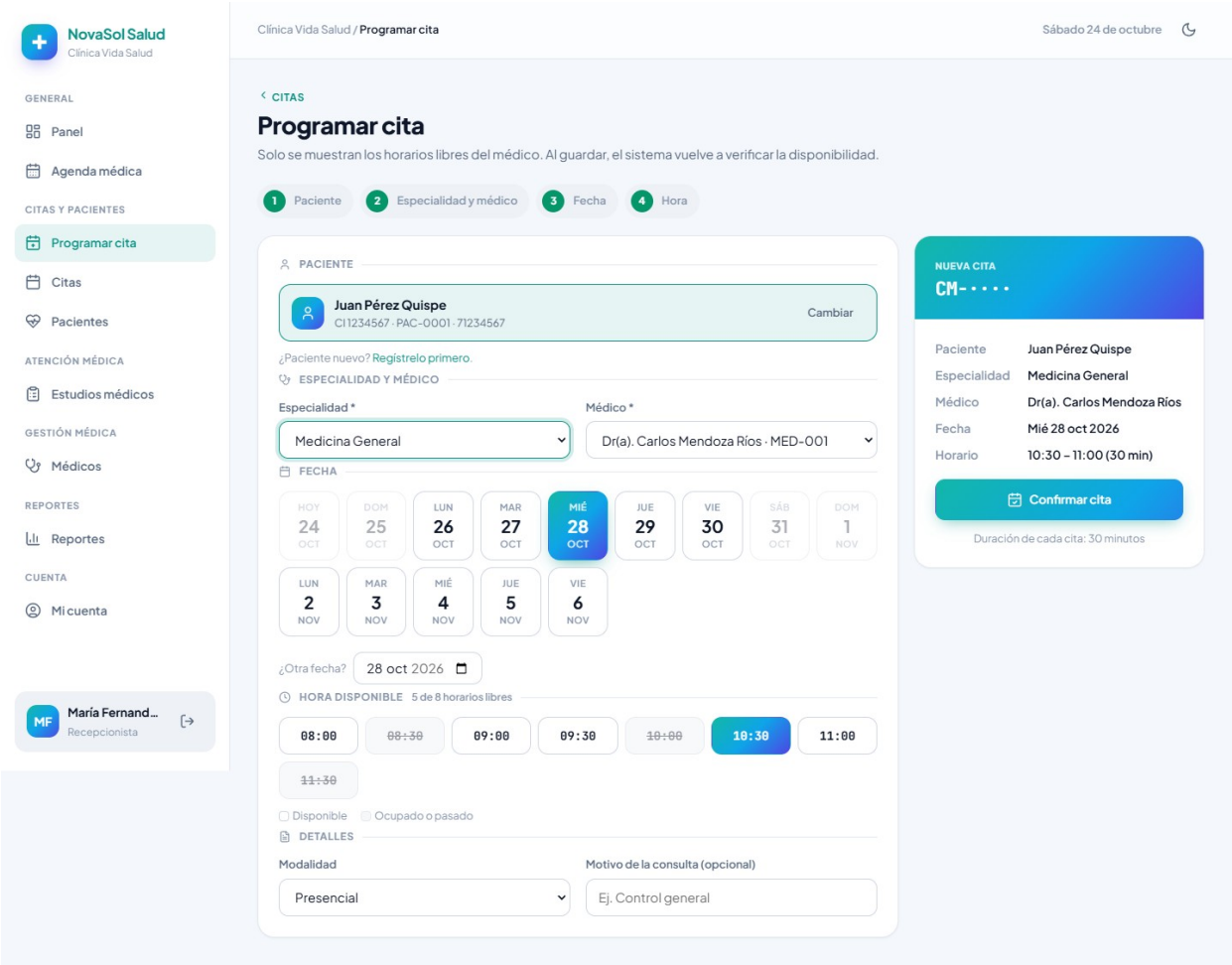


Figura 92. *CU9 y CU10 · Programar cita con horarios libres*

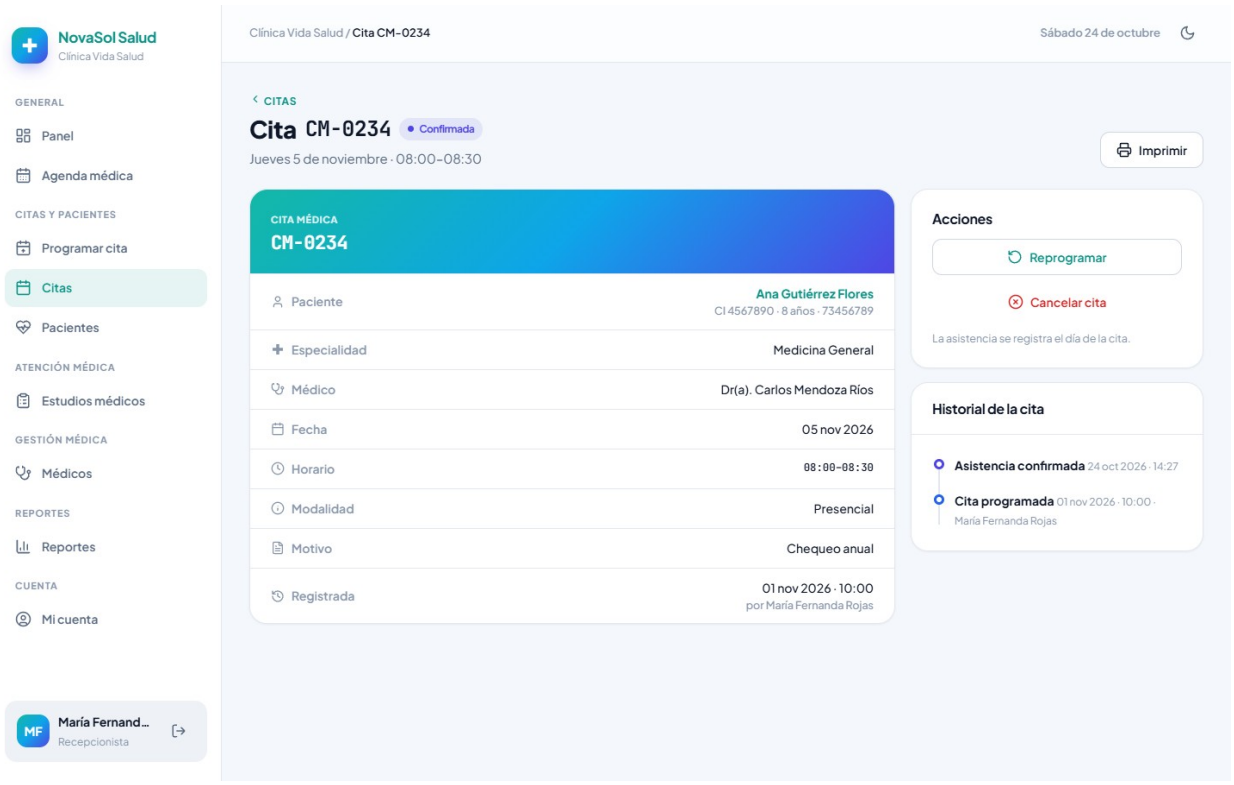


Figura 93. *CU13 · Detalle de la cita y acciones*

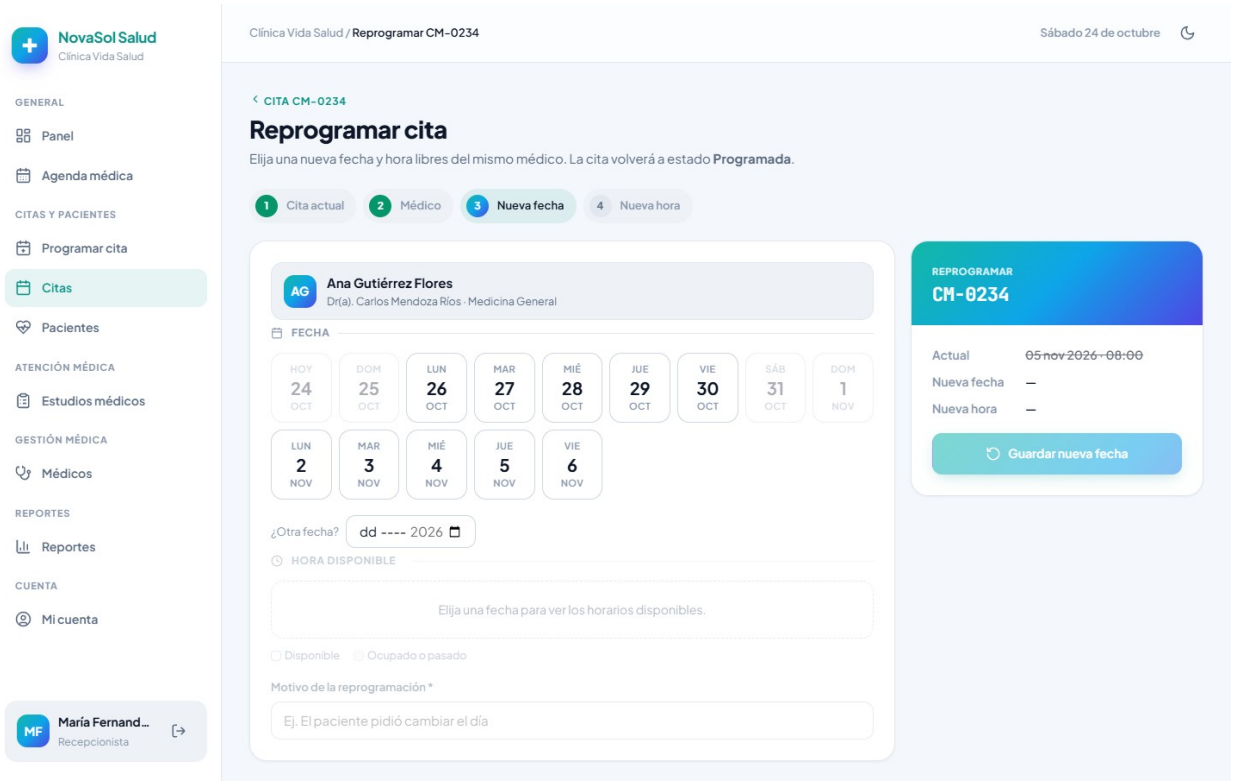


Figura 94. *CU11 · Reprogramar cita*

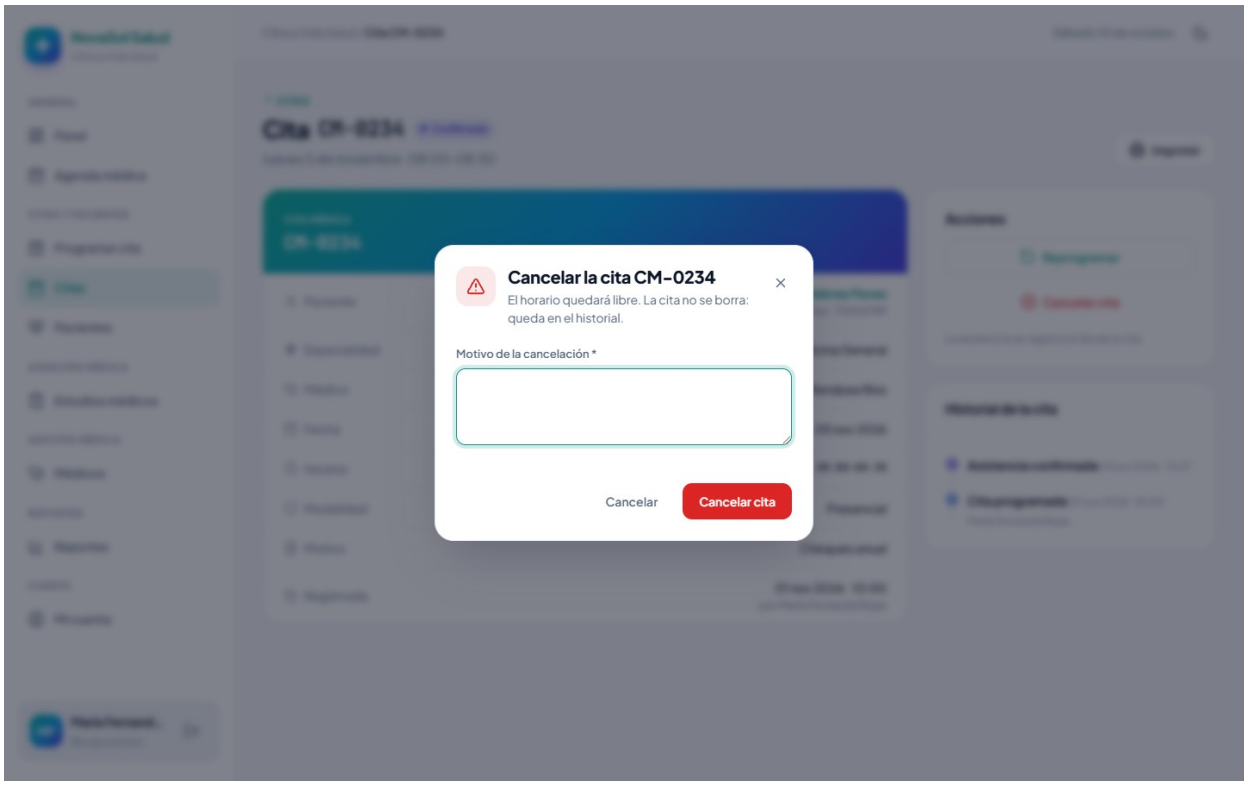


Figura 95. *CU12 · Cancelar cita con motivo*

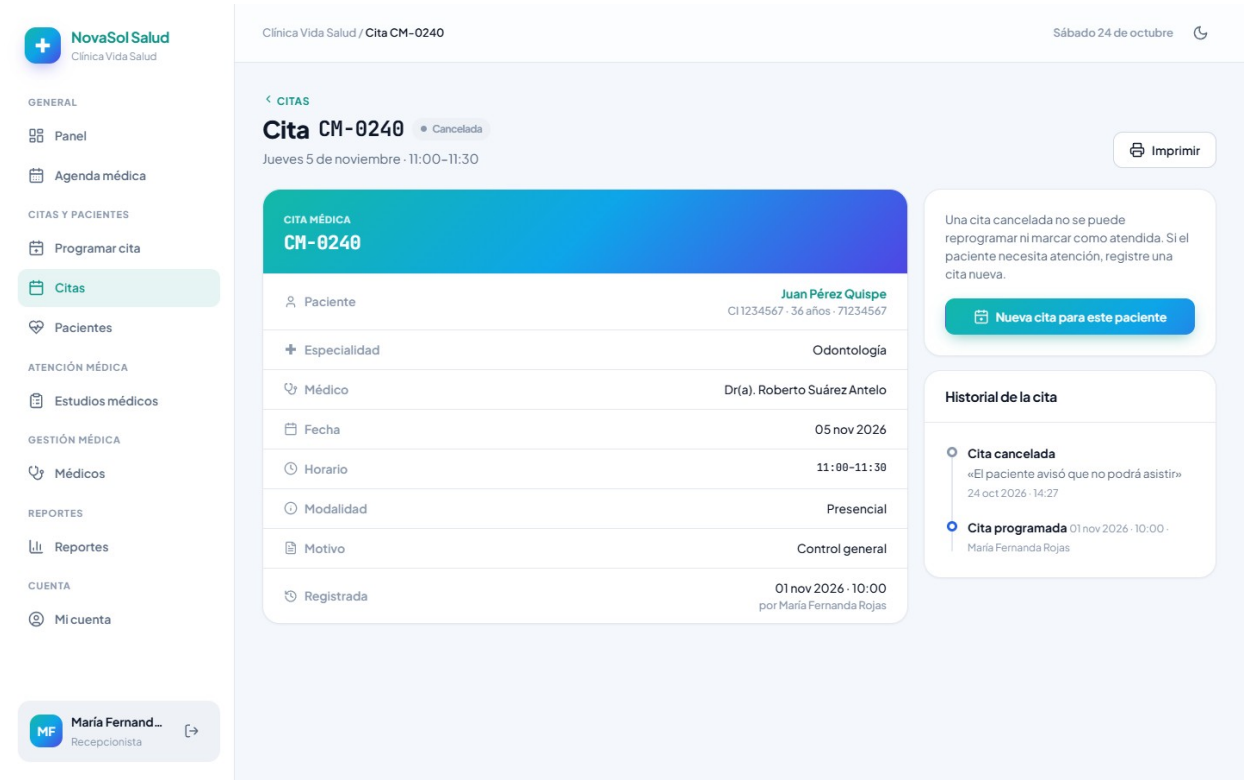


Figura 96. *CU12 · Cita cancelada con su historial*

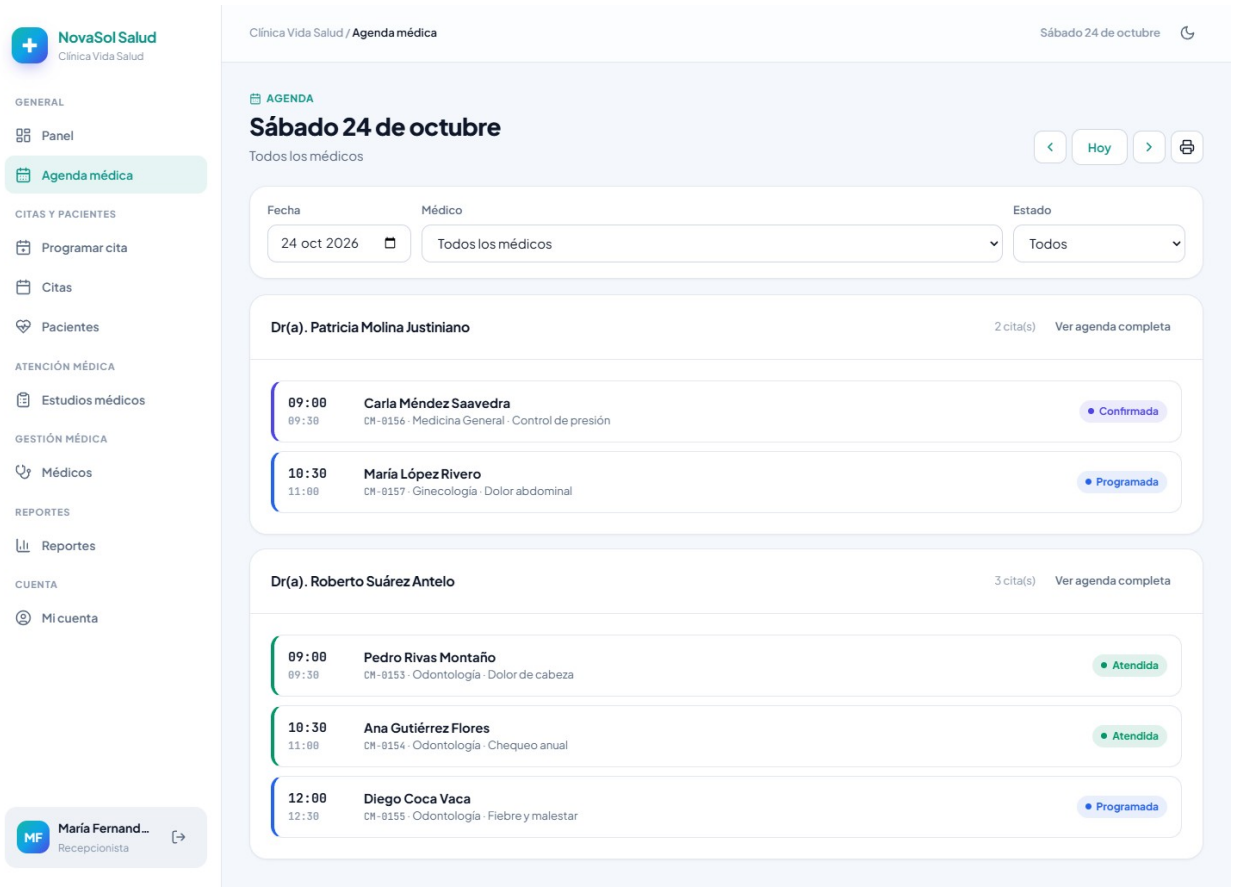


Figura 97. *CU14 · Agenda de todos los médicos*

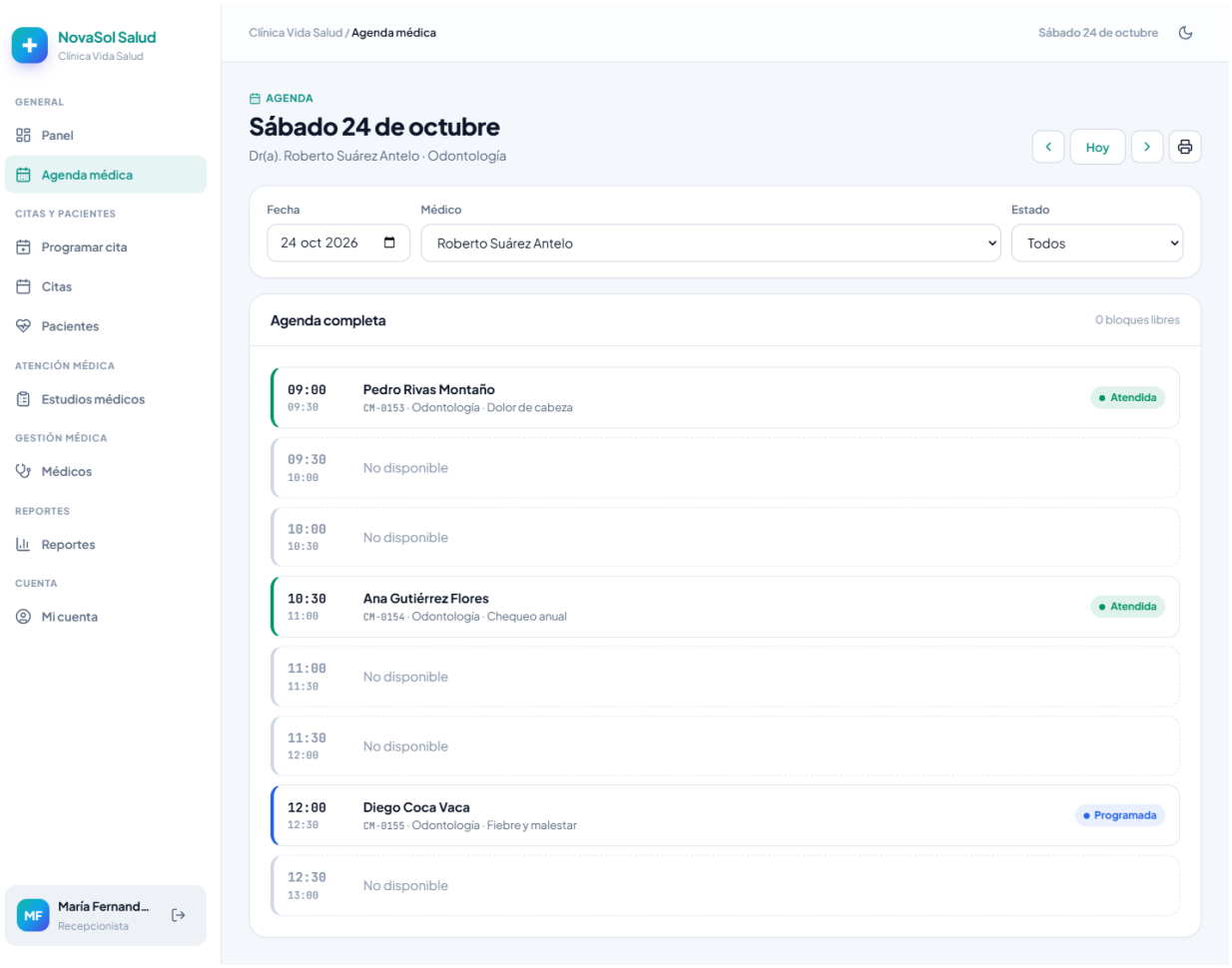


Figura 98. *CU14 · Agenda de un médico con bloques libres*

P5 · Atención médica

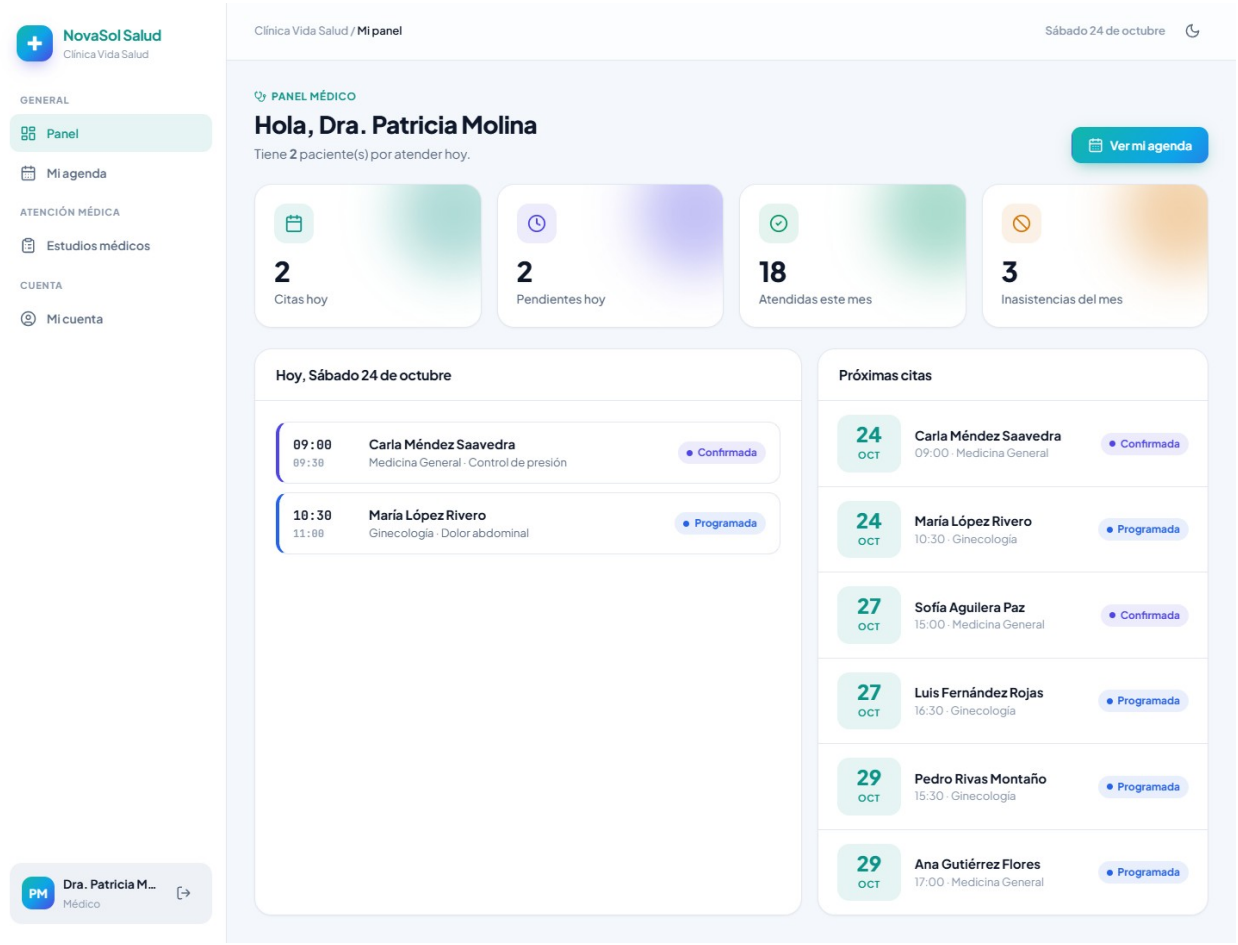


Figura 99. *Panel del médico*

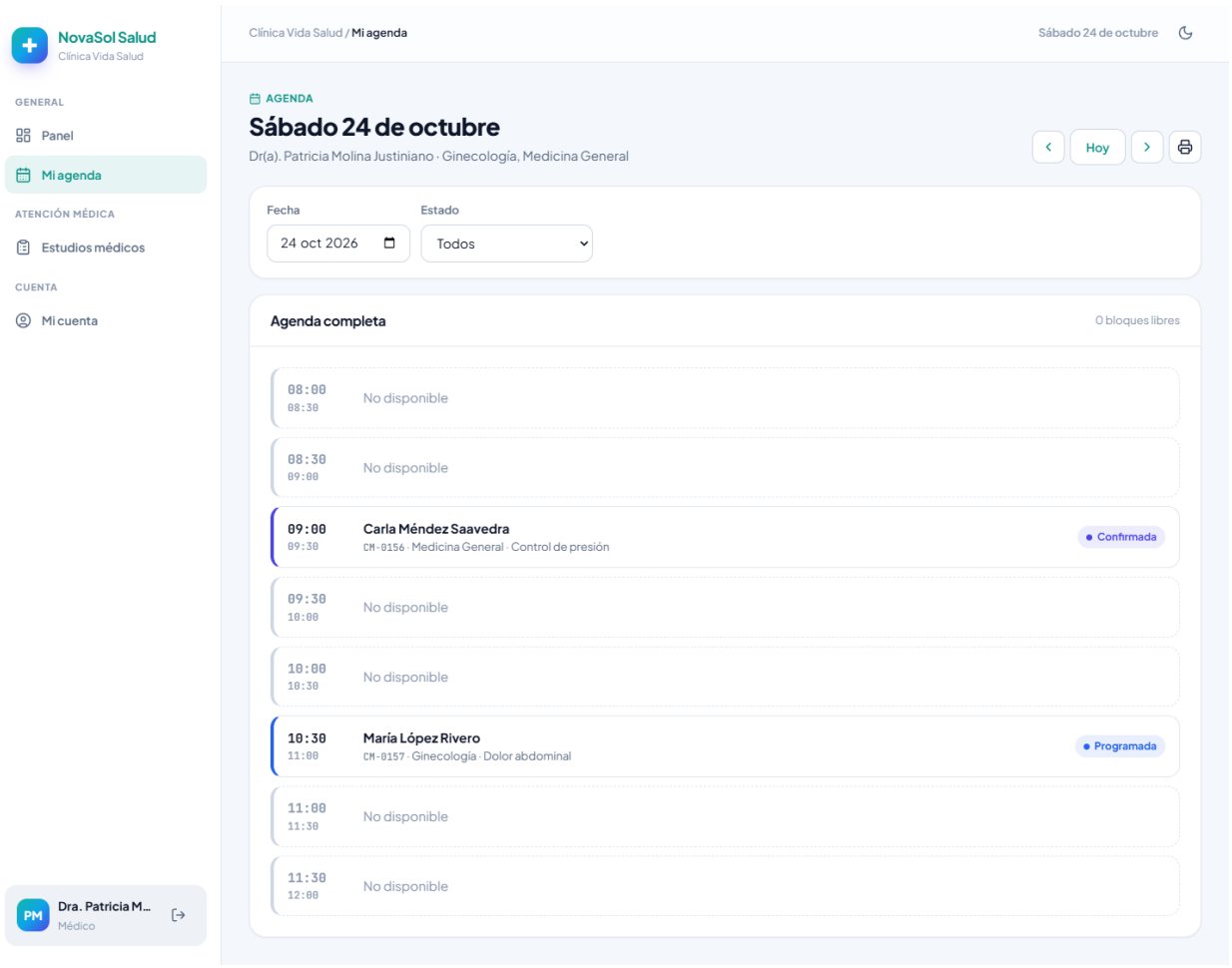


Figura 100. *CU14 · Agenda propia del médico*

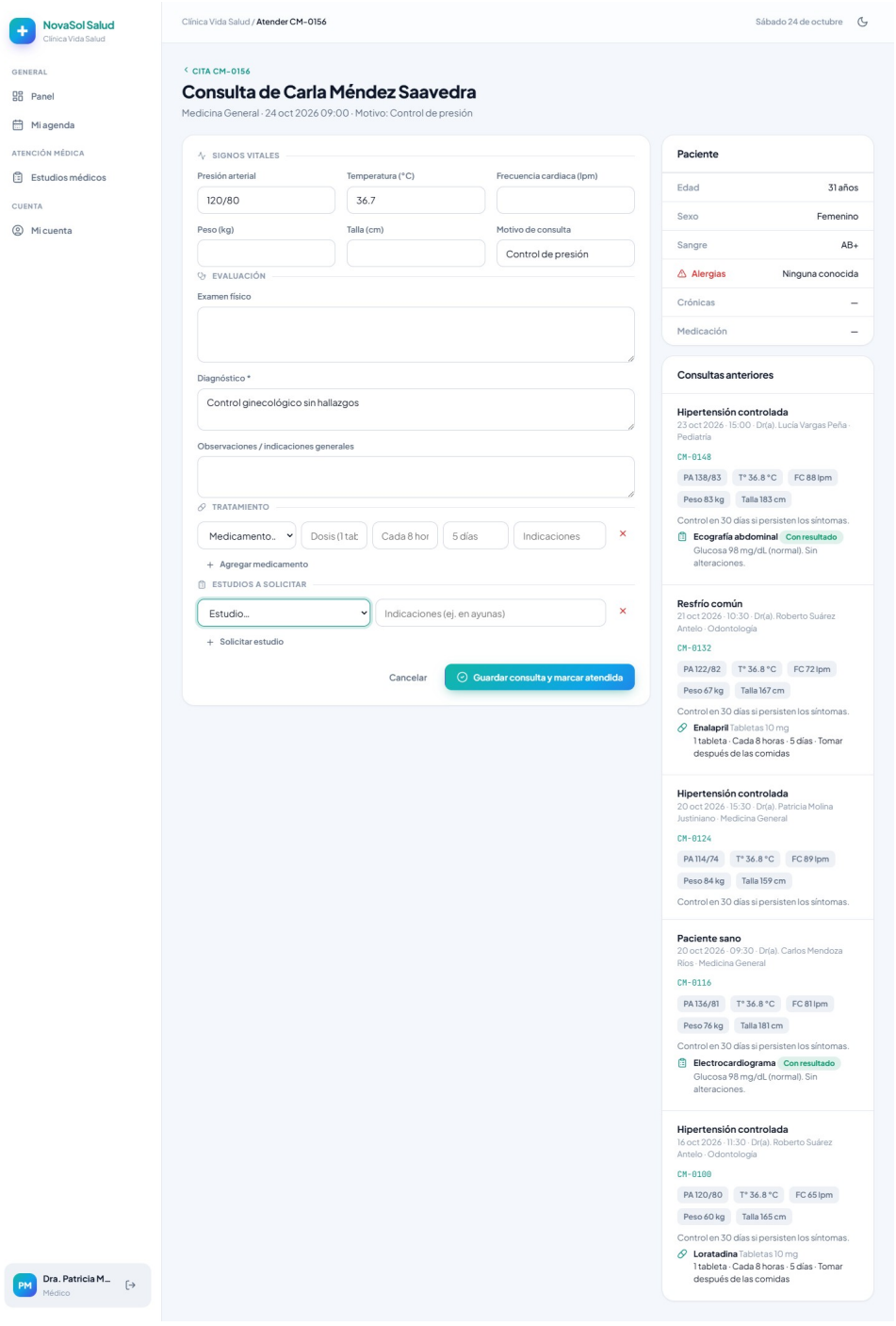


Figura 101. *CU15, CU17 y CU18 · Registro de la consulta*

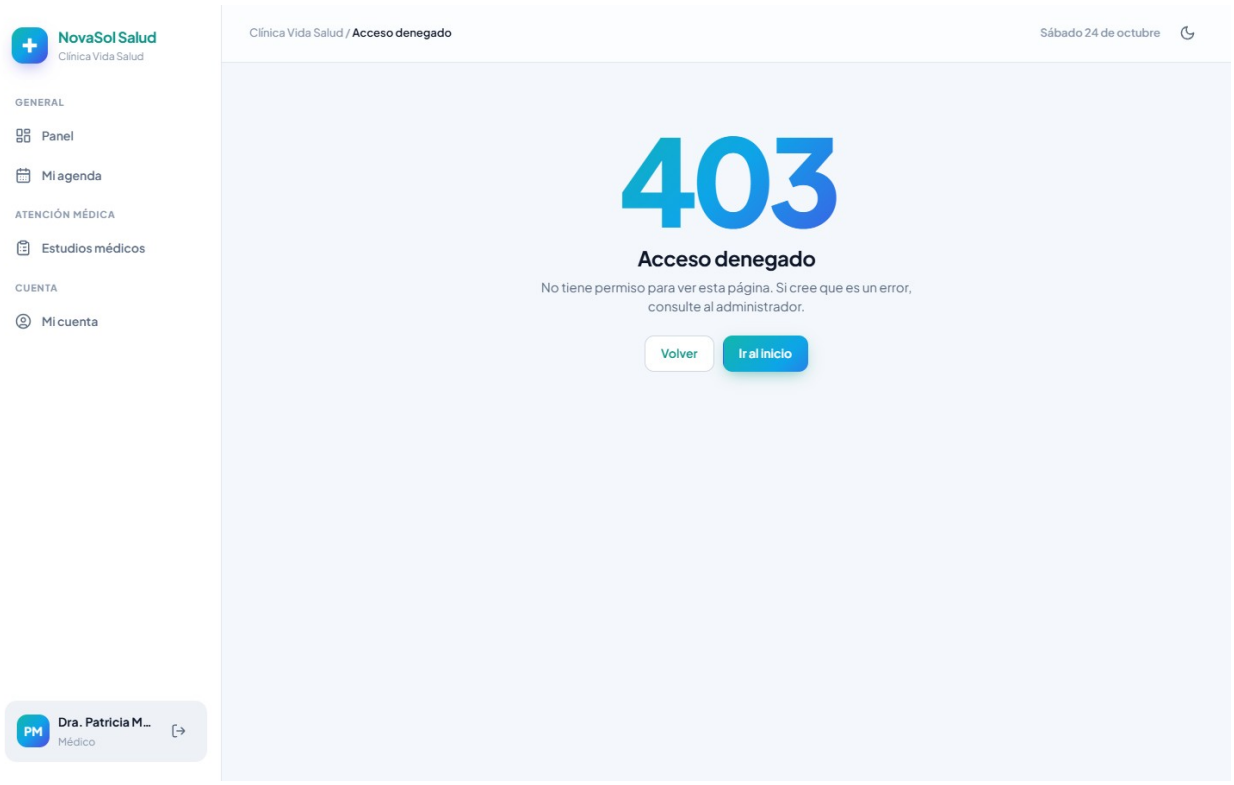


Figura 102. *CU15 · Cita atendida con su consulta*

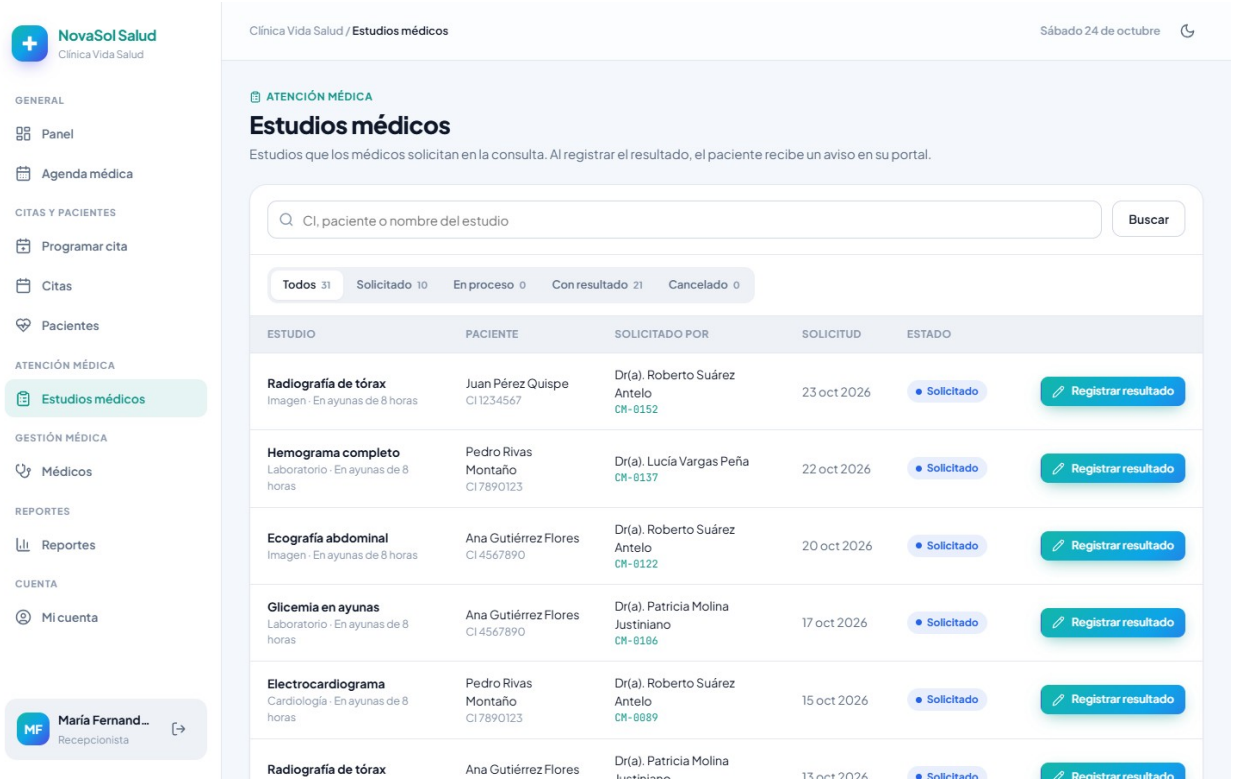


Figura 103. *CU18 · Bandeja de estudios médicos*

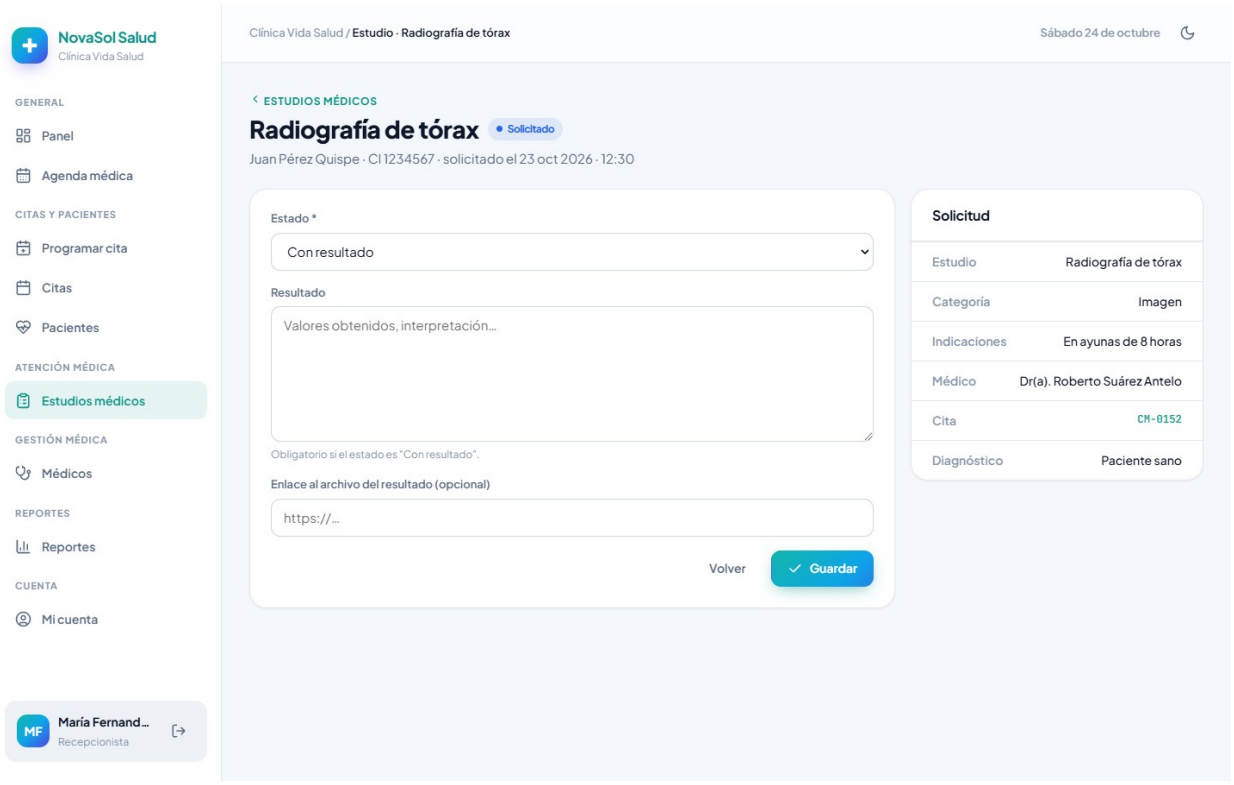


Figura 104. *CU18 · Registro del resultado*

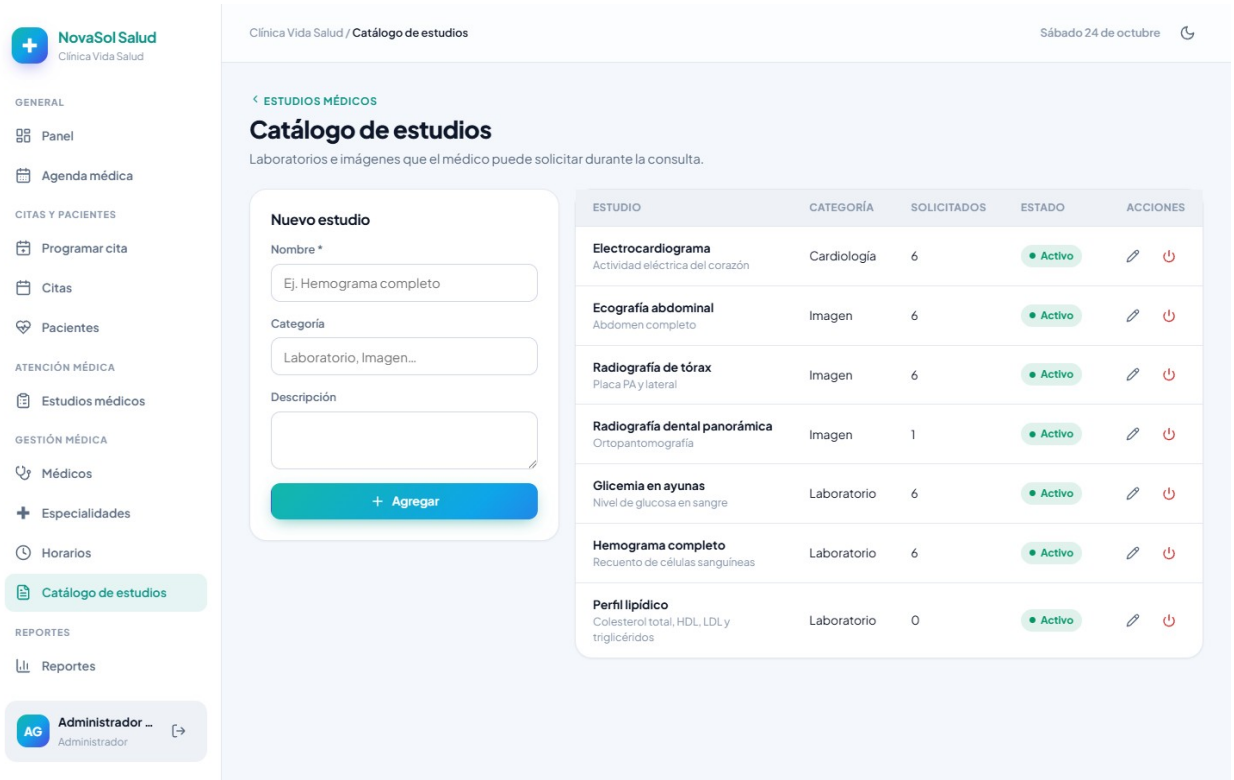


Figura 105. *CU18 · Catálogo de estudios*

P6 · Reportes

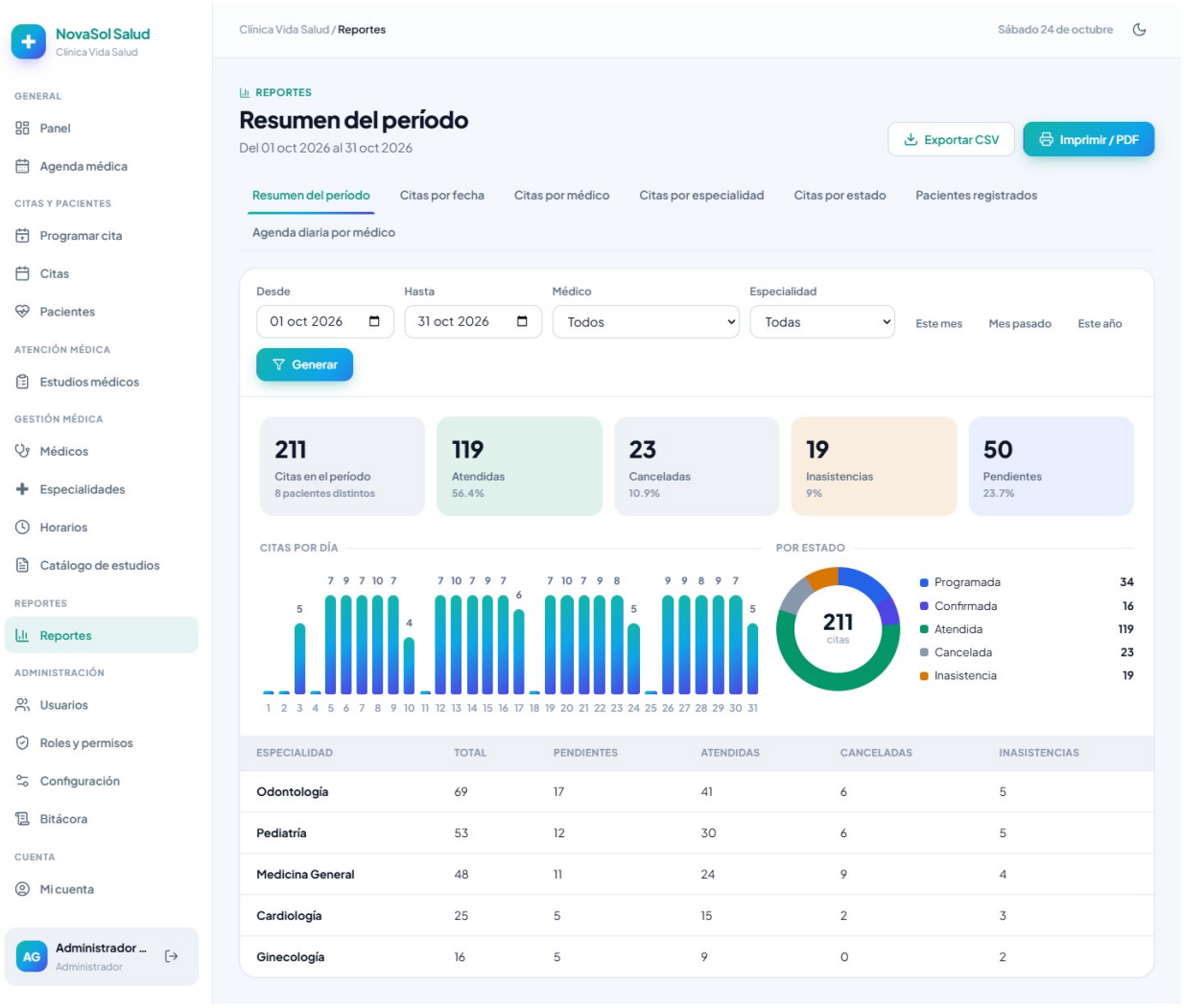


Figura 106. *CU20 · Resumen del período*

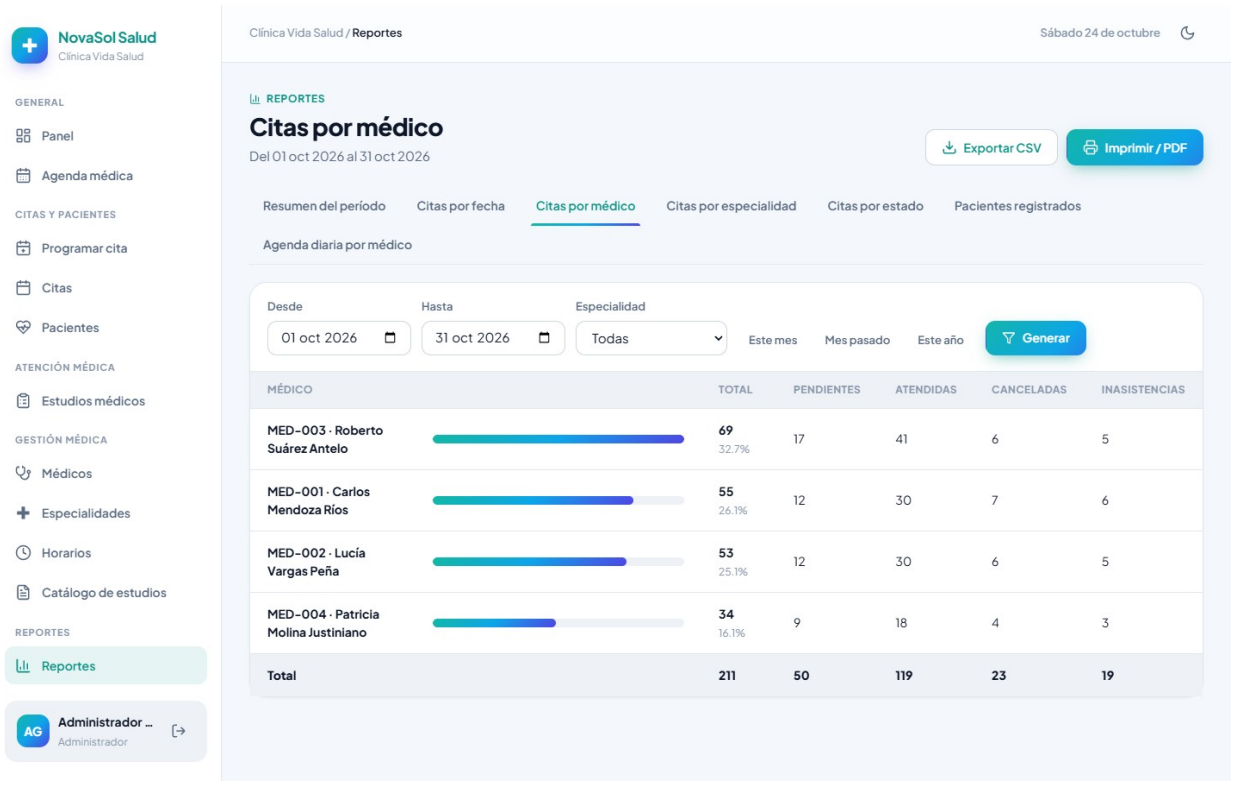


Figura 107. *CU20 · Citas por médico*

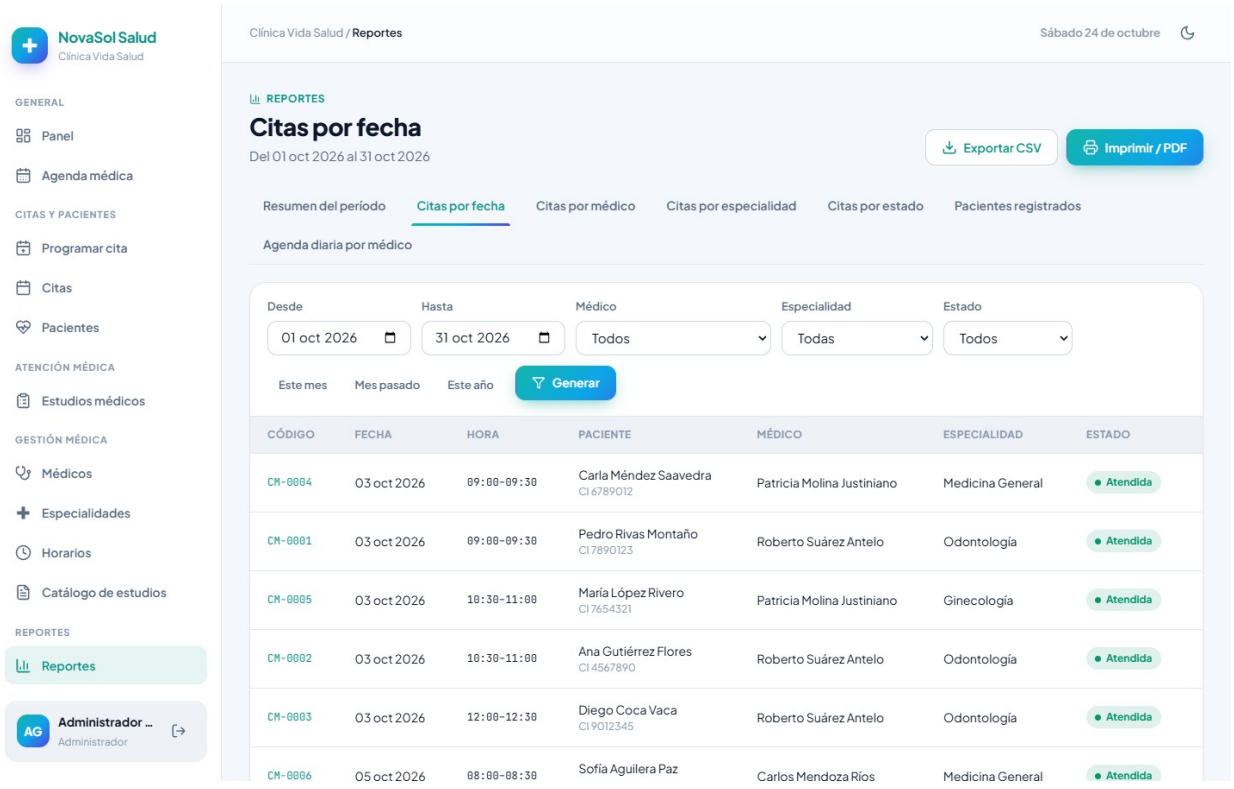


Figura 108. *CU20 · Citas por fecha*

Portal del paciente

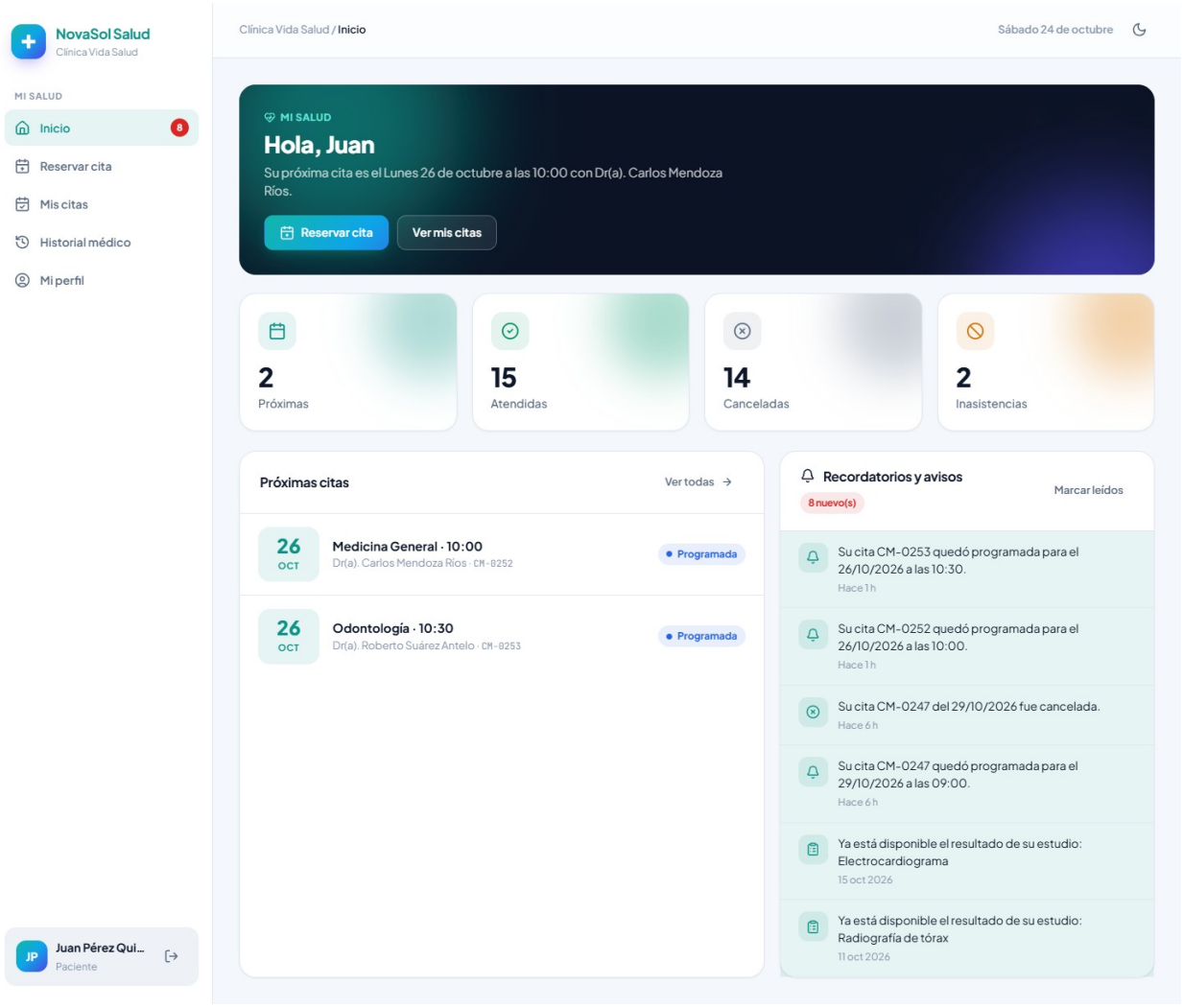


Figura 109. *Inicio del portal con recordatorios y avisos*

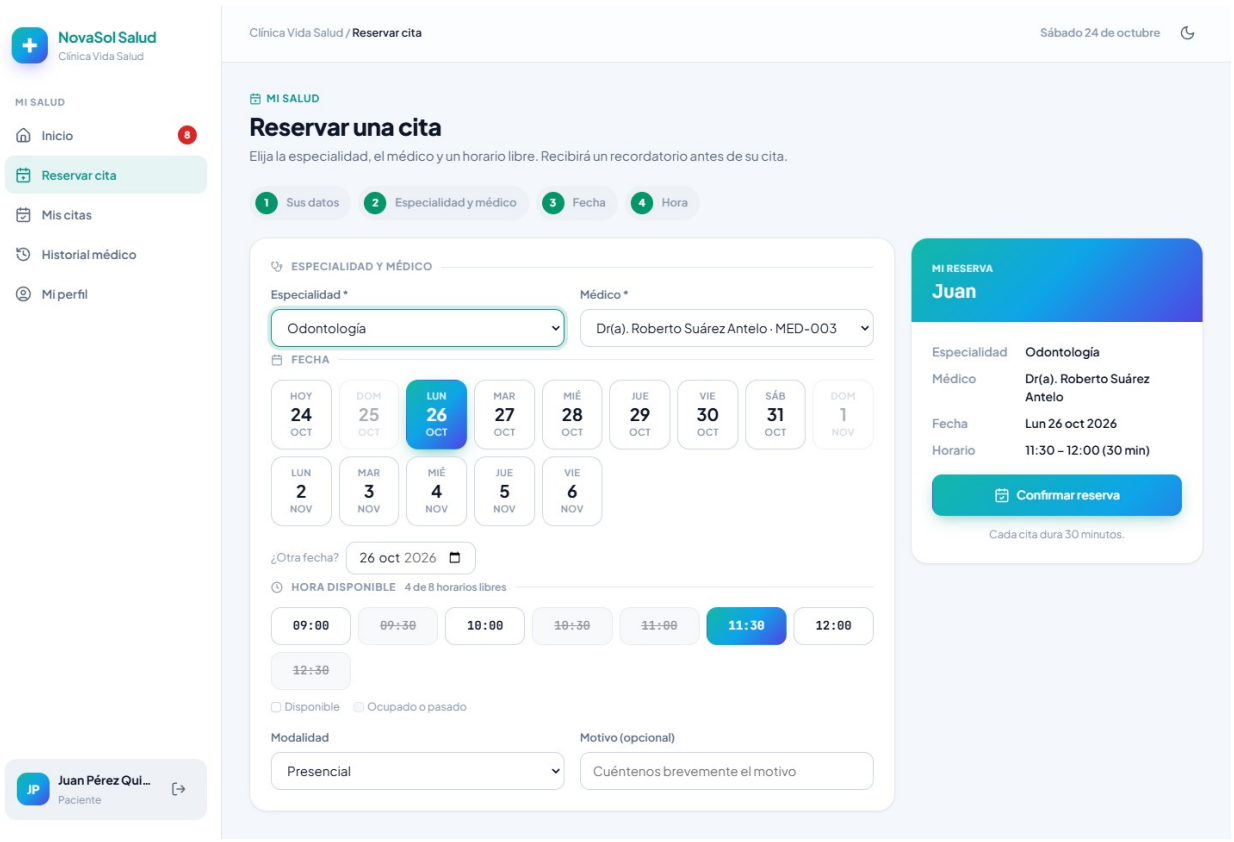


Figura 110. *CU10 · Reserva desde el portal*

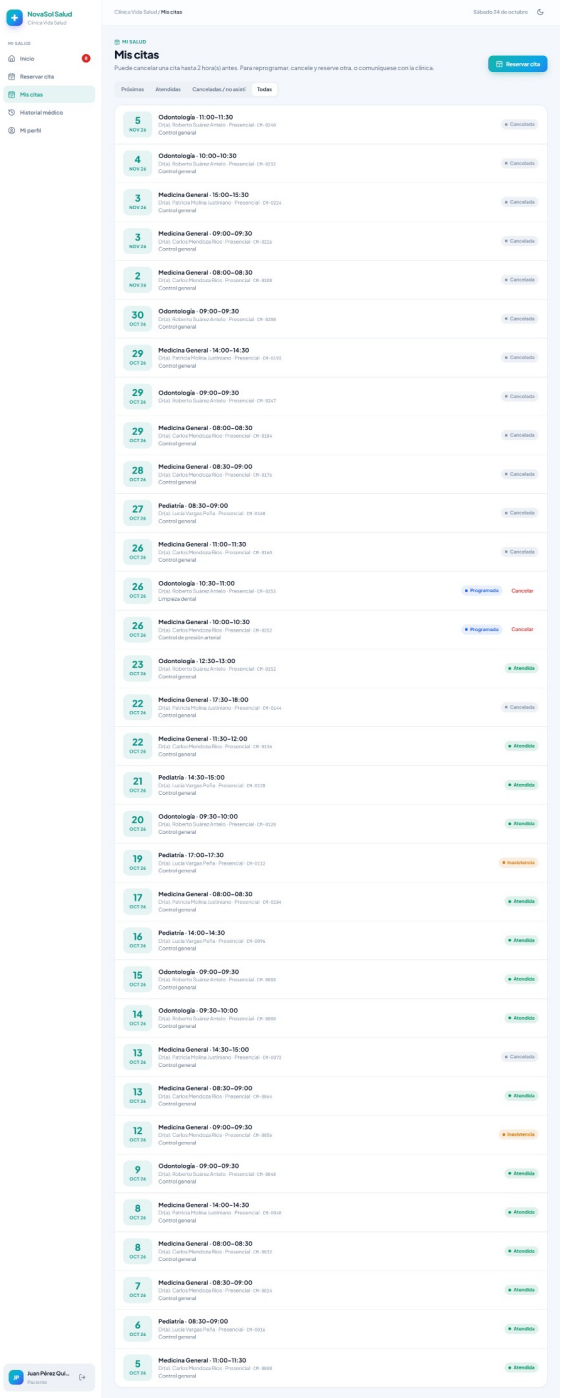


Figura 111. *CU12 · Mis citas*

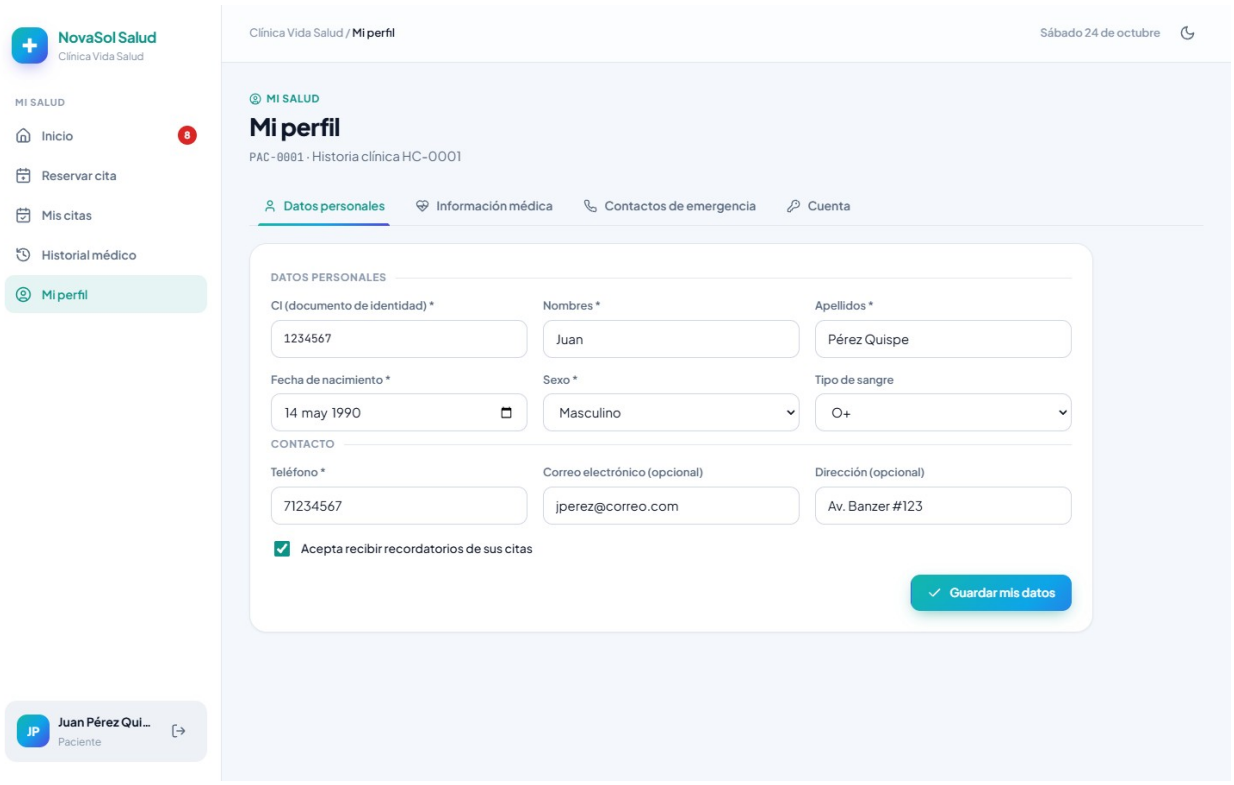


Figura 112. *Perfil del paciente*

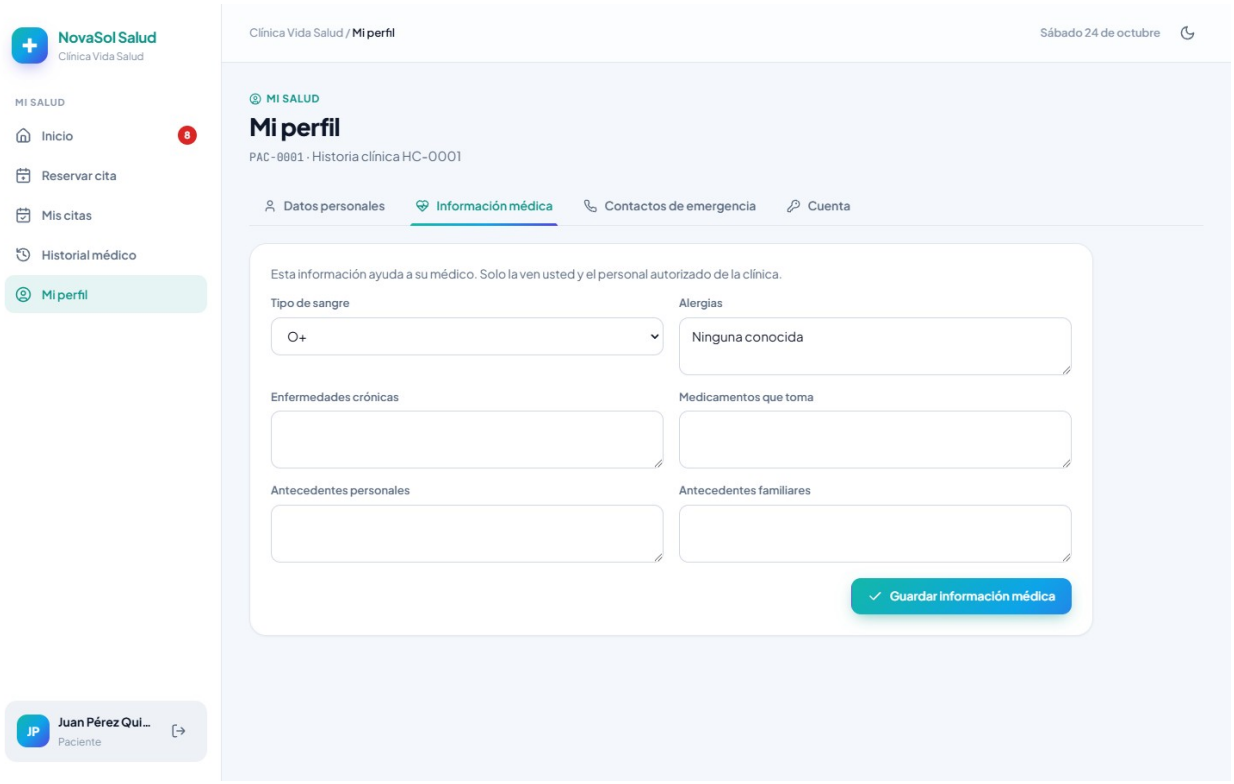


Figura 113. *Información médica del paciente*

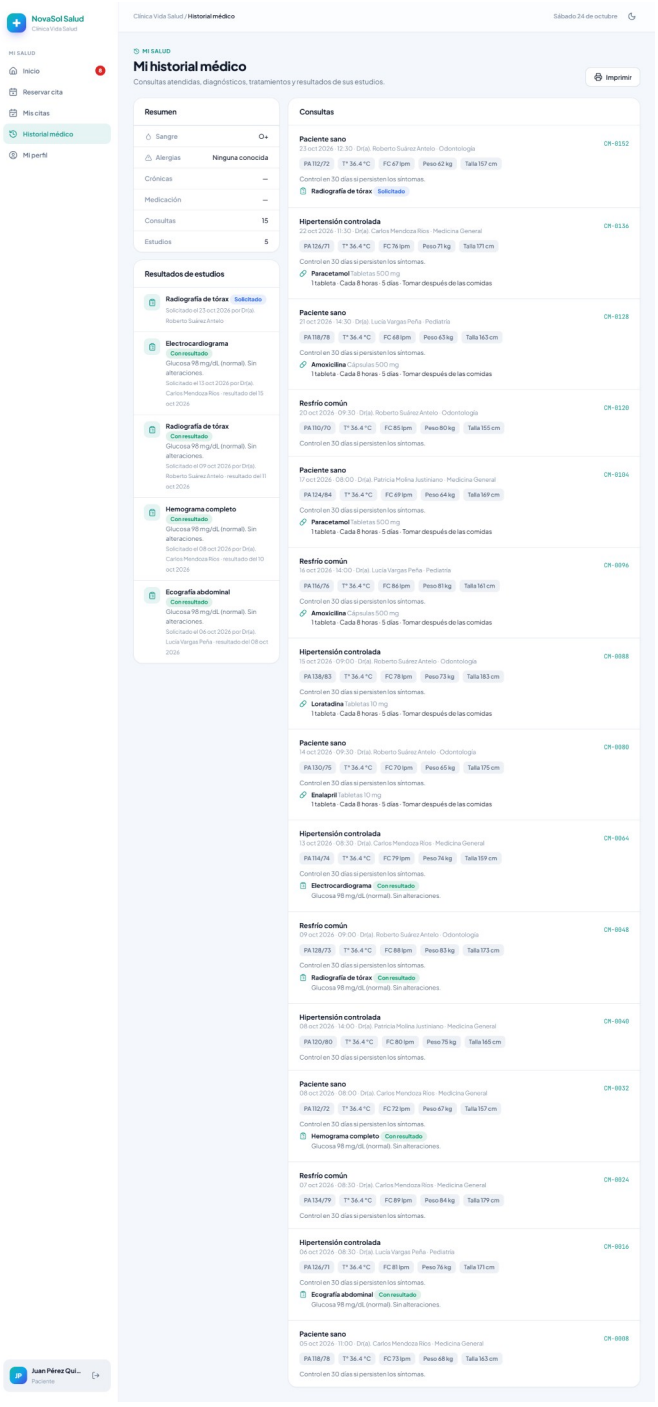


Figura 114. *CU19 · Historial médico y resultados*

Diseño adaptable y modo oscuro



Figura 115. *Panel en modo oscuro*



Figura 116. *Agenda en un celular*

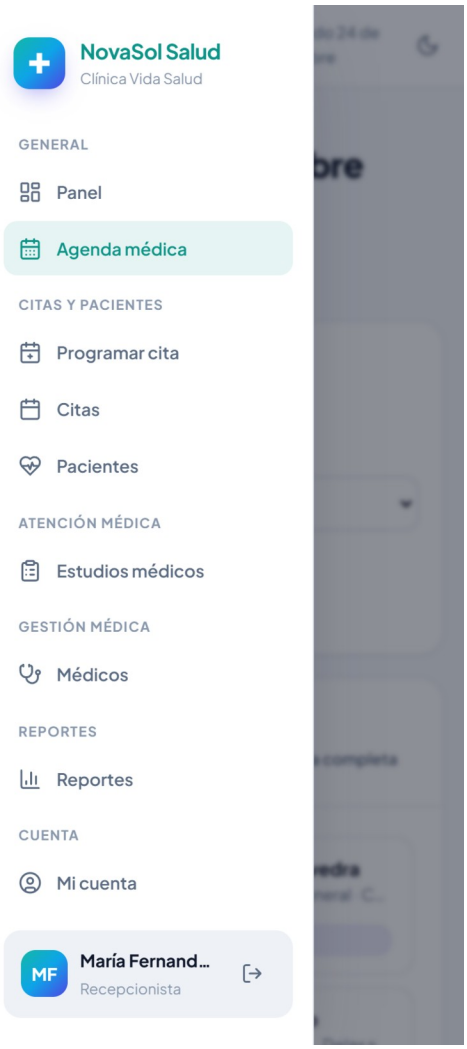


Figura 117. *Menú en un celular*

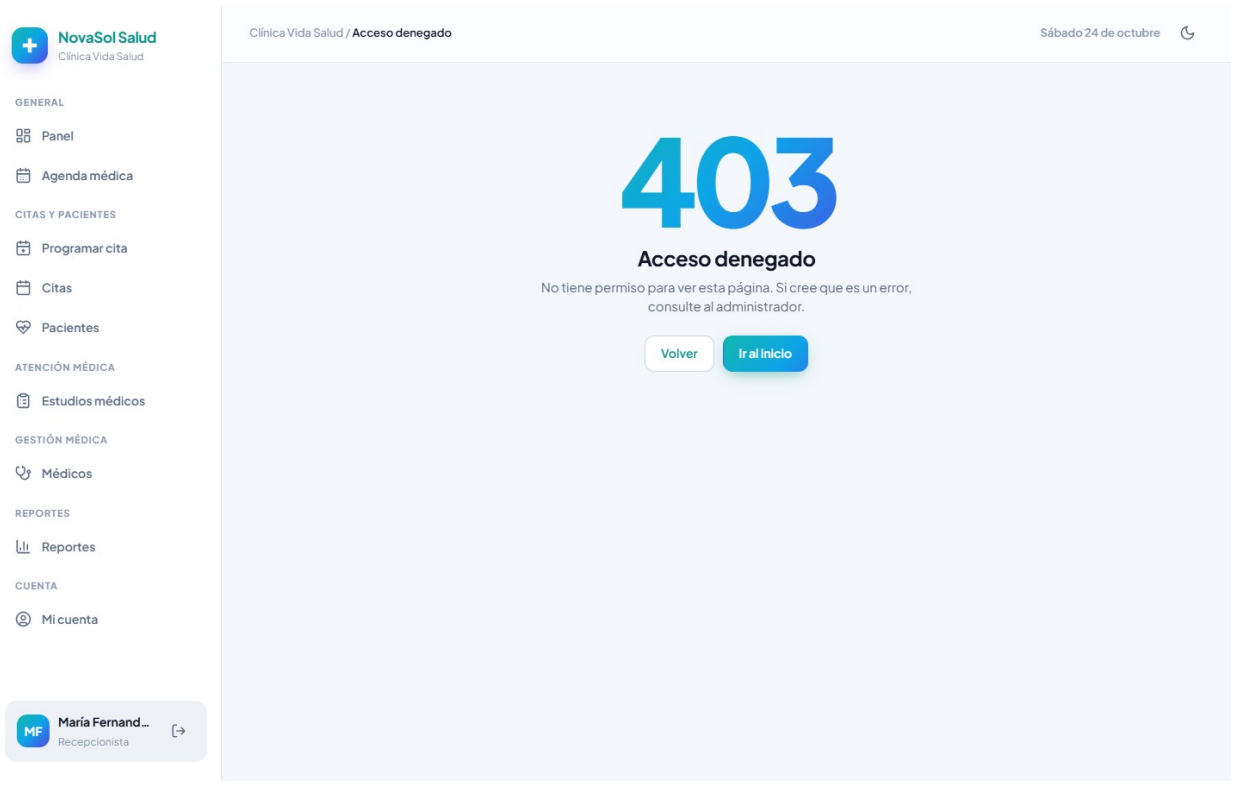


Figura 118. *Acceso denegado según el rol*

8. FT PRUEBAS

8.1 Estrategia de pruebas

Se aplicaron **pruebas de caja negra** sobre cada caso de uso, enviando peticiones HTTP reales al sistema (incluso saltándose la validación del navegador, para comprobar la del servidor), **pruebas de integridad** de la base de datos insertando datos directamente con SQL, **pruebas de seguridad** de acceso por rol y **pruebas de interfaz** con capturas automáticas en un navegador.

8.2 Pruebas mínimas requeridas

Tabla 37. *Pruebas mínimas del enunciado*

N.º	Prueba	Datos de entrada	Resultado esperado	Resultado obtenido
1	Registrar un paciente nuevo	CI nuevo, nombres, nacimiento, sexo, teléfono	El paciente queda guardado	✔ "Paciente registrado correctamente"; código y historia clínica generados
2	Paciente con CI duplicado	CI ya registrado	El sistema rechaza el duplicado	✔ "Ya existe un paciente registrado con ese CI"
3	Cita en horario disponible	Médico, fecha y hora libres	La cita queda registrada	✔ "Cita CM-0245 programada…"
4	Cita superpuesta, mismo médico	Mismo médico, fecha y hora	El sistema impide el conflicto	✔ "El horario ya no está disponible: el médico tiene otra cita"
5	Cita fuera del horario	Hora 15:00 de un médico que atiende 08–12	El sistema rechaza la cita	✔ "La hora elegida está fuera del horario de atención"
6	Cancelar una cita	Cita programada + motivo	Cambia a cancelada y conserva el historial	✔ Estado Cancelada, horario libre e historial con motivo
7	Historial de un paciente	Ficha del paciente	Aparecen sus citas registradas	✔ Citas pasadas, futuras y canceladas
8	Citas de un médico por fecha	Médico + fecha (+ estado)	Se muestran los resultados correctos	✔ Agenda filtrada por médico, fecha y estado
9	Página interna sin sesión	GET /pacientes sin sesión	El acceso es denegado	✔ Redirige a /login (302)
10	Datos inválidos	CI con letras, fecha futura, sexo X, correo inválido	El servidor valida y rechaza	✔ Rechazado por el servidor aunque se omita el JavaScript



Figura 119. *Prueba de credenciales incorrectas*

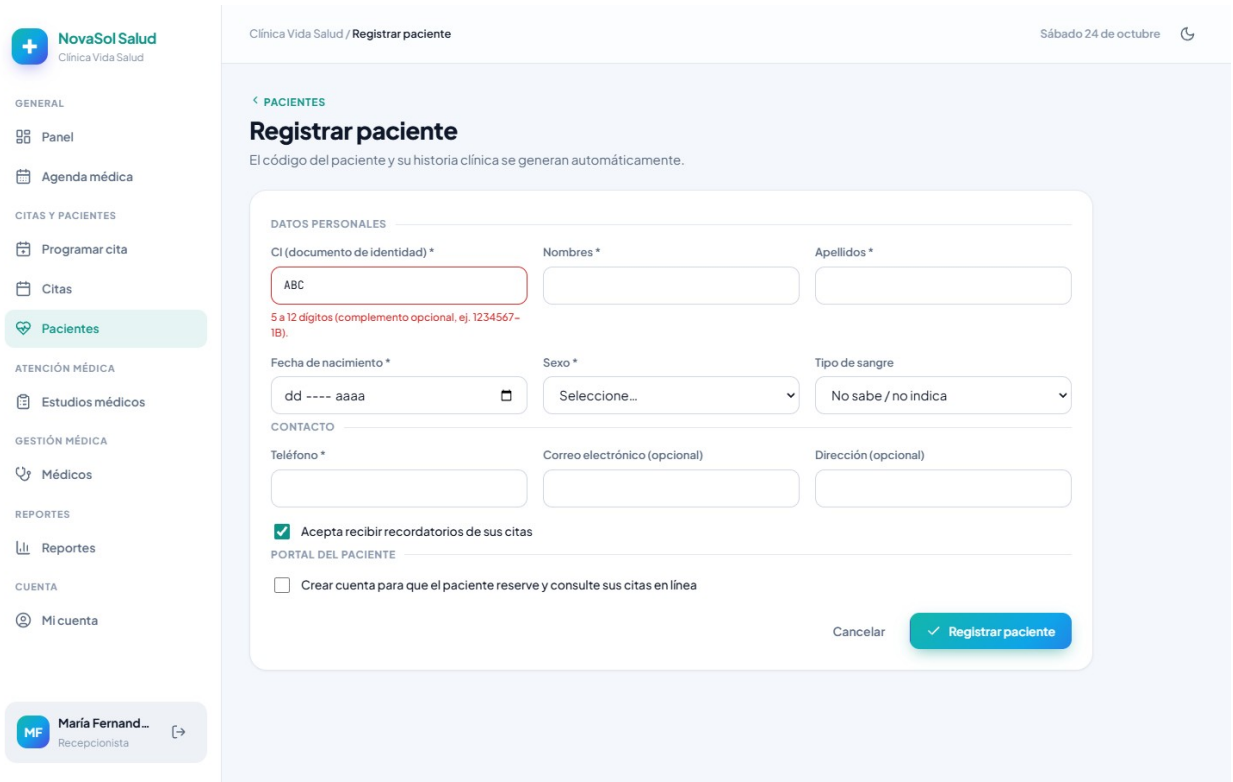


Figura 120. *Prueba 10: validación de datos inválidos*

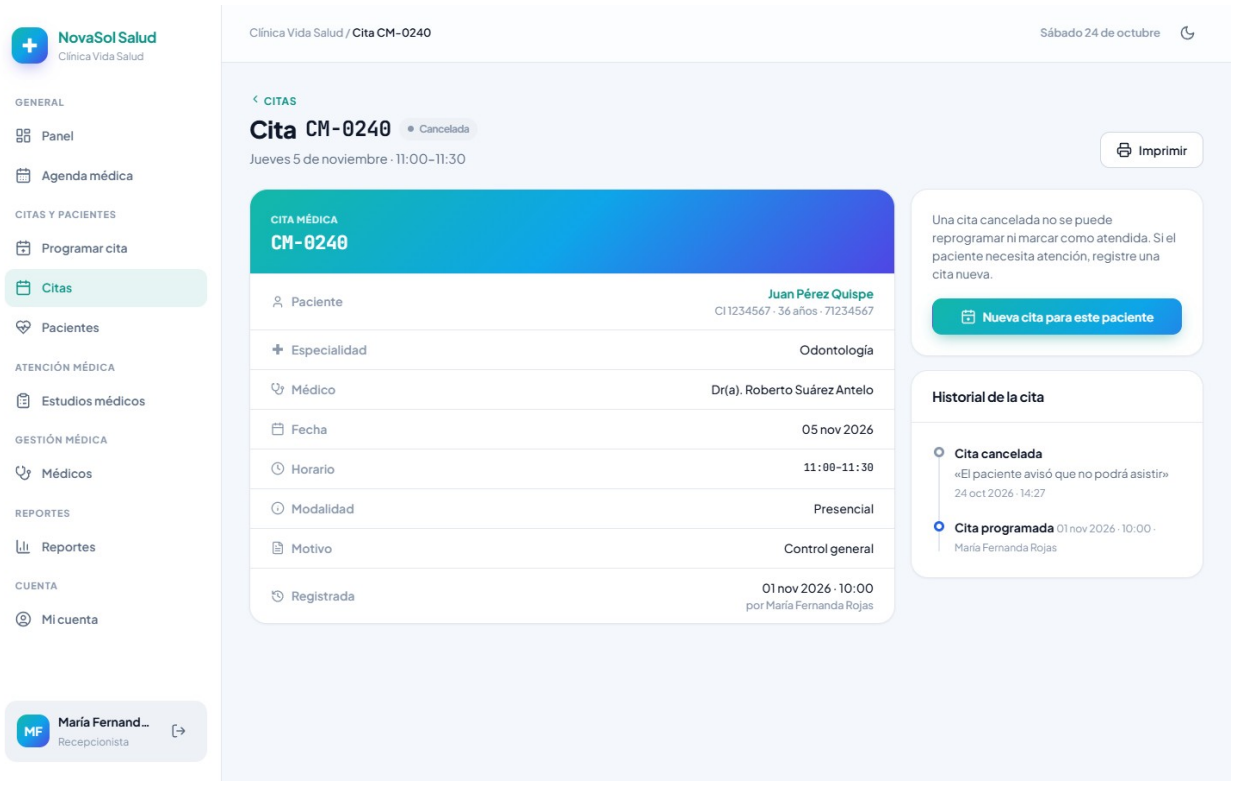


Figura 121. *Prueba 6: cita cancelada con historial conservado*

8.3 Pruebas de integridad en la base de datos

Se insertaron datos directamente con SQL, sin pasar por PHP, para comprobar que la base de datos también protege las reglas de negocio:

Tabla 38. *Pruebas de integridad*

Caso	Resultado
Cita cruzada para el mismo médico	✔ Rechazada por la restricción EXCLUDE cita_sin_cruce_medico
Cita en fecha pasada	✔ "No se pueden registrar citas en fechas u horas pasadas" (trigger)
Cita fuera de horario	✔ "El horario elegido está fuera de la atención del médico" (trigger)
Duración distinta a la configurada	✔ "La cita debe durar 30 minutos" (trigger)
Transición de CANCELADA a PROGRAMADA	✔ "No se puede pasar una cita de CANCELADA a PROGRAMADA"
Eliminar un paciente con citas	✔ Rechazado por la clave foránea (RN11)
CI duplicado	✔ Rechazado por UNIQUE
Especialidad "pediatria" ya existente como "Pediatría"	✔ Rechazada por el índice único normalizado

Caso	Resultado
Horario cruzado del mismo médico	✔ Rechazado por la restricción EXCLUDE horario_sin_cruce

8.4 Pruebas de seguridad y permisos

Tabla 39. *Pruebas de seguridad*

Caso	Resultado
Recepcionista abre /usuarios, /roles, /configuracion, /bitacora	✔ 403 Acceso denegado
Médico abre /citas, /pacientes, /reportes	✔ 403 Acceso denegado
Médico intenta atender una cita de otro médico	✔ 403 Acceso denegado
Paciente abre /panel, /citas, /pacientes, /api/pacientes	✔ 403 Acceso denegado
Paciente intenta cancelar una cita de otro paciente	✔ 404 No encontrada
Formulario enviado sin token CSRF	✔ 419 Sesión expirada
Cuenta desactivada intenta ingresar	✔ "Su cuenta está desactivada"
Texto con codificación inválida	✔ 400 Solicitud inválida
Recuperar contraseña con un correo no registrado	✔ Misma respuesta que uno registrado (no revela cuentas)
Recuperar contraseña: código incorrecto	✔ "Código incorrecto. Le quedan 4 intento(s)"
Recuperar contraseña: código correcto	✔ Contraseña restablecida, sesiones cerradas e ingreso con la nueva clave
Reutilizar un código ya usado	✔ "El código no es válido o ya venció"

8.5 Pruebas del flujo completo

1. La recepcionista programa una cita, la confirma y la reprograma: el historial muestra los tres eventos.
2. Se intenta registrar la asistencia de una cita futura: el sistema lo impide.
3. El médico atiende una cita del día, registra el diagnóstico, un tratamiento y un estudio: la cita pasa a Atendida.
4. La recepción registra el resultado del estudio: el paciente ve el aviso y el resultado en su portal.
5. Un paciente se registra solo, reserva una cita desde el portal y la cancela con motivo.
6. El administrador genera reportes por médico y exporta el resultado a CSV.
7. Un paciente olvida su contraseña, recibe el código en su correo, la restablece e ingresa con la nueva.

CONCLUSIONES

- Se desarrolló el sistema web NovaSol Salud con **PHP 8** y **PostgreSQL**, conectados mediante **PDO** con consultas parametrizadas, que cumple los trece requerimientos funcionales (RF01 a RF13) y las catorce reglas de negocio (RN01 a RN14) del enunciado.
- La aplicación del **Proceso Unificado** permitió partir de los casos de uso y mantener la trazabilidad hasta el código: cada caso de uso tiene su descripción, su diagrama de comunicación y su diagrama de secuencia, y se corresponde con controladores, servicios y modelos concretos.
- El problema central de la clínica, la **duplicación de citas**, se resolvió con tres capas de protección: el sistema solo ofrece horarios libres, el servidor vuelve a verificar la disponibilidad dentro de una transacción que bloquea al médico, y PostgreSQL impide los cruces con una restricción de exclusión.
- Los estados de la cita y sus transiciones quedaron controlados en la aplicación y en la base de datos, conservando el historial de cancelaciones, reprogramaciones e inasistencias.
- La seguridad se abordó desde el diseño: contraseñas con `password_hash()`, sesiones seguras, protección CSRF y XSS, permisos por rol verificados en el servidor y bitácora de auditoría.
- La publicación en la nube (Vercel y Supabase) permite usar el sistema desde cualquier navegador, y el portal del paciente amplía su alcance a las personas que se atienden en la clínica.
- Las pruebas mínimas y las adicionales de integridad y seguridad se superaron satisfactoriamente.

RECOMENDACIONES

- Integrar el envío real de recordatorios por correo electrónico o WhatsApp mediante una tarea programada.
- Almacenar los archivos de los resultados de estudios en un servicio de almacenamiento (por ejemplo, Supabase Storage) en lugar de enlaces externos.
- Agregar un calendario semanal y mensual a la agenda médica y una lista de espera cuando no haya horarios disponibles.
- Generar los reportes en PDF directamente en el servidor (por ejemplo, con la librería Dompdf).
- Implementar la autenticación en dos pasos para el personal.
- Realizar copias de seguridad periódicas de la base de datos y monitorear los registros de errores.
- Capacitar al personal de recepción y a los médicos antes de la puesta en marcha y recoger sus observaciones para el siguiente ciclo.

BIBLIOGRAFÍA

Jacobson, I., Booch, G. y Rumbaugh, J. (2000). *El Proceso Unificado de Desarrollo de Software*. Addison Wesley.

Booch, G., Rumbaugh, J. y Jacobson, I. (2006). *El Lenguaje Unificado de Modelado. Guía del usuario* (2.ª ed.). Addison Wesley.

Sommerville, I. (2011). *Ingeniería de Software* (9.ª ed.). Pearson Educación.

Pressman, R. S. y Maxim, B. R. (2021). *Ingeniería del software: un enfoque práctico* (9.ª ed.). McGraw-Hill.

Larman, C. (2003). *UML y Patrones* (2.ª ed.). Prentice Hall.

The PHP Group. (2026). *Manual de PHP*. https://www.php.net/manual/es/

The PostgreSQL Global Development Group. (2026). *PostgreSQL 17 Documentation*. https://www.postgresql.org/docs/17/

OWASP Foundation. (2021). *OWASP Top 10*. https://owasp.org/Top10/

Vercel Inc. (2026). *Vercel Documentation*. https://vercel.com/docs

Supabase Inc. (2026). *Supabase Documentation*. https://supabase.com/docs

PlantUML. (2024). *PlantUML Language Reference Guide*. https://plantuml.com/es/

ANEXOS

Anexo A · Repositorio y sistema en línea

Sistema en línea	https://novasol-salud.vercel.app
Repositorio GitHub	https://github.com/Kato-001/novasol-salud
Base de datos	PostgreSQL 17 en Supabase (esquema clinica)
Documentación técnica	README.md del repositorio y carpeta Docs/



Figura 122. *Código QR del repositorio en GitHub*



Figura 123. *Código QR del sistema en línea*

Anexo B · Cuentas de acceso

Contraseña de todas las cuentas de demostración: **NovaSol Salud2026**. Se puede ingresar con el usuario o el correo; los pacientes también con su CI.

Tabla 40. *Cuentas de demostración*

Rol	Persona	Usuario	Correo
Administrador	Administrador General	admin	admin@vidasalud.com

Rol	Persona	Usuario	Correo
Recepcionista	María Fernanda Rojas	mrojas	recepcion@vidasalud.com
Recepcionista	Jorge Cuéllar	jcuellar	jcuellar@vidasalud.com
Médico	Dr. Carlos Mendoza (Medicina General, Cardiología)	cmendoza	cmendoza@vidasalud.com
Médico	Dra. Lucía Vargas (Pediatría)	lvargas	lvargas@vidasalud.com
Médico	Dr. Roberto Suárez (Odontología)	rsuarez	rsuarez@vidasalud.com
Médico	Dra. Patricia Molina (Ginecología, Medicina General)	pmolina	pmolina@vidasalud.com
Paciente	Juan Pérez Quispe (CI 1234567)	jperez	jperez@correo.com
Paciente	María López Rivero (CI 7654321)	mlopez	mlopez@correo.com

Anexo C · Instalación local

1. Clonar el repositorio: `git clone https://github.com/Kato-001/novasol-salud.git`.
2. Crear la base de datos en PostgreSQL y ejecutar en orden los scripts 01, 02, 03 y 04 de `Bakend/database`.
3. Copiar `.env.example` como `.env` y completar `DATABASE_URL`.
4. Ejecutar `php -S localhost:8000 router.php` o copiar la carpeta en el directorio web de XAMPP/Laragon.
5. Abrir http://localhost:8000 e ingresar con una de las cuentas del Anexo B.

Anexo D · Manual breve de uso

Tabla 41. *Manual breve de uso*

Tarea	Pasos
Programar una cita (recepción)	Programar cita → buscar paciente por CI → especialidad → médico → día → hora libre → Confirmar cita.
Reprogramar o cancelar	Citas → abrir la cita → Reprogramar (nueva fecha y motivo) o Cancelar cita (motivo).
Registrar asistencia	Abrir la cita del día → Marcar como atendida / El paciente no asistió.
Atender (médico)	Mi agenda → abrir la cita → Atender y registrar consulta → diagnóstico, tratamiento, estudios → Guardar.
Registrar resultado de estudio	Estudios médicos → Registrar resultado → estado "Con resultado" → texto del resultado → Guardar.
Reservar (paciente)	Portal → Reservar cita → especialidad → médico → día → hora → Confirmar reserva.

Tarea	Pasos
Recuperar la contraseña	Login → ¿Olvidaste tu contraseña? → correo → revisar el correo → escribir el código de 6 dígitos y la nueva contraseña.
Generar un reporte	Reportes → elegir pestaña → rango de fechas y filtros → Generar → Exportar CSV o Imprimir / PDF.